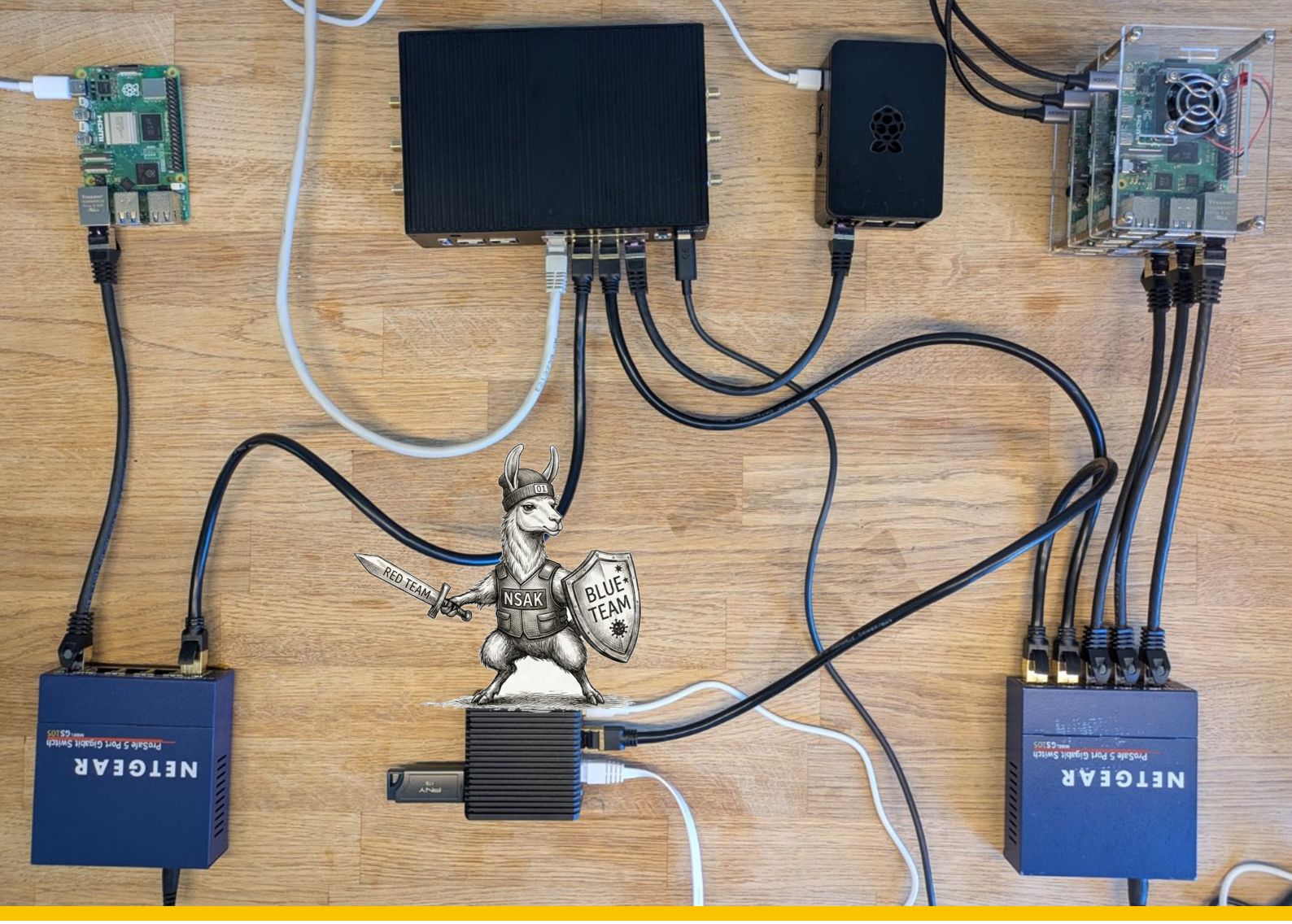


Bachelor Thesis

Nsak as Framework for Scenario Based Network Security

Course of study	Bachelor of Science in Computer Science
Author	Frank Gauss (gausf1) and Lukas von Allmen (vonal3)
Advisor	Wenger Hansjürg

Version 1.0 of June 4, 2026

Abstract

Red and blue team exercises are established cybersecurity practices, yet they remain largely inaccessible to small- and medium-sized organizations due to the expertise and recurring effort these practices demand. We present an approach that combines Large Language Models (LLMs) with existing network security tools to reduce knowledge barriers and time investment. Concretely, we integrated an Artificial Intelligence (AI) agent with tool calling capabilities into the Network Swiss Army Knife (NSAK) framework — a network security scenario automation framework established in our preceding work — and developed an agentic network reconnaissance scenario. We evaluated its effectiveness across models of varying size, benchmarking it against a scripted scenario in a reproducible network environment. The results show that even smaller models are capable of autonomously conducting standard red team activities. However, the failure rate remains considerable, though most failures are detectable, enabling retry logic. Furthermore, the AI-driven approach demonstrates a clear advantage over scripted counterparts. The agent assesses results, generates reports with prioritized findings, and classifies steps to address vulnerabilities. This work highlights the emerging capabilities of LLMs, making cybersecurity use cases more accessible and cost-effective.

Index Terms: Red and blue team exercises, Network Security Automation Framework, Automated Reconnaissance, Large Language Models, AI-Driven Cybersecurity

Contents

1. Introduction

According to the World Economic Forum's Global Risks Report 2026, cyber insecurity is listed among the top ten global risks and is projected to rise significantly from position 30 to position 5 in the long term, highlighting the growing impact of AI technologies [1]. This global perspective is further underlined by the CrowdStrike Global Threat Report which highlights a significant rise in AI-enabled adversaries [2]. These developments, combined with a shortage of security experts and the rising complexity of systems and business models [3], emphasize the increasing need for automated tools and frameworks capable of identifying and mitigating emerging cyber threats.

To address some of these issues, we established NSAK — a modular, open-source network security scenario automation framework — in a predecessor project 3.1. The framework aims to simplify the creation of automated security testing and adversary emulation scenarios in controlled network environments, by providing reusable building blocks.

While our prior work demonstrated that it is feasible to automate red and blue team activities, it also revealed the limitations of this approach. Manually engineered scenarios offer modularity and reproducibility, but lack adaptability to changing conditions and the development still requires deep cybersecurity expertise. By contrast, LLMs are able to reason over the current context and dynamically adjust their behavior depending on the environment they operate in.

The central hypothesis is that the integration of agentic AI into the framework can improve adaptability, flexibility and the degree of automation compared to static scenario implementations. Simultaneously, it may reduce the operator's required depth of expertise to interpret the results and plan follow-up steps in an attack or defense sequence.

This thesis approaches the following research questions:

- **RQ1:** How does an agentic AI scenario perform compared to a static reference scenario?
- **RQ2:** To what extent can agentic AI support operators during interactive network reconnaissance?
- **RQ3:** To what extent is the implementation suitable for deployment in real-world environments?

To address these questions, an AI agent based on LangChain is integrated into the framework and provided with capabilities for autonomous system interaction and operator support. With these features, single- and multi-agent reconnaissance scenario variants are developed, which can be run interactively and autonomously, using models of various sizes.

The evaluation is structured around three perspectives corresponding to the research questions:

Autonomous Scenario Comparison (RQ1) — A static reference scenario chaining host discovery, port scanning, and service enumeration serves as a baseline for comparison. To reduce the number of variables in the evaluation, a reproducible test environment is specified and built with Containerlab.

Interactive Scenario Execution (RQ2) — Besides the direct comparison we also want to investigate how agentic AI can interactively support an operator during network reconnaissance activities.

Real-World Usability (RQ3) — Real-world usability is evaluated by testing against a physical setup of the evaluation environment and in the BFH network security lab.

The thesis is structured as follows: Chapter 2 reviews related work and analyzes how selected security frameworks integrate AI into their products. Chapter 3 introduces the fundamental concepts and outlines the key design decisions that form the basis of this work. Chapter 4 describes the research methodology, including the approach used for evaluation and data collection. Chapter 5 presents the implementation of the proposed solution, followed by Chapter 6, which evaluates its effectiveness in virtual and physical test environments. Chapter 7 discusses the results and their implications, while Chapter 8 concludes the thesis and highlights potential directions for future work.

2. Related Research

2.1. Framework Comparison

Previous research, such as 'SOK: A Survey of Open-Source Threat Emulators', compares multiple types of threat emulation frameworks using both quantitative and qualitative metrics [4]. Among the evaluated frameworks, Atomic Red Team, Mitre Caldera, and Metasploit received high scores in the categories "environments and operator", "environment configuration" and "scenario execution".

In the following section, the selected frameworks are briefly introduced, and the unique selling point of the NSAK is highlighted. As a point of reference, the cybersecurity community increasingly relies on the concepts of MITRE ATT&CK, which has established itself as a standard framework in the field since its introduction in 2014 [5, 6].

The MITRE ATT&CK framework is a globally accessible knowledge base of adversary tactics and techniques based on real-world observations [5]. The knowledge base is structured into Tactics, Techniques, and Procedures, which are organized into 14 attack behavior groups. The Tactics, Techniques, and Procedures is getting continuously updated and extended, which helps to defend against the capabilities of threat actors.

The evaluation of the proposed frameworks is based on information obtained from official documentation [7–9] and the findings of Zilberman et al. [4].

Characteristic	Metasploit Framework	Atomic Red Team	MITRE Caldera
Primary Purpose	Exploitation and penetration testing framework	Security testing through small atomic ATT&CK tests	Automated adversary emulation platform
Relation to MITRE ATT&CK	Partial mapping via modules	Direct mapping to ATT&CK techniques	Built around ATT&CK adversary emulation
Automation Level	Medium (scripts and modules)	Low (manual execution of tests)	High (automated attack chains)
Main Functionality	Exploit development, payload delivery, post-exploitation	Execute small ATT&CK technique tests	Simulate adversary campaigns across hosts
Architecture	Modular framework with exploits, payloads, and auxiliary modules	Collection of scripts/tests organized by ATT&CK techniques	Client-server platform
Typical Use Case	Penetration testing and vulnerability validation	Detection validation and security control testing	Red team automation and purple team exercises
Difficulty Level	Medium–High	Low–Medium	Medium–High

Table 2.1.: Comparison of Metasploit, Atomic Red Team, and MITRE Caldera

As shown in table 2.1, the key characteristics of the frameworks differ greatly. **Atomic Red Team** provides libraries of attack scripts that implement individual Tactics, Techniques, and Procedures of MITRE ATT&CK. Each attack can be executed independently or chained together into a larger attack scenario. **Mitre Caldera** is a complex, agent based adversary simulation tool, whereas **Metasploit** is a framework that provides exploits, payloads and auxiliary modules. Further details of the frameworks are presented in the following sections. 2.1.12.1.22.1.3

2.1.1. Atomic Red Team

The Atomic Red Team repository provides documentation and configuration files for each technique. For every TTP, a YAML Ain't Markup Language (YAML) configuration file defines the execution parameters, while an enhanced Markdown file contains a human-readable description of the attack. This documentation includes a short description of the technique, execution instructions, and additional contextual information [8].

The framework can be executed directly from the command line and provides features such as logging, detailed error messages, and automatic prerequisite checks. If a technique cannot be executed, for example due to operating system constraints, the framework reports the issue and can optionally attempt to install the required prerequisites [8].

Zilberman et al. recommends deeper cybersecurity knowledge to use Atomic Red Team framework effectively [4]. The Open Source mentality, expandability and easily usable scripts can be a great asset for an independent framework such as the NSAK.

2.1.2. Caldera

Mitre Caldera is also built on the MITRE ATT&CK knowledge base and functions as a command-and-control center. A central server controls agents running on the target system, executes attacks called abilities, and sends the operation results back to the server. Every Ability is a single attack step built on the Tactics, Techniques, and Procedures. The framework ships with a graphical user interface and is primarily used for red team attack simulation, purple teaming, and detection engineering. The additional features, like a training plugin for educational purposes, include facts that can be configured as dynamic safe values, which can be stored and parsed in later steps [7].

With its provided interface and functionality, Caldera is most suitable as a threat emulator, offering a wide range of built-in features, including lateral movement and Command and Control (CNC) communication [4].

2.1.3. Metasploit

Metasploit is a widely used penetration testing framework for testing or attacking systems and exploiting known vulnerabilities. Metasploit follows a sequence of steps to identify a vulnerability in a target system, select a vulnerability for the attack, select an exploit that uses this vulnerability, and deliver a payload that runs on the target system. The configuration via the Command Line Interface (CLI) requires knowledge of the target network and system and requires partial configuration [9, 10].

According to Zilberman et al., an operator requires extensive cybersecurity knowledge to effectively use the framework. To support the operator, the Graphical User Interface (GUI) “Armitage”, which is not further evaluated in this work, can be used [4].

2.1.4. NSAK

The NSAK framework, as an independent universal Network Swiss Army Knife, was implemented with a strong focus on networking and hardware devices in mind. This is underlined by the provided resource types, such as the network environment 3.1.2, device 3.1.1 and the reference hardware evaluated in the predecessor project 3.1. The scenarios 3.1.4 and drills 3.1.3 are implemented as simple Python scripts, with a few conventions instead of extensive boilerplate code or complicated concepts. Even though the framework can not compete in terms of maturity and library size, the modular design allows to incorporate the tools of other frameworks as drills if we see fit. An example could be, to use an AI-Enhanced Network Discovery Scenario, which provides the necessary information to configure a Metasploit attack that leads to an attack operation in Caldera or an Atomic Red Team detection script. Further, the framework provides container based isolation, scenario simulation in virtual environments and flexible positioning in physical labs. In summary NSAK can be seen as a harness for network focused cybersecurity attack and defense scenarios, which should help reduce the operational costs of red and blue teams [11].

2.2. AI Research

2.2.1. AI and Language Models

Language Model (LM) describes the technique for advancing language intelligence in machines and its development history can be divided into several stages. The first stage, Statistical Language Model (SLM) emerged around the 1990s, and the basic idea is to assume, that such a model can predict the next word. But the fact that an exponential number of transition probabilities are needed, caused a lack of accuracy and led limited success.

To overcome these limitations, Neural Language Model (NLM) was introduced to learn a distributed representation of words and capture more complex patterns, leading to a major breakthrough. Building on this, the advancement of pre-trained language models Pre-trained Language model (PLM), which are first trained on large datasets and then fine-tuned for specific tasks, represented another significant development.

Building on this development, LLMs further increases the scale of PLMs by expanding parameter count and computational power [12].

In large language models, parameters are the trainable numerical values that define how the model processes input and generates output. During training, these parameters are continually adjusted to enable the model to learn patterns and language

context [13]. The models most of us use today, such as ChatGPT, Claude and Gemini are pre-trained LLMs used for language inference.

These advances led to MCP, a protocol that allows tools and services to expose an interface specifically for AI. Further, the introduction of AI-agents is enabling a new level of autonomy for achieving concrete goals with LLMs. The combination of these technologies is now resulting in systems, where LLMs empower autonomous agents capable of decision-making to execute complex tasks.

2.2.2. MCP

Anthropic, a US software company focused on AI research, introduced the protocol in 2024. The MCP was introduced as an open source standard that enables AI assistants to connect to external systems, including databases, business tools, and development environments [14]. Dynamic tool discovery and schema negotiation is helping clients and agents to access these functions in a standardized manner and allows autonomous interaction. Additionally, MCP allows information to flow in both directions, so the model can send requests, and tools can send updates or events.

This architectural approach shifts the AI integration from static toward an interoperable ecosystem of AI-enabled services. On the other hand, the flexible design and open source approach of MCP unlocks multiple novel and systematic security risks that are in need of enhanced security and governance [15].

2.2.3. Agentic-AI

Agentic Artificial Intelligence (Agentic-AI) can be understood as an advanced machine intelligence that is capable of perceiving, reasoning, and acting with a certain level of autonomy. Traditional systems and agents are more limited to executing predefined commands, whereas Agentic-AI is not restricted in this way. Due to their ability to adapt in real time, they can continuously refine their knowledge, techniques, and data. The ability to access multiple inputs, from natural language processing to sensor data, allows Artificial Intelligence Agent (AI Agent)s to develop a good understanding of their environment [16].

Multi Agent System (MAS) defines a way in which multiple specialist agent coordinate and cooperate with each other to solve a task, that would not be solvable to one agent alone.

Therefore, the agents need a form of interaction that allows them to act reactively, often through a shared mental model, which helps create a common understanding of the complex tasks to be solved. To implement a MAS efficiently, modern design

principles such as encapsulation and modularity are important, which are helping to balance resource usage of the system and LLM [17]

Huang et al. provide evidence that Agentic-AI and MAS are capable of solving complex tasks across various sectors, including security and banking. However, these systems also introduce new potential vulnerabilities, both within AI Agent systems themselves and through their interactions with external environments.

The identified vulnerabilities can be categorized into two types: accidental and deliberate. Accidental vulnerabilities include software bugs, logical errors introduced by agents, hardware malfunctions, and poor data quality. Deliberate attacks, on the other hand, encompass adversarial attacks targeting LLMs, data poisoning, and model theft.

To mitigate these risks, a comprehensive framework consisting of governance mechanisms, proactive monitoring, regulatory compliance, and rigorous testing is required to ensure the secure and reliable operation of agentic systems [18].

Conclusion: Tool Calling & Agentic AI

In the cybersecurity podcast Risky Business, it is mentioned that MCP is “dead”. The open-source strategy of Anthropic (2.2.2) has led to the development of multiple MCP implementations that were not designed with a strong focus on security, but rather on rapid adoption of a novel technique.

This provocative claim is based on the observation that, in order to handle the excessive MCP tooling, AI Agents may instead rely on direct access to a root shell, as it provides a more efficient way of interaction [19]. Recently, other concepts for calling tools were established, such as . These observations are directly relevant to this thesis, because the AI based scenarios will heavily relly on tool usage, and we don’t want to restrict the agent to MCP based tools. Consequently, from now on we will focus on the more general concept of **Tool Calling** instead.

Another issue with tools is that they need to be described to the LLM, which contributes to the input tokens and thus the context size. Depending on the model, its size and the available hardware resources, this can lead to descriptions filling a significant portion of the context window, degrading the overall performance and quality of the responses. The architectural decision is not to remove tools completely, but instead to expose a minimum set and an overview of available capabilities. An LLM can now request a tool description from the agent, after it decided that it needs this capability — this is called dynamic tool selection.

Evaluated Frameworks and Tool Calling

Caldera uses a MCP plugin as a bridge between the model and the Representational State Transfer Application Programming Interface (REST API), allowing the agent to plan and issue commands in a manner similar to a human operator.

Caldera introduces three main research areas. First, a planner determines the order and selection of abilities and can dynamically construct new adversaries based on user requirements through prompt-based interaction. Second, an adversary factory model is used to generate adversaries tailored to the operator's specific use cases. Last, a forward-deployed agent is enhanced with Retrieval Augmented Generation (RAG) for Cyber Threat Intelligence (CTI), leveraging Structured Threat Information Expression (STIX)-based data to create reproducible and testable adversaries [20].

The MCP-Server, provided by **Metasploit**, enables a LLM to interact with the Metasploit Framework dynamically. The Interface covers module information (a Metasploit module is a known exploit), exploitation workflows, payload generation, session management and handler management to help manage an attack. The documentation includes a security warning that emphasizes the importance of reviewing every prompt, using a test environment, and being aware of post-exploitation commands. **Metasploit** is a powerful exploitation framework, and the simplicity that AI could bring can also harm [21].

The **Atomic Red Team** MCP Server provides an interface that allows AI systems to access and use tests from Atomic Red Team. It enables searching, validating, and optionally executing attack techniques — e.g., based on MITRE ATT&CK [22].

Comparison Outcome & Framework Choice

The framework comparison highlights the difference between NSAK and the established frameworks. Each compared framework has its own focus and is able to perform a specific task in a complex operation. For this thesis we decided to go forward with the NSAK framework for the following reasons.

Network Focus Because this thesis focuses on the evaluation of AI based red and blue team activities in virtual and physical network environments, we need a framework which has a strong focus on networking and is able to run on small embedded devices.

Harness Concept In the context of AI, the term "harness" is used to describe the software layer surrounding an agent that provides guardrails, tooling, configuration, logging, monitoring, and other supporting functionality required for secure

and reliable operation. Because NSAK already acts as a harness for network focused cybersecurity scenarios 2.1.4, we can implement an AI agent and reuse the already implemented tooling.

Familiarity We are already familiar with the NSAK framework, the Python based library and their development workflow, which allows us to efficiently extend the core and develop new scenarios and drills for the library.

Interoperability If needed we can leverage the strengths of the other frameworks by using their Application Programming Interfaces (APIs) or building and adapter, to call specific functionalities in drills and scenarios.

The following chapter therefore introduces the core concepts of NSAK, inherited from the predecessor project and extended in this thesis, which are forming the basis for the implementation presented later.

3. Concepts

This chapter introduces the core concepts that underpin the implementation and evaluation presented in this thesis. It begins with the core terminology of the NSAK framework inherited from the predecessor project. It then describes the layered configuration management, followed by the test environment setup. Finally, it completes the AI-related concepts relevant for this work, without repeating the terms already introduced in section AI Research.

3.1. Predecessor Project: Project 2

This section summarizes the predecessor project that led to the bachelor's thesis. The NSAK is a cybersecurity framework that operates with the following resources and concepts: *Devices*, which are physical or virtual machines capable of running the NSAK. The security operation is executed in a network environment and can involve one or more attacks or defense scenarios. The smallest unit in the framework is a drill, which can be orchestrated in a scenario and chained with other drills.

While the modularity and proof of concept were confirmed, several aspects like enhanced configuration management, GUI or a REST API are mentioned for future work.

Some of the goals of this thesis directly address these open points and are defined in the section A.

The NSAK framework is built around four resource types that were originally defined in "Project 2" [11]. Each resource type has a distinct role in the framework and is stored as a YAML file in the resource library.

3.1.1. Device

A **Device** resource describes a physical or virtual machine that is capable of running the NSAK framework. It serves as the execution host for scenarios and drills and is typically deployed at a strategic network position to give it the access required for the operations it performs.

A device exposes its network configuration to the framework by declaring its interfaces and their Internet Protocol addresses in its resource YAML. As an alternative, a special device with the id “Unknown” was added, which auto discovers the hosts network configuration. This allows scenarios and drills to resolve interface names and target addresses without hardcoding network details, making the same scenario portable across different devices and network environments.

3.1.2. Environment

An **Environment** represents a specific network topology, including its infrastructure components, hosts, and services [11]. It describes the target network context in which a scenario is intended to operate. For example, a simple client–server setup over a switch, a Wireless LAN access point with connected clients, or a segmented business network.

Environments are referenced by scenarios to make explicit which network context they are designed for, helping operators select and validate the right configuration before executing an operation.

3.1.3. Drill

A **Drill** is a self-contained, reusable unit of execution with a specific, well-defined goal [11]. Its goal can be an active or passive attack step, network discovery, traffic capture, a configuration change, or any other discrete action relevant to a red or blue team activity.

Examples of drills include Address Resolution Protocol spoofing, host discovery via Address Resolution Protocol scan, transparent Transmission Control Protocol proxying, traffic capture with T-Shark, and network configuration tasks such as enabling IP forwarding or Network Address Translation (NAT).

Each drill declares the system and Python packages it requires as well as a typed interface that lists its input arguments and return type. The return value of one drill can be consumed as the input of a subsequent drill within a scenario, enabling data to flow through a chain of execution steps.

3.1.4. Scenario

A **Scenario** describes a concrete red or blue team use case and orchestrates a sequence of drills targeted at one or more environments [11]. Where a drill is a single technical reusable and modular building block, a scenario is an implementation of a specific attack or defense operation.

Like drills, scenarios declare a typed interface with input arguments, allowing operators to parameterise a scenario at execution time via the CLI without modifying the scenario definition itself. This separation keeps scenarios reusable across different network configurations while still allowing runtime flexibility.

3.2. Configuration Management

The NSAK framework distinguishes three configuration layers, each with a different scope and lifetime. Keeping them separate avoids coupling concerns that change at different rates and makes it easier to reason about where a given piece of configuration comes from.

3.2.1. Static Configuration

The static configuration layer involves values which are specific for a deployment and are very unlikely to change after the initial provision of the NSAK device. It mainly establishes the filesystem layout that all other parts of the framework depend on. For example, where the resource library is located, where runtime data is written, and what the base directory of the installation is.

3.2.2. Runtime Configuration

The runtime configuration layer holds mutable state, which might be changed by an operator via the CLI. It is stored as a YAML file, which is currently loaded at the start of every invocation. If no configuration file exists yet, a default is created automatically on first access, so no explicit initialization step is required from the user.

3.2.3. Resource Configuration

The resource configuration layer is provided by the resource library rather than the framework core. Each resource declares its own configuration inside a YAML file stored alongside its implementation. The framework reads these files to discover and load resources at runtime.

Typed Interface Arguments

Drills and scenarios declare a typed interface in their resource configuration YAML file, that lists the arguments with an explicit type annotation and an optional de-

fault value. The framework reads these annotations at runtime and automatically generates the correct CLI options for each resource. This means that adding a new argument to a drill or scenario immediately exposes it as a typed CLI option without requiring any changes to the framework or the CLI code.

Mergeable YAML Structure

All resource YAML files follow a common structure in which resources are nested under a resource-type key (e.g. drills:, scenarios:). This structure ensures that YAML files from different parts of the library could be merged into a single in-memory data structure without key conflicts, which is a prerequisite for supporting multiple library paths and composite library configurations in the future.

3.2.4. Reproducible Test Environment

To ensure a controlled and safe evaluation of the implementation, a simulated test environment has to be established. This approach prevents unintended interference with production systems and ensures reproducibility during the development and testing phases. For this exact purpose the “Environment”3.1.2 resource type was implemented in the predecessor project 3.1. Concretely, a simplified network topology comprising a client (Alice) and a server (Bob) is used to emulate and analyze a Man In The Mittle (MITM) scenario.

While Docker and Podman provide a general-purpose containerization platform, they abstract away most of the lower levels of the network stack and thus lack the support for modeling network topologies. In contrast, Containerlab builds upon Docker Compose and optimize its functionalities for networking purposes.

The primary advantages of Containerlab are:

- It’s ability to create a reproducible containerized network topology based on a simple configuration file
- Support multiple interfaces nodes which closely resemble real-world network devices (in comparison Docker typically operates with a single network interface).
- The rapid creation, extension and teardown of complete network environments, ensure that each experiment starts from a clean and controlled state.

In summary, while OCI containers serves as the underlying container engine, Containerlab provides a higher-level network simulation.

3.2.5. Additional AI Related Concepts

This section will not repeat the concepts already introduced in section AI Research.

LangChain

LangChain is an open-source Python framework that provides abstractions for building applications powered by LLMs [23]. Building upon these abstractions, LangGraph provides a low-level orchestration framework for implementing stateful and long-running agent workflows. Unlike LangChain's higher-level agent interfaces, LangGraph represents applications as graphs consisting of nodes and edges, enabling explicit control over execution flow, state management, persistence, and multi-agent coordination [24]. On top of LangGraph, LangChain provides higher-level agent abstractions, standardized interfaces for models and tools, and pre-built agent architectures. At an even higher abstraction level, Deep Agents provide a pre-built agent harness built on top of LangGraph. In addition to the standard agent loop, Deep Agents include capabilities such as task planning, context management through file-system based memory, sub-agent delegation, and long-term memory, making them suitable for complex autonomous workflows [25]. In this thesis, LangGraph is used as the primary framework for implementing both single- and multi-agent reconnaissance scenarios.

Structured Output

By default, LLMs produce free-form text responses. In contrast, structured output allows to generate responses conforming to a predefined schema, such as a JavaScript Object Notation (JSON) object with typed fields. This is typically enforced through grammar-based sampling or constrained decoding, which restricts the set of valid tokens at each generation step to those that keep the output consistent with the target schema []. While most commercial inference APIs support the described *native* implementation, it is also possible achieve the same functionality on the agent side with tool calling. In LangChain both methods are supported, which allows to use setups without native support for structured output [26]. The AI based scenarios are leveraging this feature to return Python data structures, which simplifies the comparison and evaluation of the results.

Prompt Engineering

This term describes the practice of designing and refining the input provided to an LLM to improve the quality of the output. This includes structuring instructions,

providing examples, and defining output constraints. In agentic systems, the system prompt is the primary mechanism through which an agent's behavior, scope, and reasoning style are defined. Even tough we did not set our focus on Prompt Engineering, we implicitly used it to improve and fine tune the agents behavior during the implementation and evaluation phase of the thesis.

Reasoning

In the context of LLMs, reasoning refers to the model's ability to decompose a problem into intermediate steps and derive a conclusion from them. This is typically achieved through prompting strategies such as chain-of-thought, where the model is instructed to think step by step before producing a final answer. LangChain leverages reasoning by default for agents with tool calling support, which is relevant during the implementation and evaluation phase of the thesis.

RAG

Retrieval Augmented Generation is a technique that extends an LLM by retrieving relevant documents or knowledge at inference time and including them in the context window alongside the user prompt. This allows the model to ground its responses in external, up-to-date, or domain-specific information without requiring retraining. We did not implement a RAG system in the AI based scenarios, because the evaluation is focused on the pure tool calling capabilities of the current models.

The concepts introduced in this chapter form the foundation for the decisions made throughout the rest of the thesis. The following chapter defines the methodology, specifying the evaluation setup, scoring criteria, and test environments used to assess the implementation presented in Chapter 5.

4. Methodology

This chapter defines the evaluation strategy used to assess the agentic AI integration across the three research questions. The evaluation is structured around a controlled benchmark in a reproducible virtual environment and a real-world assessment in the BFH network security lab. A static reference scenario serves as the baseline against which all AI-driven configurations are compared.

4.1. Evaluation Setup

Three models are benchmarked across 10 runs in three environments (*containerlab*, *physical lab*, and *BFH Cyber Lab*): a frontier model (Claude Opus 4.7), an institutionally hosted open-weight model (gpt-oss:120b), and a locally-run open-source model (qwen3:30b via Ollama).

4.1.1. Criteria

The evaluation criteria are divided into quantitative and qualitative categories. Quantitative criteria are collected automatically by the benchmarking suite introduced in Section 5.5. Qualitative criteria require manual assessment and are scored against the scale defined in Table 4.2.

Qualitative criteria:

- **Correctness and Task completion/completeness:** Are the scenarios completed fully and are the results correct?
- **Consistency network discovery results:** How much does the output vary across $n = 10$ runs with an identical prompt, per model and environment?

Quantitative criteria:

- **Speed:** Time from scenario start to complete output of all discovered hosts and services
- **Token usage: AI-specific** Number of tokens consumed per run (AI Reconnaissance only)

To fix most of the influencing factors we defined a system prompt and a scenario prompt to set the starting line for all models equally. With 3 models $\times$ 3 environments and 1 implemented static scenario $\times$ $n = 10$ runs, this yields 120 measurements for Reconnaissance.

4.1.2. Environments

Three environments are used across the evaluation, each representing a different level of control, reproducibility and realism.

1. Virtual environment (containerlab)
2. Physical environment
3. BFH lab

4.1.3. Models

Table 4.1 lists the LLMs used and how they were accessed. The three models span a range of parameter counts, deployment modes, and capability levels, allowing the evaluation to draw conclusions about the suitability of different model categories for agentic network reconnaissance.

Model	**Provider**
qwen3:30b	Self-hosted via Ollama
gpt-oss:120b	BFH-TI Machine Learning Management Platform
claude-opus-4-7	Anthropic API

Table 4.1.: Evaluated Inference Models

Table 4.2.: Qualitative Evaluation Scale for LLM Reconnaissance Results

Criterion	Score	Level	Description
	1–2	Insufficient	Results are mostly wrong or irrelevant; no actionable information
Correctness	3–4	Poor	Few correct findings; many errors in services, ports, or versions
	5–6	Adequate	Majority of hosts/services correct; isolated errors in version or protocol details
	7–8	Good	Findings largely correct and complete; minor inaccuracies
	9–10	Excellent	All key hosts, services, and versions correctly and precisely identified
	9–10	Severe	Numerous fabricated hosts, services, or vulnerabilities without basis
Hallucination	7–8	Noticeable	Several unsubstantiated findings; confusion of IPs or services
	5–6	Moderate	Isolated unconfirmed statements; version numbers or banners slightly distorted
	3–4	Low	Rare inaccuracies; findings are traceable to raw scan data
	1–2	None	All statements directly verifiable from scan output; no fabrications

5. Implementation

To address these questions, an AI agent based on LangChain is integrated into the framework. The agent is equipped with tools for retrieving the hosts network configuration, autonomous CLI command execution, requesting human input and notification via email. To simplify the usage of NSAK a configuration management is introduced, which allows users to configure different LLMs and manage secrets for external systems such as email and logging.

5.1. Configuration Management Implementation

This section describes the implementation of the three concepts, — *static*, *runtime* and **resource configuration** — introduced in the concepts section of the thesis 3.2 and specified by the according milestone B.3.1.

5.1.1. NSAK Configuration Management

The NSAK configuration is split into two distinct layers: a static layer that resolves fixed paths at import time, and a mutable runtime layer that is persisted to disk and loaded on every invocation.

Static Configuration

The static configuration is defined in src/nsak/core/settings.py and shown in listing 24. It derives all relevant paths from a single BASE_PATH, which is either supplied via the NSAK_BASE_PATH environment variable or resolved automatically to the repository root at import time. RUN_PATH points to the run/ directory where runtime data such as the persisted config is stored, and LIBRARY_PATHS holds the set of directories that are searched for resource libraries. Overriding NSAK_BASE_PATH is the primary mechanism to run NSAK from a different base directory, e.g. inside a Docker container or during automated testing.

Runtime Configuration

The runtime configuration is defined as the “Config” dataclass in src/nsak/core/configuration/configuration.py and holds all state that can change during normal operation, currently the debug flag and the active “Device”. It is persisted as a YAML file at run/config.yaml and loaded on every CLI invocation.

To avoid loading the configuration on import, the module-level config object is wrapped in a lazy_object_proxy.Proxy. The proxy defers the actual Config.load() call until the object is first accessed. Since the root CLI group imports config on every command invocation as displayed in listing 26, this ensures the configuration is always initialized before any command logic runs. If run/config.yaml does not yet exist, Config.init() is called automatically to create a default configuration and write it to disk, eliminating the need for manual setup on first run.

Mutations to the runtime configuration always follow the same pattern: a manager method (e.g. DeviceManager.load()) updates the relevant field on the config object and calls config.save() to persist the change. Custom YAML representers are

registered for IPInterface and PosixPath so that these types are serialised as plain strings rather than Python-specific tags. Listing 27 shows an example of the persisted run/config.yaml after loading the bananapi_r4 device.

5.1.2. Device Configuration Management

In the context of "Project 2"3.1, this resource type was already defined conceptually, and the BananaPI R4 was added as an example in the library under lib/devices/bananapi_r4/. But the resource type was not actually added to the NSAK core, and to manage device configurations, we need to implement the Device resource type first.

Create New Device Resource Type

Similar to the other resource types, a YAML specification for defining devices in the library and multiple classes for representing and managing devices in the core are required. Because the existing resource YAML specifications do not allow merging YAML files, this implementation proposes a new base structure that fulfills this trait. Section 5.1.2 describes the necessary changes to the existing YAML specifications. The listing 29 shows the initial specification for a device resource, which will get extended later on with a section for configuration management.

In the table 5.1 below, the required Python classes are listed and described. The classes will be located under src/nsak/core/device within their respective files.

Class	File Name	Description
Device	device.py	Dataclass representing a device resource at runtime, closely reflecting the YAML specification.
DeviceLoader	device_loader.py	Loads and returns Device instances from the resource library.
DeviceManager	device_manager.py	Implements business logic and exposes an API for use by the CLI.

Table 5.1.: Device Resource Classes

In "Project 2"3.1, it became already clear that the described classes share a lot of boilerplate code between each other. For this reason, the base classes described in 5.1 were added to eliminate duplicated logic, thereby improving compliance with the Don't Repeat Yourself principles. The classes will be located under src/nsak/core/resource within their respective files and the adoption of the existing classes is described in section 5.1.2.

To complete the integration of the device resource, the new functionality must be exposed via CLI. The available bash commands are shown in the listing 28.

Class	File Name	Description
Resource	resource.py	Base class for all resources: Scenario, Drill, Environment and Device.
ResourceLoader	resource_loader.py	Base class which handles searching and loading resources from the filesystem.
ResourceManager	resource_manager.py	Base class which defines common methods for all resource manager classes.

Table 5.2.: Resource Python Classes

Device Configuration Management Implementation

To allow a device to expose its configuration options, the YAML specification needs to be extended accordingly. For now, the focus is only on network configuration, as this is the most important information for the scenarios and drills. The specification was designed so that other sections can be added later. The excerpt 30 shows the specification for the configuration section of the device resource YAML.

The code snippet, located under src/nsak/core/device/device_loader.py 31 shows how the YAML configuration gets parsed to the respective datastructures defined in src/nsak/core/network/configuration.py and src/nsak/core/device/device.py 32.

The referenced listing shows the newly available CLI commands for managing device configurations 33:

Refactor Existing Resources

The implementation of the device resource 5.1.2 introduced an improved base structure for the resource YAML specifications and three base classes which are abstracting the common structure and functionality. This section describes the resulting changes in the existing code and resource YAML specifications.

As already stated in section 5.1.2 all resource type specific classes were refactored to inherit from their respective base classes as described in table 5.1. With the example of the EnvironmentLoader, the diff below 5.1 illustrates how the abstraction of common properties and methods allows for the removal of a lot of boilerplate code.

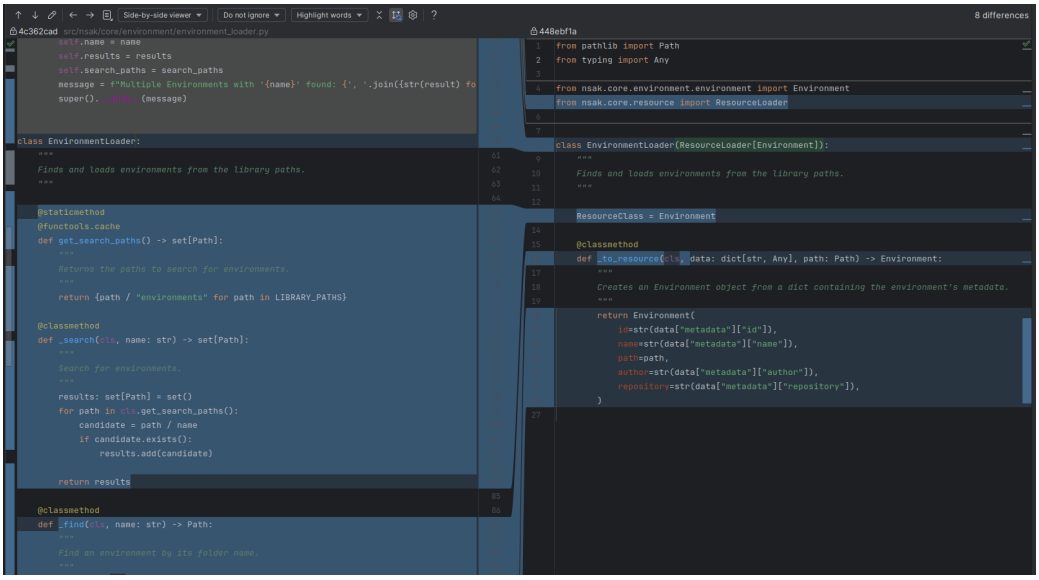


Figure 5.1.: EnvironmentLoader Base Class Refactoring Diff

The structure of all resources was aligned by nesting the resource under their respective resource type key (scenarios, drills, environments, and devices), so that we could merge all YAML resource files into a single library data structure without key conflicts. In the YAML specification 34, the new base structure derived from the device resource 5.1.2 is illustrated. All existing resource YAML files in the library were adjusted accordingly, and thanks to the introduction of the ResourceLoader base class, the required adjustments in the code were minimal.

5.1.3. Scenario & Drill Configuration Management

Both the scenario and drill resource types were already fully implemented in “Project 2”3.1. Their YAML specifications already contained an interface section describing argument and return types, but this information was purely documentary and had no effect at runtime.

As part of this project, the interface.arguments section was extended 35 so that each argument carries an explicit type annotation and an optional default value.

The CLI reads these annotations at runtime to generate typed command options for each drill and scenario automatically, without requiring any additional handwritten code. The discover_hosts drill in listing 38 illustrates this: its single interface argument of type str is translated directly into a --interface option on the generated CLI command.

Scenarios follow the same interface convention and additionally declare the drills

and environments they depend on. The mitm scenario in listing 36 chains the discover_hosts, arp_spoof, and transparent_tcp_proxy drills together and exposes a single typed interface argument, which the CLI surfaces in the same way as for individual drills.

5.2. Reproducible Test Environment

This section describes the implementation of the reproducible test environment as introduced in the concepts chapter **??** and specified in the according milestone B.3.1. The virtual_enterprise_network_lab.clab.yaml described in the last subsection 5.2.7 fully describes, specifies and implements the reproducible test environment as required for testing and evaluation.

5.2.1. Network Topology

The network topology shown in Figure 5.2 illustrates the IP addressing scheme and the interconnections between the individual components.

The external node represents an external attacker interacting with the network from the Wide Area Network (WAN). The nsak container can be deployed in different network segments, enabling the simulation of various attacker positions and reconnaissance scenarios.

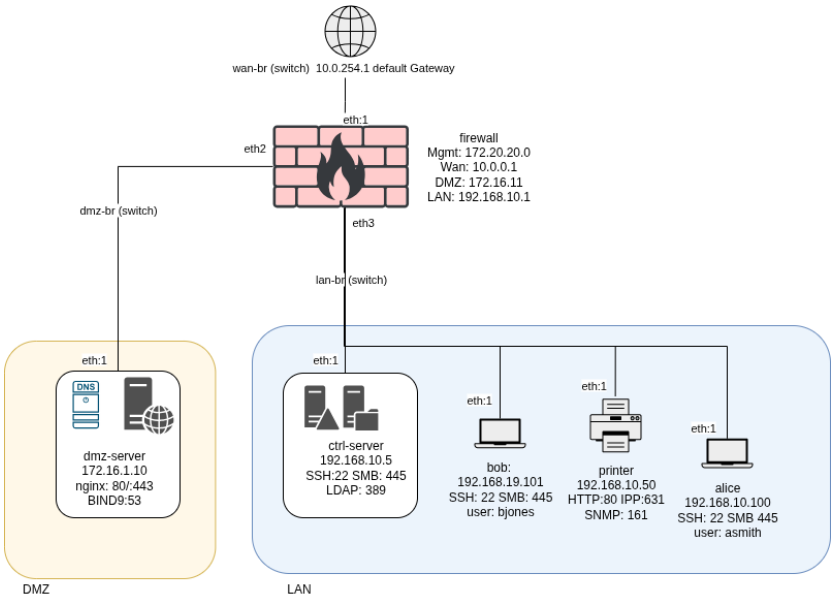


Figure 5.2.: topology-test-environment

5.2.2. DMZ-Server

To evaluate reconnaissance scenarios, common services must run on endpoints.

Berkley Internet Name Domain (BIND) is a complete implementation of the Domain Name System (DNS) protocol that can be configured through the named.conf file as an authoritative name server and resolver.

In the test lab illustrated in 5.2, the functionality of the web server and DNS server was combined for simplicity.

During the De-Militarized Zone (DMZ)-server container build process, a setup shell script is executed.

In the first part, the authoritative BIND named.conf file, DNS options, and master zones for forward and reverse lookups are created.

In the forward zone, domain names map to IP addresses, while in the reverse zone, IP addresses map to domain names **??**.

Figure 5.3.: Authoritative Zone Transfer Snippet

The output of a full zone transfer illustrated in figure 5.3 displays the resolved IP addresses and domain names. In our test environment, the restriction of Authoritative Zone Transfer (Full Zone Transfer) requests to only trustworthy IPs is intentionally omitted; the devices are deliberately left unhardened.

In the second part, a minimal NGINX web server with HTTP and HTTPS support is created. Like in real setups, HTTP traffic on port 80 is redirected to HTTPS on port 443. To provide an artifact for reconnaissance purposes, a file named {secret.txt} containing sensitive data is placed on the server.

The Local Area Network (LAN) zone of the network, consists of 4 devices.

5.2.3. Domain Control and File Server

The internal server, reachable at 192.168.10.5, combines the role of domain controller and file server into a single node.

It exposes Secure Shell (SSH) on port 22, SMB on ports 139 and 445, and Lightweight Directory Access Protocol (LDAP) on port 389.

The LDAP service is intentionally misconfigured to permit anonymous read access to the entire dc=lab,dc=local directory tree, exposing user accounts and group memberships without authentication.

The SMB service provides three shares: public is readable without credentials, while finance and it require valid user accounts.

Sensitive files such as payroll exports and budget summaries are placed in the restricted shares as intentional post-exploitation artifacts **??**.

5.2.4. Workstations

Two workstations reside on the LAN segment: Alice at 192.168.10.100, assigned to the Finance department, and Bob at 192.168.10.101, assigned to the IT department.

Both nodes expose SSH on port 22 and SMB on port 445, and both SSH banners intentionally disclose the organization name.

On Alice, the home directory of user asmith contains a confidential finance report and a drive mapping file that reveals the path to the central file share on the domain server.

On Bob, the home directory of user bjones contains a scripts directory with a host inventory that lists all internal IP addresses and their respective roles **??**.

5.2.5. Printer

The printer service, reachable at 192.168.10.50, provides two different interfaces. The first web interface is administrative and is available on port 80. It shows internal printer metadata, such as device name and contact information. The printer version is intentionally disclosed in this admin view. The second interface, a REST API, listens on port 631 at the {/jobs} endpoint, where it displays a predefined list of print jobs. All printer requests are logged in {/var/logs/printer_sim_logs} inside the container **??**.

5.2.6. Introduced Vulnerabilities

For reconnaissance purposes, several vulnerabilities and flaws were introduced, which are partly mentioned in the previous sections and are presented in detail in Listing **??**. These flaws are necessary to compare the results of the larger and smaller models as well as the implemented vs the AI enhanced scenarios.

5.2.7. Containerlab virtual_enterprise_network_lab.clab.yaml

Listings 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, in the appendix are showing the Containerlab virtual_enterprise_network_lab.clab.yaml file. The nodes are representing hosts as OCI-Containers, connected with virtual ethernet links to the bridge, which acts as a multi-port switch. For simplicity, we decided to use bash scripts to configure and start the services, instead of building dedicated containers for all hosts.

Together, these nodes form a deliberately unhardened network, containing a range of misconfigured services, providing a realistic target for evaluating reconnaissance scenarios.

5.3. AI Scenarios

This section describes the design and implementation of the milestone AI Scenarios B.3.1, which is based on the user story map workshop AI Purple Team D.9. In the workshop we defined the features which such a scenario may provide and categorized them in “Must”, “Could” and “Should”. Because of time constraints, we set the focused only on the user stories categorized as “Must”. The main goal is to implement an AI reconnaissance scenario, which is described in the 5.3.8. But on the way it made sense to also create an intermediary scenario which can run custom prompts 5.3.7. Further, we already explored the AI based intrusion detection scenario 5.3.9 which should be seen as proof of concept, because the corresponding milestone and goals are categorized as “Should”.

5.3.1. AI Agent

An AI agent is an autonomous system that combines a large language model (LLM) with a set of tools it can invoke to accomplish a goal. Rather than just generating text, the agent follows a reasoning loop: it plans which tool to call, executes it, observes the result, and continues until the task is complete [27].

The project team chose LangChain as the agentic framework without comparing it with alternative solutions, for the following reasons:

- Free and open-source
- Python package with native integration into the project’s toolchain
- Suggested by various sources and has a good reputation in the community
- Extensive documentation and active community support

For the integration into the NSAK framework we decided to implement an AiAgent class as the central abstraction, which wraps a BaseChatModel together with a set of LangChain tools and delegates the reasoning loop to LangChain’s built-in create_agent().

AI Configuration

Before any AI based scenario can run, the AiConfiguration entry must be set in the runtime configuration, containing the model, provider and credentials. Listing 10 shows the dataclass, which holds two fields: model and base_url. For authentication, the basic auth can be provided in the base_url field, otherwise the api_key must be set.

With the CLI command nsak config set ai the configuration can be set interactively, alternatively the values can be set individually like in the following example:

- nsak config set ai\.provider "ollama"
- nsak config set ai\.model "qwen2.5:7b-instruct"
- nsak config set ai\.base_url "http://user:pass@localhost:11434"
- nsak config set ai\.api_key "secret"

Provider Abstraction

The AiAgent.create_model() method parses the provider prefix and dispatches to the appropriate LangChain chat model class via PROVIDER_MAP, as shown in listing 3. This means any supported provider can be swapped out by changing a single configuration value, without touching scenario code. Most providers allow to set the "temperature" of a model, where lower values produce more deterministic responses, while higher values increase creativity and diversity [28]. For our intents and purposes always set temperature=0, as we want to make agent behavior as deterministic as possible.

5.3.2. LangChain Concepts

LangChain provides a framework for developing applications powered by large language models (LLMs) and agent-based systems. It enables developers to combine modular components and external integrations, simplifying AI application design while maintaining flexibility as underlying technologies evolve [23].

The primary LangChain concepts used in the NSAK framework are:

- BaseChatModel - provider-agnostic interface for the underlying LLM.
- BaseTool - interface for callable tools that the agent can invoke.
- create_agent() - builds a LangGraph-based ReAct agent graph that wires the model and tools together.
- messages - a list of conversation messages (user, AI, tool) maintained across the agent's reasoning loop.

The listings in the appendix section E.1.1 are showing the agents implementation in detail. The AiAgent.__init__() method calls create_model() to instantiate the provider-specific chat model and then passes it to create_agent() together with the tool list and a fixed system prompt that establishes the cybersecurity simulation context. The run() method invokes the agent once and streams all resulting messages (user,

AI, tool calls, tool results) as formatted strings. The run_interactive() method extends this into an interactive loop, where after each agent response the operator is prompted for the next instruction, and the full conversation history is retained in the messages list until the operator types an empty line or exit.

Tool Calling Concepts

Tool calling is the mechanism by which the LLM declares that it wants to invoke an external function. The framework intercepts the model's output, parses the tool call, executes the corresponding Python function, MCP, or Skill, and appends the result to the message history before asking the model to continue. This loop repeats until the model produces a final text response with no tool calls.

In NSAK, tools can be provided through several complementary mechanisms:

- **LangChain Tools** - native Python functions decorated with @tool, which LangChain automatically wraps with type-safe JSON schemas for the model. This is the primary mechanism used for NSAK-specific tools (host configuration, CLI access, e-mail, human interaction hook). Further, all NSAK drills are wrapped as LangChain tools via generate_drill_tools(), giving the agent type-safe access to all drill capabilities.
- **MCP** - the Model Context Protocol allows external tool servers to expose additional capabilities; used here for the draw.io integration (see 5.3.2).
- **CLI Access** - the cli_tool gives the agent direct shell access, allowing it to run any installed binary (nmap, tshark, etc.).

Tool Calling Models

Considering its limited size of 7 billion parameters, we were quite happy with our first test model "qwen2.5:7b-coder". But as it turns out, not all models are capable of calling tools. Often, multiple variants of the same model are available, and they don't only differ in size, but also for different strengths or capabilities such as tool calling. The tool calling variants are often have the suffix "instruct". Luckily, "qwen2.5:7b-coder" has such variant called "qwen2.5:7b-instruct".

Explain the issue with limited context windows and high token usage of certain MCP tools, such as drawio mcp

The create_ai_agent() factory function shown in listing 9 is the central assembly point used by all three AI scenarios. It reads the ai configuration block, assembles the tool list (always including cli_tool, host_configuration, and send_email), optionally adds the human interaction hook or drill tools, loads MCP tools with graceful fallback, and returns a configured AiAgent instance.

The enable_drill_tools flag, when set to **True**, calls generate_drill_tools(), which dynamically wraps every NSAK drill as a type-safe LangChain tool. This gives the agent

access to structured capabilities such as discover_hosts and port_scan without requiring CLI invocation.

MCP Integration

The MCP [14] is an open protocol that standardises how AI models connect to external tool servers. Rather than implementing tool calling ad-hoc, MCP servers expose a well-defined interface that any compatible LLM framework can consume [29]. LangChain Agents must be invoked in an event loop to properly support MCP, this forced us to refactor the implementation “ai_agent.py” to be async. Further we need to add support for async drills and scenarios, as otherwise we could not run the AI agent inn those contexts.

NSAK integrates an external draw.io MCP server to allow the AI agent to create and manipulate diagrams during a scenario run.

MCP tools are loaded asynchronously inside create_ai_agent() and appended to the tool list. If the server is unavailable, the exception is caught and logged as a warning so the scenarios can still run without diagram support.

LangChain Tool: Host Configuration

The host_configuration tool shown in listing 14 gives the agent a structured view of the host’s current configuration. It returns a dictionary containing the loaded device configuration with interface names, Internet Protocol addresses, and the is_target / is_management flags loaded from the device YAML. This bootstraps the agent’s situational awareness without requiring it to run ip addr or parse raw network output - the agent knows from the start which interfaces are part of the attack surface and which are for management traffic.

LangChain Tool: CLI Tool

The cli_tool shown in listing 15 allows the AI agent to execute arbitrary shell commands with a configurable timeout (default 120 s). It returns a tuple of (exit_code, stdout, stderr), giving the agent full feedback to diagnose failures and adapt its next action. The scope is intentionally broad: the agent can invoke nmap, tshark, apt, or any other binary available in the execution environment. Besides the execution of drills, this flexibility is what makes the AI scenarios powerful, but it is also why the kill switch described in 5.3.6 is a necessary safety measure.

5.3.3. Human Interaction Hook

LangChain Tool: Human Interaction Hook

We implemented a simple LangChain python tool which allows a human operator to interact with the agent 16. If enabled via the interactive flag when calling the create_ai_agent() function, the agent can pause its execution and ask the operator a question, then incorporate the answer into its next reasoning step.

LangChain Middleware: Human in the Loop (HITL)

After the implementation we read in the LangChain documentation, that it has its own middleware for allowing human interactions, called Human-in-the-loop. We speculated that this solution is more sophisticated than the few lines of python we wrote and tried to refactor our implementation accordingly. It turned out, that it required a lot more changes than we first thought and the proof-of-concept did not work as stable as the current tool based implementation. As we focus on the agents non-interactive mode for the evaluation, we don't want to spend more time on this potential improvement. For these reasons we decided not to merge the code and let the implementation as is. The branch containing the proof-of-concept is still available under 310-human-in-the-loop.

LangChain Agent Chat UI

We did not add the LangChain Agent Chat UI because we made some implicit architecture decisions which prevent easy integration.

If we decide to implement it regardless, we would need to redesign and refactor the AI Agent implementation, for example by switching to a callback-based output model. A useful reference for such a migration is available at this LangChain UI tutorial.

5.3.4. Logging

LangChain Agent Logs

Explain the general python logging setup and implementation of NSAK

Explain that the LangChain agent already provides extensive logging, where we found

Remote Logging System: Grafana / Loki

Uses the package python-logging-loki. We went for this method over more conventional approaches, because we can provide more metadata in form of “tags”.

instead of collecting the logs from stdout Filters: Regex: \[(AI)|(Prompt)|(Tool)\]

F.5.7

Complete this section

Add a figure

5.3.5. E-Mail Alerting

The python standard library already includes packages for creating and sending E-Mails via Simple Mail Transfer Protocol (SMTP). The “EmailBackend” implemented to the NSAK framework is an opinionated wrapper around those packages and leverages the previously introduced 5.1 to set up necessary configuration. If no SMTP configuration is provided as specified in 12, the system will write the E-Mails as logs instead. The working tool can be seen in actions in the test run of the AI Reconnaissance scenario 5.3.8, where the operator instructs the agent to send the results via E-Mail.

LangChain Tool: E-Mail Alerting

The send_email tool shown in listing 17 exposes the EmailBackend to the AI agent. During a scenario run the agent can autonomously send e-mails to operators - for example, when the AI intrusion detection scenario identifies a suspicious host or the operator instructed the AI reconnaissance scenario to send a report. The recipient of the mail is defined via the configuration management and cannot be changed by the agent, so that the operator always knows which mails where sent and who received them. Optional cc_recipients and attachments parameters allow the agent to inform additional stakeholders or attach diagrams and reports. Each email comes with some predefined For better traceability a template is defined, which is filled with meta information including the current scenario and a unique ID, which can be used to associate the mail in grafana. Further, the subject line is always prefixed with [NSAK], so that operators can filter them in their inbox.

5.3.6. CLI Kill Switch

It’s possible that an AI-Agent starts doing unexpected, unwanted or even destructive actions. This was also identified as a possible risk in the risk assessment B.7.1. This kind of misbehavior, combined with elevated permissions and tool calling capabilities via direct CLI access, could lead to a wide range of issues:

- The AI-Agent is stuck in a loop while using most system resources.
- The AI-Agent misconfigures the system, rendering it inaccessible or even unusable.
- The AI-Agent burns too many API credits because of a bug, an inefficient request, or a huge task.
- The AI-Agent starts an attack against a production network.

For these reasons we want to be able to kill a specific or all scenarios as fast as possible. As a first measure, the project team decided to add this functionality as simple CLI commands:

- nsak scenario kill <scenario-id>: Kill a specific scenario (sends SIGTERM to the container)
- nsak scenario killswitch: Kill all running scenarios
- nsak killswitch: Alias for nsak scenario killswitch

5.3.7. Scenario: AI Agent

The AI Agent scenario is the most generic of the three: it accepts a free-form prompt argument and runs the AI agent in an interactive loop, maintaining the full conversation history across turns. This makes it the primary entry point for ad-hoc AI-assisted tasks where no dedicated scenario exists yet. The operator can guide the agent step by step, observe its reasoning, and issue follow-up instructions until they type an empty line or exit. Listing 18 shows the scenario entrypoint and listing 19 shows its YAML interface declaration.

Scenario: AI Agent - Test Run

nsak scenario execute ai_agent --prompt "Scan the target network and report all findings" --interactive True

Describe and add a reference to the result of the test runs and the limited tool calling capabilities of smaller models

5.3.8. Scenario: AI Reconnaissance

The AI Reconnaissance scenario automates host and service discovery on a specified network interface. It encodes a structured five-step plan directly into the initial prompt: the agent is instructed to call host_configuration to determine the Internet Protocol addresses and subnets on the interface, then use the cli tool to invoke nmap for host scanning, then again for service enumeration, and finally to report all

findings including vulnerability indicators. If the interactive flag is set, the last step invites the operator to issue follow-up instructions via the human_interaction_hook.

This prompt-driven approach means the scenario does not contain any network scanning logic itself — the agent decides which nmap flags to use, how to interpret results, and what to scan next. Listing 20 shows the scenario entrypoint and listing 21 shows its YAML interface declaration.

Scenario: AI Reconnaissance - Test Run

nsak scenario execute ai_reconnaissance --interface eth0

Describe and add a reference to the result of the first testrun

5.3.9. Scenario: AI Intrusion Detection

The AI Intrusion Detection scenario is a proof of concept, as it was specified as "Should" goal. It demonstrates a two-phase approach: first it performs host discovery using DrillManager.execute("discover_hosts"), then it captures live network traffic using tshark (a system dependency declared in the scenario YAML). Up to 200 packets are captured over a 60-second window and extracted as Comma Seperated Values (CSV) fields (frame number, source and destination Media Access Control, source and destination Internet Protocol, Transmission Control Protocol/User Datagram Protocol (UDP) destination port, protocol stack, and frame length).

Both the host discovery table and the packet CSV are injected into a structured PROMPT_TEMPLATE that instructs the AI to identify at most five suspicious events, each with source/destination Internet Protocol and Media Access Control, protocol, and the reason for suspicion, and to identify any malicious hosts. By constraining the number of events the prompt keeps the agent's output concise and actionable. Listing 22 shows the full scenario entrypoint and listing 23 shows its YAML interface declaration.

Scenario: AI Intrusion Detection - Test Run

nsak scenario execute ai_intrusion_detection --interface eth0

Describe and add a reference to the result of the first testrun

5.4. Red Team: Reconnaissance Scenario

The reconnaissance scenario is an engineered red team scenario that systematically maps an unknown network. It chains four drills in sequence: host discovery, port scanning, and service enumeration. If no interface is specified by the operator, the scenario first discovers all active physical interfaces and, if any lacks an IP address, requests one via Dynamic Host Configuration Protocol (DHCP) before scanning. The scenario results serve as the reproducible baseline for comparison with the AI-driven reconnaissance approach. The full scenario implementation is provided in Listing 37.

5.4.1. Goal

The goal of the scenario is to identify all reachable hosts, their open ports and running services in a target network without prior knowledge of the topology. The drill chain is designed to work generically across different network environments: the operator can either provide a specific interface and subnet or let the scenario discover the available interfaces automatically.

5.4.2. Discover Hosts Drill

The *Discover Hosts* drill is the first active step of the scenario. For local network segments, it runs arp-scan --localnet on the given interface, which sends Address Resolution Protocol requests and collects the Internet Protocol address, Media Access Control address, and vendor of each responding host. If an explicit subnet is provided, the drill falls back to nmap -sn instead, which uses Internet Control Message Protocol (ICMP) and Transmission Control Protocol probes and therefore works across Layer 3 boundaries. Media Access Control addresses are only available for hosts on the same Layer 2 segment; remote hosts are recorded without a Media Access Control. Further limitations and possible extensions of the drill, such as supporting a static Internet Protocol address in networks without DHCP, are discussed in the sprint review B.8.4. The full implementation is provided in Listing 39.

Drill Interface

The drill exposes two arguments: a required interface and an optional subnet. If subnet is omitted, the drill defaults to a local ARP scan; if provided, it switches to the nmap-based subnet scan. The resource configuration is shown in Listing 38.

5.4.3. Enumeration

After host discovery, the scenario runs two further drills that progressively deepen the collected information.

Port Scan

The *Port Scan* drill receives the ReconnaissanceScenarioResult produced by host discovery and runs nmap -sV --open against each discovered Internet Protocol. For every open port, the drill records the port number, transport protocol (Transmission Control Protocol/UDP), service name, and detected version string and appends a NetworkService entry to the existing result map. The full implementation is provided in Listing 40.

Service Enumeration

The *Enumerate Services* drill takes the enriched result map and runs targeted Nmap Scripting Engine (NSE) scripts for each discovered service. Well-known services such as HTTP, DNS, FTP, SMTP, SMB, and LDAP are mapped to a specific set of scripts (e.g. http-title, dns-zone-transfer, smb-security-mode). Unknown services fall back to the generic banner script, which captures the initial response of any Transmission Control Protocol service. The drill returns a mapping of ip:port keys to lists of finding strings extracted from the NSE output. The full implementation is provided in Listing 41.

5.4.4. Output Artifacts

The primary output of the scenario is the ReconnaissanceScenarioResult, which aggregates all results per interface. Each entry maps an interface name to a NetworkDiscoveryResult containing the list of discovered NetworkService objects with their endpoints (IP, MAC, port, protocol, version). After port scanning, the scenario prints the result as a human-readable table. The service enumeration findings are printed separately as a structured list grouped by ip:port.

Add Snippet

5.5. Benchmark Suite

As described in the evaluation criteria 4.1.1 there will be at least 120 evaluation runs, which will be a lot of data to analyze and assess. For this reason, we decided to implement a benchmarking suite, consisting of a benchmark harness that executes benchmark runs and summarizes their results into a report. The idea is to fully automate the aggregation of the **quantitative criteria** (duration, token usage) and to prepare the reports needed for the **qualitative criteria** (correctness and consistency) as good as possible. The key concepts for this implementation are described the table below 5.3:

Term	Concept
BenchmarkManager	The automation harness
BenchmarkRun	Represents a single execution
BenchmarkResult	The result data type of a single execution
BenchmarkReport	The collected output aggregating all results

Table 5.3.: Benchmark Suite Concepts

6. Evaluation

This chapter evaluates the agentic AI integration across the three research questions defined in the 1. The assessment is structured into two parts, a controlled evaluation in the reproducible Containerlab environment 6.2, and a real-world evaluation in the BFH network security lab 6.3. Each part examines the same qualitative and quantitative criteria specified in Chapter 4.1 (Evaluation Setup), allowing for a direct comparison across configurations and environments. The Benchmark Suite (5.5) was leveraged for investigating the research questions Introduction and Introduction, as they do not require the interactive mode of the scenarios.

6.1. Scenario Overview

Table 6.1 presents an overview of the used scenarios and their characteristics.

Scenario	Type	Output
Reference Scenario	Static	Structured
AI Scenario	Single Agent	Structured
AI Scenario	Multi Agent	Structured
AI Scenario	Multi Agent	Unstructured

Table 6.1.: Evaluated Scenarios

The main distinction must be made between the static reference scenario and the AI based scenarios, because they significantly differ in how they function conceptually and technically. The static scenario is limited to a fixed execution order of drills 3.1.3 (namely, arp-scan, nmap), which also results in a more predictable exception based error handling. The agent driven scenarios on the other hand can choose a broader palette of LangChain tools (e.g., CLI, host configuration) depending on the situation and the error handling can become quite intransparent, because a model might retry failed tool calls, start to hallucinate or even decides to skip entire steps. Additionally, there are three variations of the AI scenario with different implementation details, with slightly different implementation details compensating different limitations. The characteristics and which limitations they try to address are described in the following paragraphs:

Type As already described the reference scenario does not incorporate on an LLM, but on a **Static** sequential implementation. A **Single Agent** scenario in non-interactive mode rely on one prompt to achieve its goal. This can result in a long chain of tool calls, all contributing to the context. Even though it is hard to quantify, its widely accepted that large contexts in relation to the maximum context size (context window) are degrading the models' performance. To address this issue and optimize for smaller models, we implemented the **Multi Agent** variants. This implementation tries to reduce the context size with a divide-and-conquer approach — the prompt is split into multiple steps and each result is fed into the next agent with the next instruction — which probably results in longer runtimes. For large models this approach could be even harmfully, as they probably don't have a problem with the context size and even call the same tools in multiple agents.

Add a reference for the claim

Output While **Structured** output comes with many benefits, especially for quantitative criteria, it seems to result in a high failure rate for LLMs — especially for smaller models. This is probably caused by the increased context size and complexity resulting from the generated grammar required for validating the datastructure. Because all models struggled to produce any valid responses in the BFH network security lab, we developed an AI scenario variant with **Unstructured** output on the spot. This resolved the issue and the agents could successfully generate reports on their findings, but we could not relly on the automation provided by the benchmarking suite, which resulted in much more manual labor.

Add a source for the claims

Maybe move the scenario overview to concepts or implementation. IDK?

6.2. Virtual Environment (Containerlab)

Overview Tables 6.2 and 6.3 present results for both the single-agent and multi-agent implementation.

Table 6.2.: Benchmark Results — Containerlab, Single-Agent ($n = 10$ runs each)

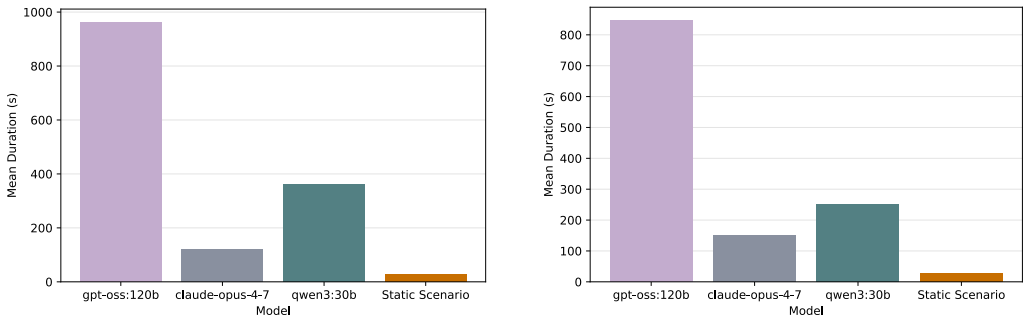
Model	Avg. Duration	Avg. Tokens	Avg. Hosts	Avg. Services	Avg. Findings	Success Factor
claude-opus-4-7	122 s	52,888	5	9	8	10/12
qwen3:30b	362 s	86,446	4	7	5	10/23
gpt-oss:120b (BFH)	963 s	62,475	5	6	4	10/18
Static Scenario	29 s	—	5	13	8	10/10

Table 6.3.: Benchmark Results — Containerlab, Multi-Agent ($n = 10$ runs each)

Model	Avg. Duration	Avg. Tokens	Avg. Hosts	Avg. Services	Avg. Findings	Success Factor
claude-opus-4-7	150 s	59,988	5	9	8	10/10
qwen3:30b	252 s	27,789	4	8	5	10/11
gpt-oss:120b (BFH)	847 s	46,736	5	7	5	10/14
Static Scenario	29 s	—	5	13	8	10/10

Tables 6.2 and 6.3 summarise the high-level numbers for both configurations. Overall, *claude-opus-4-7* keeps near-identical discovery quality in both modes, while the smaller models *qwen3:30b* and *gpt-oss:120b* benefit visibly from task decomposition in all compared quantitative measurements points. The *Static Scenario* serves as the deterministic reference baseline. The individual metrics are discussed in detail alongside the figures in the following sections.

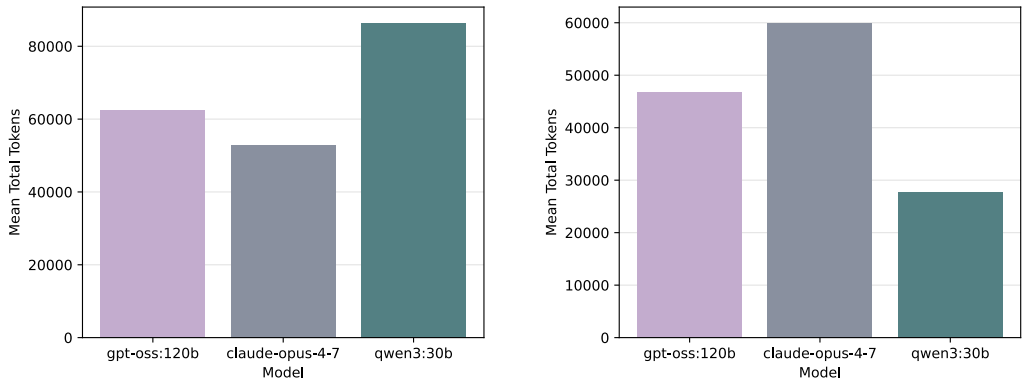
6.2.1. Quantitative Criteria



(a) Single-agent (b) Multi-agent

Figure 6.1.: Mean run duration per model (Containerlab, $n = 10$ runs each).

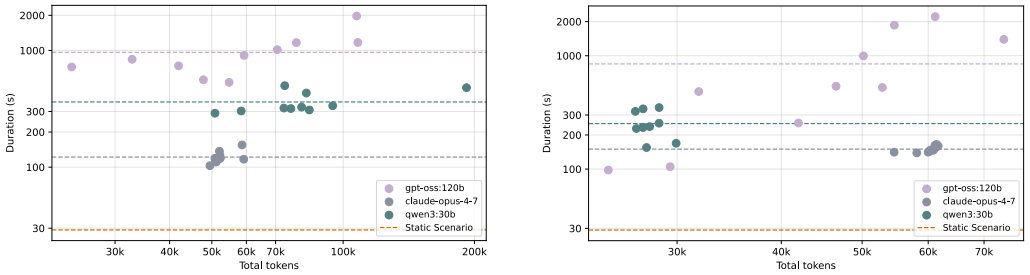
Figure 6.1 visualises the mean run duration per model for both configurations. In single-agent mode, *gpt-oss:120b* leads the duration with 963 s, followed by *qwen3:30b* at 362 s and *claude-opus-4-7* at 122 s. Multi-agent decomposition reduces duration for all three models: *gpt-oss:120b* drops to 847 s ($-12\,\%$), *qwen3:30b* to 252 s ($-30\,\%$), and *claude-opus-4-7* increases slightly to 150 s The Static Scenario remains constant at 29 s as the reference.



(a) Single-agent (b) Multi-agent

Figure 6.2.: Mean total token usage per model (Containerlab, $n = 10$ runs each).

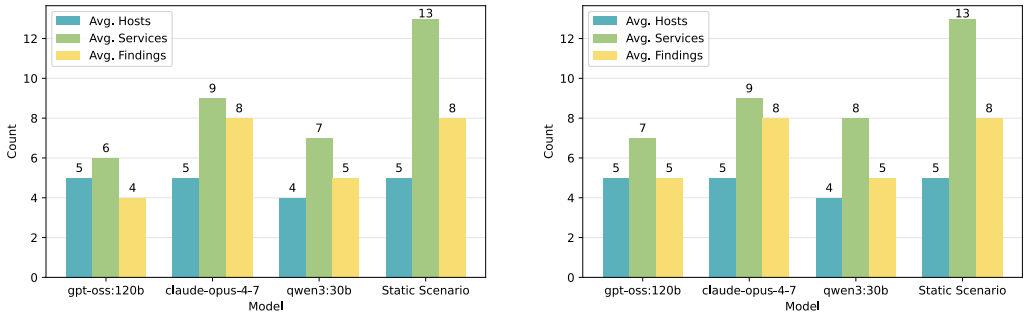
Figure 6.2 shows mean token consumption and reveals the most striking difference between the two configurations. *Qwen3:30b* reduces its token usage by 68 % from 86,446 (single-agent) to 27,789 (multi-agent), the largest relative drop of any model. *Gpt-oss:120b* decreases from 62,475 to 46,736 ($-25\,\%$), while *claude-opus-4-7* increases slightly from 52,888 to 59,988 ($+13\,\%$). Token count alone is not a reliable indicator of result quality, which is addressed in later sections.



(a) Single-agent (b) Multi-agent

Figure 6.3.: Total tokens vs. run duration per individual run (Containerlab, $n = 10$ per model).

Figure 6.3 plots total token consumption against run duration for every individual run, allowing to compares the models across configurations. In single-agent mode (left), *claude-opus-4-7* (gray) clusters in low-duration, mid-token area; *qwen3:30b* scatters widely in the mid-duration, high-token region; and *gpt-oss:120b*; spreads across the far-right high-duration and is much more unreliable with token consumption and execution duration. In multi-agent mode (right), all three clusters shift toward lower token counts, and the *qwen3:30b* cluster more closely. *claude-opus-4-7* remains tightly grouped in both modes, confirming its low variation between the runs. The horizontal dashed line for the Static Scenario reflects its fixed 29 s execution with no token consumption in either configuration. The per-model averages are shown as dashed lines.



(a) Single-agent (b) Multi-agent

Figure 6.4.: Average hosts, services, and findings discovered per model (Containerlab, $n = 10$ runs each).

Figure 6.4 compares discoveries grouped as hosts, services (a port, service combination) and findings are introduced as port and ip combination discovered by the nmap nse scripts. *claude-opus-4-7* discovers 9 services and 8 findings in both modes, confirming that the frontier model extracts equivalent intelligence regardless of setup. The results are almost similar in both setups but shifts slightly in favor of the multi-agent approach for smaller models. *qwen3:30b* gains one service (7 to 8) and *gpt-oss:120b* gains one service (6 to 7) and one finding (4 to 5). Host discovery remains stable at 5 for *claude-opus-4-7* and *gpt-oss:120b*, and at 4 for *qwen3:30b* in both configurations. The Static Scenario's 13 services exceed all AI models because the script issues exhaustive port sweeps unconditionally, while AI agents terminate scanning early once they reason the network is sufficiently enumerated. The static scenario list every host and follows with the services which result in 13 services. The AI scenarios consolidate the host information directly into the service rows, so a host with open ports is not listed a second time and only the gateway (without open ports) keeps its own entry, which adds up to 9 rows. Both approaches discover the same eight open services. The difference reflects the representation of the discovery map rather than the actual coverage.

I think the above part is already part of the discussion / also a bit speculative? / Can we defend this claim?

6.2.2. Qualitative Criteria

The evaluation criteria and scoring scale are defined in table 4.2. One of the key factors in the Containerlab environment is whether a model extracts the cleartext LDAP credentials exposed by the anonymous bind on *192.168.10.5* (*uid=asmith*/*uid=bjones*, password *Password123!*) —the scenario's single *Critical* finding.

Table 6.4.: Qualitative Evaluation – ContainerLab (Multi-Agent)

Model	Correctness	Hallucination
claude-opus-4-7	8	3
qwen3:30b	5	3
gpt-oss:120b	6	5
Static Scenario	10	-

Claude-opus-4-7 achieves the highest correctness score by consistently identifying the critical LDAP anonymous bind and extracting the plaintext credentials in every single run. Furthermore, the agent follow with a correct attack-chain reasoning (credential reuse → / lateral movement via SSH/SMB). It wrongly characterizes *192.168.10.50* as a honeypot based on the Python *BaseHTTPServer* banner behind an HP LaserJet facade, which lowers its correctness score and raises its hallucination score.

gpt-oss:120b shows high variance between runs. In roughly half the successful runs it extracts the credentials and produces a thorough report, while in the other half it

notes the LDAP rootDSE and recommends restricting anonymous binds without actually performing the bind or reading the *userPassword* attribute. Several reports also reference fabricated CVE identifiers (e.g. *CVE-2023-xxxx*), raising the hallucination score.

qwen3:30b consistently performs the host and service discovery steps correctly but fails at the enumeration depth required to surface the critical finding. In 9 of 10 runs it rates the LDAP service as "Low" risk with "no immediate vulnerability detected", missing the cleartext credential exposure entirely. The low hallucination score reflects that the model makes few unverifiable claims —but this is a consequence of shallow analysis.

Table 6.5 maps each introduced vulnerability against what each configuration actually discovered, allowing a per-finding comparison against the ground truth defined in the test environment.

Table 6.5.: Vulnerability Coverage – Containerlab LAN Scope (multi-agent, $n = 10$ runs)

Vulnerability	**Static**	**claude-opus**	**gpt-oss**	**qwen3**
ctrl-server (192.168.10.5) — LDAP				
LDAP anonymous bind (rootDSE / naming context)	☐	☐	☐	–
LDAP cleartext credentials (*asmith*/*bjones*: *Password123!*)	–	☐	(☐)	–
ctrl-server (192.168.10.5) — SMB				
SMB signing not required (relay attack surface)	☐	☐	☐	☐
SMB NTLM/LanMan weak auth protocols	–	(☐)	–	–
SMB public share anonymous read (*public*/*finance*)	–	(☐)	–	–
alice / bob (192.168.10.100, .101) — SSH				
SSH password authentication enabled	☐	☐	☐	(☐)
SSH banner leaks organisation name	–	☐	–	–
printer (192.168.10.50)				
Unauthenticated web admin UI (port 80)	(☐)	☐	(☐)	(☐)
Print job history without auth (port 631)	(☐)	(☐)	–	–
SNMP public community string	–	–	–	–
Printer identified as honeypot (banner mismatch)	(☐)	☐	–	–
SSH hardening				
Legacy HMAC-SHA1 still offered	–	☐	–	–

☐ found (☐) partial (port detected, not fully enumerated) – not found

The Static Scenario discovers all 13 (minus the 5 hosts) 8 LAN services via intensive

check-mark not rendered correctly

port sweeps and runs *ldap-rootdse* to retrieve the naming context (*dc=lab,dc=local*), but it never performs a *ldap-search* or anonymous bind, so the cleartext credentials remain unextracted.

first sentences is a bit confusing

It identifies the printer's dual *Server* header (*BaseHTTP/0.6* alongside *HP-WebServer/2.6.5*) but due to its generalistic implementation, the banner mismatch is not further elaborated.

Simple Network Management Protocol (SNMP) is not probed by any configuration, as the scenario issues only Transmission Control Protocol port sweeps and no model independently adds *snmpwalk* to its tool calls. The SMB public and finance shares are listed as a recommended next step only by *claude-opus-4-7*, but authenticated share enumeration (*smbclient*) is not executed in every run. The DMZ host (*172.16.1.10*) with its open DNS zone transfer and exposed */secret.txt* lies outside the *192.168.10.0/24* target subnet and is not reached by any configuration.

6.3. Network Security Lab BFH

Table 6.6.: Benchmark Results — Network Security Lab BFH

Configuration	Avg. Duration	Avg. Tokens	Avg. Hosts	Avg. Services	Avg. Findings	Success Factor
Multi-Agent (claude-opus-4-7)	510 s	116,738	28	53	52	5/27
Unstructured (claude-opus-4-7)	739 s	222,094	–	–	–	6/109
Multi-Agent (qwen3:8b, local)	—	—	–	–	–	0/6
Static Scenario	1780 s	—	–	–	–	–

Overview Table 6.6 summarises the quantitative benchmark values, the configurations are discussed below.

6.3.1. Quantitative Criteria

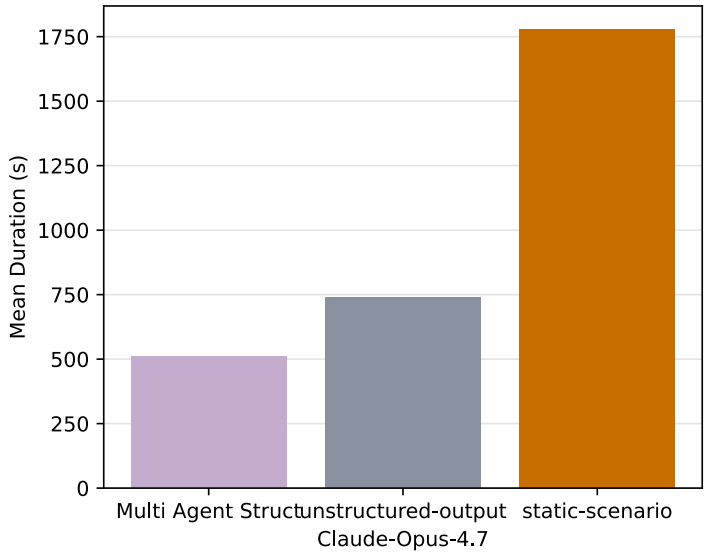


Figure 6.5.: Mean run duration per model (Cyberlab BFH).

Figure 6.5 shows the mean run duration per configuration. The multi-agent setup completes in 510 s on average, while the unstructured output scenario requires 739 s —more than twice as long. The extended duration of the unstructured version is due to the fact from its tendency to produce lengthy free-form responses before each tool invocation. The static benchmark scenario needs 1780 s to enumerate the whole network which exceeds the AI configurations by far.

Figure 6.6.: Mean total token usage per model (Cyberlab BFH).

Figure 6.6 visualises mean token consumption. The unstructured output scenario consumes 222,094 tokens on average, nearly three times the 116,738 tokens used by the multi-agent configuration, roughly 100% higher token consumption.

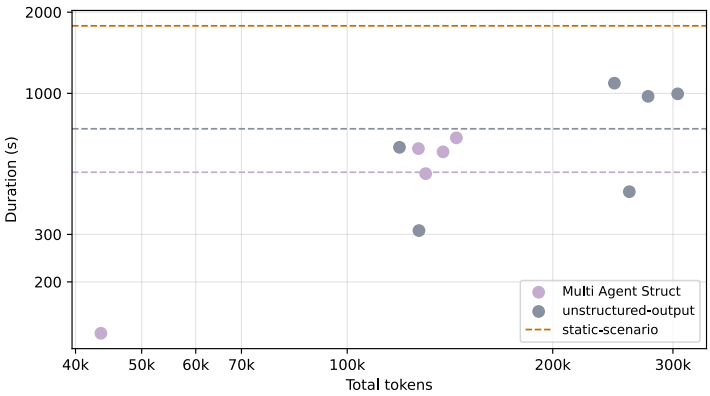


Figure 6.7.: Total tokens vs. run duration per individual run (Cyberlab BFH).

Figure 6.7 plots total token consumption against run duration for every run. The unstructured output runs shows high variance, spanning 310 s to 1,091 s and 119 k to 305 k tokens. Multi-agent runs cluster more tightly, with durations between 84 s and 684 s. The positive correlation visible within each group confirms that longer runs consume more tokens.

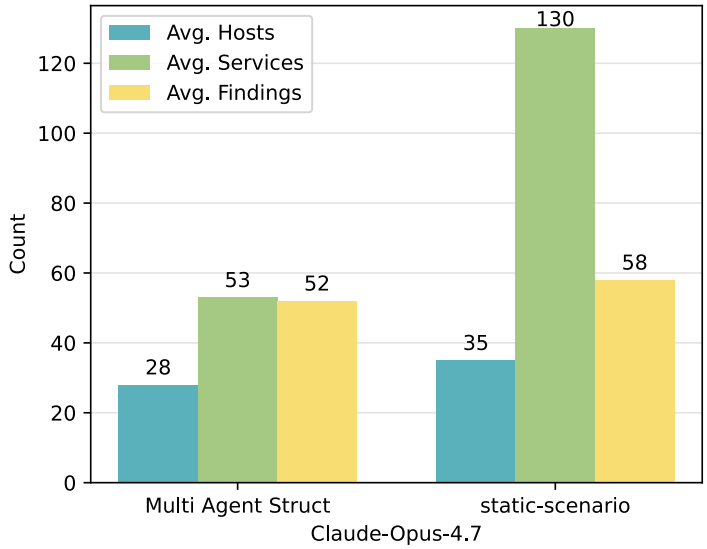


Figure 6.8.: Average hosts, services, and findings discovered per model (Cyberlab BFH).

Figure 6.8 compares services, host and findings across the two configurations. The multi-agent setup discovers an average of 28 hosts, 53 services, and 52 findings per run on the Cyberlab BFH segment. The unstructured output scenario does not produce structured discovery metrics, as its assessment is written as free-form text rather than a machine-readable result map, so no comparable figures are available for that configuration. The static scenario lists again every service per host which result in more services (130) but rather close finding count with 52 findings for AI vs 58 for the static scenario.

6.3.2. Qualitative Criteria

The evaluation criteria and scoring scale are defined in table 4.2. The qualitative findings are not evaluated in detail here, because the BFH network security lab is still in use. The most severe findings include the full extraction of end-of-life systems, open DNS recursion, and cloned host keys. Flaws in this network are partly introduced to train the students which further complicates the evaluation.

Three findings serve as quality indicators:

Model	Correctness	Hallucination	Notes
claude-opus-4-7 (multi-agent)	7	2	All major structural risks found; SNMP, open DNS recursion and cloned keys missed
claude-opus-4-7 (unstructured)	9	2	All three discriminating findings surfaced; ESXi host discovered
qwen3:8b (local, multi-agent)	1	N/A	Zero CLI calls; no reconnaissance executed
Static Scenario (reference)	10	N/A	Deterministic script; exhaustive enumeration; no assessment text

Table 6.7.: Qualitative Evaluation – Cyberlab BFH

claude-opus-4-7 in multi-agent mode achieves a correctness score of 7. In the five successful runs the model consistently identifies the six high-severity structural risks present in the network. All findings are accompanied by severity ratings and actionable remediation steps. However, none of the successful runs includes UDP enumeration, so all flaws on this protocol went undiscovered. Similarly, dns-recursion and ssh-hostkey comparison scripts are absent from most runs, leaving the open resolver risk and the cloned key finding unexplored. The hallucination score is 2: every claim in the final reports is directly traceable to nmap NSE output, with no fabricated CVE identifiers or version numbers. A notable reliability concern is the high error rate of the multi-agent setup: only 5 of 27 attempts produced a fully structured result.

claude-opus-4-7 in unstructured output mode achieves the highest correctness score of 9 among the AI configurations. When successful (six of 109 attempts), the model issued UDP scans alongside Transmission Control Protocol and discovered all the flaws listed above. The hallucination score is equally 2: all assessment statements are grounded in concrete scan artifacts. The main limitation of this configuration is its very low success rate (5.5 %) and the substantially higher token cost (mean 222,094 tokens), driven by the unstructured reasoning traces produced before each tool call.

Qwen3:8b is not further assessed due to its low success rate, which is nearly zero.

7. Discussion

This chapter connects and interprets the findings of Chapter 6 (Evaluation) with respect to the research questions defined in Chapter 1 (Introduction), which are addressed at the end of this chapter in Section 7.7. While working with Agentic-AI, the results cannot be separated from a model evaluation. The summary draws the overall picture across both evaluation environments. The subsequent sections then discuss the virtual Containerlab and the real Cyberlab BFH segment separately, because the two environments differ in scale and complexity. Because the BFH Network Security Lab is still in use, we were instructed to not disclose any details, which limits the discussion of the results.

7.1. Summary

The evaluation shows that AI-assisted reconnaissance is feasible but strongly dependent on the underlying model. As expected, the frontier model *claude-opus-4-7* delivers the most convincing results across both environments. In the rather small virtual Containerlab environment it reaches the highest correctness (9) and lowest hallucination (1) scores. Next to all discovered findings, the detailed report on how to deal with the flaws provides help, recommendations, and presents a plan for the next step to secure the network. The extraction of credentials exceeds the basic steps of reconnaissance, which according to MITRE ATT&CK [5] belongs to a later phase of the credential access step of the attack chain. The two smaller models struggle with in depth results and are prone to hallucinate or just present basic tips in the report. *Gpt-oss:120b* is adequate but prone to fabricating CVE identifiers, *qwen3:30b* performs host and service discovery correctly but lacks the enumeration depth for critical findings. The implemented static scenario provides the highest success rate, with 5 hosts, all 8 services and a fast execution speed of 29 s. No reasoning and evaluation is provided and leaves the operator with the task to assess the results. But the rapid speed and not existing token costs make up for it. This captures the central trade-off of the comparison: the engineered scenario offers holistic and deterministic coverage, whereas the AI scenario offers more in-depth analysis.

Multi-agent task decomposition improves the usability of smaller models without harming the frontier model. *qwen3:30b* cuts its token usage by roughly 68 % (86,446 to 27,789) and its duration by ca 30 %. *Claude-opus-4-7* keeps identical discovery quality with slightly more overhead (+13 % tokens, 122 s to 150 s). Decomposition

also raises execution reliability. The success factor improves for all three models, with *claude-opus-4-7* reaching a perfect 10/10.

In the more realistic BFH network security segment, a network roughly seven times larger than the Containerlab with about 35 live hosts, the multi-agent *claude-opus-4-7* setup discovers on average 28 hosts, 53 services, and 52 findings per run and consistently surfaces the six high-severity structural risks. The unstructured variant of the same model reaches an even higher correctness (9 vs. 7) because it issues additional UDP scans and NSE scripts, but only at a 5.5 % success rate and roughly three times the token cost (222,094 vs. 116,738). The locally hosted *qwen3:8b* fails completely with zero tool calls, and the static scenario needs 1780 s to enumerate the whole network. The BFH network security lab environment therefore confirms the Containerlab findings. The frontier model scales, while the engineered scenario stays reproducible but slow.

7.2. Virtual Environment (Containerlab)

AI vs. engineered reconnaissance. On just the level of reproducibility and raw coverage the engineered scenario is unbeatable. It executes a fixed drill order and returns an identical structured *ReconnaissanceScenarioResult* on every run. All services are discovered issued by an unconditional port sweep, where the AI frontier model extract the same hosts and services with a higher duration and token costs. However, raw coverage is not an indicator of qualitative results. The generalistic way how the engineered scenario was implemented, prevent the extraction of the cleartext credentials, does not elaborate the printer's banner mismatch, and produces no severity-rated assessment. The wrongly addressed honeypot 6.2.2 shows the weakness of AI agents. The reports must still be proved to be correct. Next the two different agent setups are elaborated.

Single- vs. multi-agent. The two configurations confirm that smaller models benefit more from decomposition, where the larger models are not influenced or less influenced by bigger context. A frontier model already manages a large context well, so decomposition mainly adds orchestration overhead in the Containerlab.

Smaller models are more failure prone as their context fills with accumulated tasks and history. The smaller tasks and focused context restore both efficiency and token reduction for the local hosted model and a more consistency that reflects within a tighter scatter cluster in Figure 6.3. This is the primary architectural and conceptual finding which motivates to use the multi-agent setup as the default configuration.

Interactive operator support In regard of operator support, the AI scenario differs most from the static baseline. Instead of a raw *ReconnaissanceScenarioResult*, the frontier model returns an assessment that ranks findings by severity, explains why each matters, and proposes the next step in the attack chain, which lowers the interpretation effort and the depth of expertise the operator has to bring. Multi-agent decomposition widens access to this support, by cutting the token footprint and tightening output consistency it brings smaller, self-hosted models closer to a usable assistant. Because the report can still contain a misclassified honeypot or a fabricated CVE, the operator stays in the loop as a reviewer rather than a passive consumer, so the agent augments rather than replaces human judgement.

7.3. Real Network (Cyberlab BFH)

Scaling to a real network. The Cyberlab BFH segment validates the approach on a real network roughly seven times larger than Containerlab (≈35 live hosts vs. 5). The multi-agent setup scaled to this size, discovering on average 28 hosts, 53 services, and 52 findings per run, and consistently surfaced the six high-severity structural risks. Unlike the Containerlab, the lab is a black box without a published topology and must not be disclosed, so the discussion is limited to the successful runs and the effectiveness of the agent to gather information. In this more complex environment the frontier model also benefits from decomposition through a higher success rate, where 5 of 27 multi-agent attempts produced a fully structured result.

Structured multi-agent versus unstructured reasoning. The real network exposes a trade-off that the virtual environment does not. The unstructured-output configuration issues additional UDP scans alongside Transmission Control Protocol and executes NSE scripts that the specialised sub-agents omit, surfacing all three discriminating findings (SNMP, open DNS recursion, and the cloned SSH host keys). The schema constraints helps the multi-agent setup with speed and token usage where the reasoning succeed in enumeration depth. The gain comes at a price: a 5.5 % success rate (6 of 109 attempts) and roughly three times the token cost (222,094 vs. 81,453). The local *qwen3:8b* is unusable on this scale, executing zero tool calls and producing no reconnaissance.

7.4. Unexpected Results

First, the **gain of the token reduction** from decomposition was larger than expected. A 68 % drop for *qwen3:30b* in the containerlab and shows that prompt-context growth, dominates the cost of single-agent reconnaissance for smaller models or more complicated environments which results ultimately in larger context.

Second, the **complete failure of the local qwen3:8b model** (zero *cli_tool* calls across all runs) was more absolute than anticipated. The model did not perform any reconnaissance, falling back to a generic recommendation report or a report with connectivity issues.

7.5. Limitations

Reliability and success rate. The headline figures report $n = 10$ *successful* runs, but the success factors reveal unreliability. In the cyberlab the success factor concludes 5 of 27 attempts for the multi-agent and 6 of 109 runs for the unstructured attempts. The runs were interrupted because of high token usage and missing comparison with other models, only the frontier model could perform the tasks, though prone to errors. The AI scenario is not yet dependable enough for unsupervised operation, and the reported averages reflect a filtered best-case subset.

Incomplete environment and scenario matrix. The methodology (Section 4.1.2) foresaw three environments and a blue-team intrusion-detection scenario (Section A.3.4). The evaluation covers the virtual Containerlab and the Cyberlab BFH segment but not a physical environment, and the evaluation focuses on the red-team reconnaissance scenario; the blue-team comparison (a *should* goal) remains open. In the process to implement AI agents with LangChain, the challenge to extend the agents with guardrails, dynamic tool calling to enhance the results and decompose the scenarios to address speed and token usage for smaller models where rated more critical than another similar physical environment.

decision in project management

Subjectivity of qualitative scoring. The correctness and hallucination scores were assigned manually against the scale in Table 4.2. Although the scale is explicit, the scoring involves human judgement and is not independently inter-rated, which introduces a degree of subjectivity into the qualitative results.

7.6. Recommendations

1. **Match the model tier to the task.** Autonomous reconnaissance requires a capable model; the frontier *claude-opus-4-7* is the only configuration that reliably extracted the *Critical* findings. Small local models such as *qwen3:8b* are unsuitable for tool-driven reconnaissance and should not be relied upon.

2. **Use multi-agent decomposition for smaller or self-hosted models.** Where a frontier model is unavailable for cost or data-locality reasons, task decomposition recovers much of the efficiency and consistency gap, most visibly the 68 % token reduction for *qwen3:30b*.
3. **Combine engineered breadth with AI depth.** A hybrid pipeline: the deterministic scenario for exhaustive, reproducible service enumeration, followed by an AI layer for semantic assessment and attack-chain reasoning, would pair the static scenario's coverage with the interpretive strength of the AI.
4. **Add a human-in-the-loop and retry strategy.** Given the low success rates, an operator checkpoint or automatic retry on incomplete runs would raise dependability to a level acceptable for practical use, and would mitigate the silent "ping-sweep-only" failures. While partly implemented in we omitted the branch, because the agent always tried to use just the recommended tools and did not deliver a more sophisticated result.

Mention RAG systems

add section in AI implementation with middleware and dynamic tool call

7.7. Answering the Research Questions

This section returns to the three research questions posed in Chapter 1 and answers each one in light of the evaluation.

RQ1 – Agentic AI versus a static reference scenario. The agentic AI scenario matches the static reference scenario on raw discovery, extracting the identical hosts and services in the Containerlab, but on no run does it exceed the baseline in speed or determinism. The advantage of the AI is qualitative rather than quantitative. It extracts cleartext credentials, reasons about the printer banner mismatch, and produces a severity-rated assessment with follow-up steps that the deterministic scenario cannot. Agentic AI therefore does not replace deterministic enumeration but adds an interpretive layer on top of it. The two smaller models, by contrast, fall short of the static baseline on both coverage and trustworthiness.

RQ2 – Operator support during interactive reconnaissance. Agentic AI supports the operator primarily by condensing raw scan output into a structured, severity-rated report with concrete recommendations and a proposed next step, which lowers the depth of expertise required to interpret the results. Additionally, the scenario can be run as a guided reconnaissance by setting the --interactive flag. In this mode the agent does not complete the reconnaissance on its own. The agent relies on the human interaction hook, a LangChain tool that lets the agent pause its reasoning loop, present its intermediate findings and proposed next step 5.3.3. The operator's answer is used as additional input in the total context for the next observation, keeping

the operator in charge while the agent handles the tool execution. The flexibility to choose the right support level based on the situation and the operator's knowledge, makes agentic AI a useful assistant. This support is limited by model capability and trust, the false classification as honeypot and the fabricated CVE show that teh operator must still be alert and able to categorize findings correctly.

RQ3 – Suitability for real-world deployment. The approach scales to a real network. In the Cyberlab BFH segment (≈35 live hosts), the multi-agent frontier model discovered on average 28 hosts, 53 services, 52 findings, and consistently surfaced the six high-severity structural risks. Deployment readiness is limited by reliability, and only the frontier model performed. 7.5 The implementation is therefore suitable for supervised, observed use on real networks, but not yet for unsupervised autonomous operation. Closing this gap requires the retry and human-checkpoint strategy outlined in Section 7.6, together with attention to the safety considerations that tool-using agents raise in production environments.

8. Conclusion

8.1. Personal Section

8.1.1. Frank

In this project, I gained experience configuring a network in a virtual, containerized environment using ContainerLab. The simplicity of deploying and reconfiguring the environment is particularly impressive. I had the opportunity to implement a file server, DNS, a firewall, and other services, which motivated me to further pursue building a home cyber lab. I also benefited greatly from working with and researching various state-of-the-art papers, models, and the LangChain framework. Personally, I prefer using LLMs mainly for reasoning and recommendations. The most surprising aspect for me was the difficulty of deploying the NSAK autonomously in the NSLab. I had expected the execution process to be much more seamless in completing the task. I also appreciated the project management tools and ceremonies, as they help structure the work, although they introduce some overhead and are not ideally suited for a part-time bachelor's project. In particular, the constraint of working with fixed iterations in GitLab did not align well with our approach of two blocked focus weeks. Overall, I developed a deep respect for well-written academic papers. Writing professionally and maintaining a consistent "red thread" throughout an entire thesis is quite challenging.

8.1.2. Lukas

Initially very skeptic of the actual capabilities and usefulness of LLMs, a year ago I would not have guessed that I will write an AI focused bachelors thesis. This changed drastically with the rapid improvement of the models and especially the rise of AI agents. Suddenly, combining the newly found interest with our previous work seemed to be the logical choice, and the implementation of the agent into NSAK kick-started my journey into the topic. It felt like putting together a puzzle of all the stuff we learned at BFH and beyond. Of course, we repeatedly made the rookie mistake of underestimating tasks and documentation. On the other hand, I think this kind of naivety leads to walking the extra mile, instead of limiting the scope. For me the biggest challenge was to keep the focus on the goals and research questions instead of implementing *just another thing* to improve the overall setup. I second

Frank's statement about scientific writing — It is very time-intensive and difficult to keep up the motivation for achieving an overall high quality. Working together with Frank was a pleasure and I think we did leverage the strengths and compensate the weaknesses of one another. The experience has convinced me that AI is here to stay as one more amazing engineering tool.

8.2. Technical Conclusion

The goal of this thesis was to integrate agentic AI into the NSAK framework and assess how the agent can strengthen network security beyond static scenarios. Using the NSAK directly as an agentic harness proved effective, allowing the agent to use a set of *LangChain Tools* to autonomously and interactively execute network discovery. The evaluation showed that agentic AI complements deterministic enumeration.

Scripted scenarios excel in speed and reproducibility, and the agent adds reasoning and assessment, limited only by the model capabilities. Its main strength lies in its interactive mode, where additional input from the operator can guide and refine the investigation process.

One key finding is that multi-agent task decomposition improves the performance of smaller models by reducing context complexity and increasing consistency. Despite these advances, hallucinations remain a limitation, making supervised operation with a human reviewer the proposed deployment model.

8.3. Future Work

The implementation and evaluation surfaced several directions worth investigating beyond the scope of this thesis.

- **Dynamic tool calling.** Exposing only the tools relevant to the current task instead of the full set would reduce context bloat and improve the reasoning of smaller models. While partly implemented on branch 310 in the project, but rejected because of time limitations.
- **Human-in-the-loop middleware.** Replacing the custom human interaction hook with LangChain human in the loop middleware to introduce a robust way to approve or reject a tool call. Introduced also on branch WIP branch 310, but the agent unfortunately is prone to use every tool which is presented in this way even if the tool is not necessary to fulfill the task. We decided not to merge the changes in this state, but recommend for the next steps.

- **Guardrails and security boundaries.** Explicit scope restriction and sandboxing would prevent the agent from acting outside of the boundaries and would be crucial to use the framework in a productive environment.
- **Broader attack-chain coverage.** Extending the agent beyond reconnaissance to later phases of the attack chain. Pivot from human against agent to human enhancing multi agents that build a sophisticated attack chain with switch from human implementation to agentic support seamlessly.

Bibliography

[1] World Economic Forum. The global risks report 2026, 2026.

[2] CrowdStrike. Global threat report 2025, 2025.

[3] Muriel Figueredo Franco, Fabian Künzler, Jan von der Assen, Chao Feng, and Burkhard Stiller. Rcvar: An economic approach to estimate cyberattack costs using data from industry reports. *Computers & Security*, 2023.

[4] Polina Zilberman, Rami Puzis, Sunders Bruskin, Shai Shwarz, and Yuval Elovici. Sok: A survey of open-source threat emulators. arXiv preprint, 2020.

[5] Blake E. Strom, Andy Applebaum, Doug Miller, Adam Nickels, Cody Pennington, and Chris Thomas. Mitre att&ck: Design and philosophy. Technical report, MITRE Corporation, 2020.

[6] Bader Al-Sada, Alireza Sadighian, and Gabriele Oligeri. Mitre att&ck: State of the art and way forward. *ACM Computing Surveys*, 2023.

[7] MITRE Corporation. Mitre caldera documentation, 2024.

[8] Red Canary. Atomic red team, 2024.

[9] Rapid7. Metasploit framework documentation, 2024.

[10] David Kennedy, Jim O'Gorman, Devon Kearns, and Mati Aharoni. *Metasploit: The Penetration Tester's Guide*. No Starch Press, 2011.

[11] Frank Gauss and Lukas von Allmen. Nsak (network swiss army knife). https://github.com/nsak-team/nsak, 2026. Accessed: 2026-01-10.

[12] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2):1–124, 2023.

[13] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020.

[14] Anthropic. Model context protocol. https://www.anthropic.com/news/model-context-protocol, 2024. Accessed: 2026-03-18.

[15] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. Model context protocol (mcp) at first glance: Studying the security and maintainability of mcp servers, 2025.

[16] K Huang. Define ai agent. In Ken Huang, editor, *Agentic AI: Theories and Practices*. 2025.

[17] K Huang and J Huang. Define ai agent. In Ken Huang, editor, *Agentic AI: Theories and Practices*. 2025.

[18] J Huang, K Huang, K Jackson, and C Hughes. Ai agent safety and security considerations. In Ken Huang, editor, *Agentic AI: Theories and Practices*. 2025.

[19] Risky Business Feature Podcast. ...mcp is dead. https://risky.biz/, 2026. Podcast episode , Accessed: 2026-03-18.

[20] Ethan Michalak and Mark Perry. Connecting llms to caldera with the mcp plugin. https://medium.com/@mitrecaldera/connecting-llms-to-caldera-with-the-mcp-plugin-da1450254827, 2025. MITRE Caldera Blog, Accessed: 2026-03-16.

[21] GH05TCREW. Metasploit mcp server. https://github.com/GH05TCREW/MetasploitMCP, 2025. Accessed: 2026-03-16.

[22] Hare Sudhan. Atomic red team mcp server. https://github.com/cyberbuff/atomic-red-team-mcp, 2025. Accessed: 2026-03-16.

[23] LangChain AI. Langchain: The agent engineering platform. https://github.com/langchain-ai/langchain, 2026. Accessed: 2026-05-02.

[24] LangChain Inc. Langgraph overview, 2026.

[25] LangChain Inc. Deep agents overview, 2026.

[26] LangChain. Structured output, 2026. LangChain Python Documentation.

[27] Ajay Bandi, Bhavani Kongari, Roshini Naguru, Sahitya Pasnoor, and Sri Vidya Vilipala. The rise of agentic ai: A review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges. *Future Internet*, 17(9), 2025.

[28] Anthropic. Model temperature, 2026. Accessed: 2026-05-16.

[29] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (mcp): Landscape, security threats, and future research directions. *ACM Trans. Softw. Eng. Methodol.*, February 2026. Just Accepted.

[30] Advisera. Iso 27001 risk assessment, treatment, and management: The complete guide, 2025. Accessed: 2026-03-04.

[31] Simplr. General recommended vram guidelines for llms, 2025. Online; accessed 10 March 2026.

List of Figures

List of Tables

List of Listings

Glossary

This document is incomplete. The external file associated with the glossary 'main' (which should be called `thesis.gls`) hasn't been created.

Check the contents of the file `thesis.glo`. If it's empty, that means you haven't indexed any of your entries in this glossary (using commands like `\gls` or `\glsadd`) so this list can't be generated. If the file isn't empty, the document build process hasn't been completed.

If you don't want this glossary, add `nomain` to your package option list when you load `glossaries-extra.sty`. For example:

```
\usepackage[nomain]{glossaries-extra}
```

Try one of the following:

- Add `automake` to your package option list when you load `glossaries-extra.sty`. For example:

  ```
  \usepackage[automake]{glossaries-extra}
  ```

- Run the external (Lua) application:

  ```
  makeglossaries-lite.lua "thesis"
  ```

- Run the external (Perl) application:

  ```
  makeglossaries "thesis"
  ```

Then rerun LaTeX on this document.

This message will be removed once the problem has been fixed.

A. Project Goals

For a goal to be considered **S.M.A.R.T.**, it must fulfill the following criteria:

- **Specific**: What do we want to achieve?
- **Measurable**: What are the success criteria?
- **Achievable**: How do we plan to achieve this goal?
- **Relevant**: Why is it important to achieve this goal?
- **Time-bound**: Goals categorized as *must* have to be achieved within the thesis, while *should* or *could* goals are considered optional or nice to have.

A.1. Research Goals

A.1.1. Compare Frameworks (must)

To identify the USP (unique selling point) of the NSAK framework, we will analyze and compare existing security emulation frameworks. First, relevant frameworks such as MITRE Caldera and Atomic Red Team will be identified and reviewed. Based on this analysis, the identified concepts will be compared with the architecture of the NSAK framework. The comparison will focus on how scenarios and drills are defined, how configuration and parameter handling are structured, how reusable components are organized, how automation is achieved, and which AI capabilities they provide.

The analysis and comparison will be based on the following properties:

- Concepts
- Modularity
- Configurability
- Automation
- AI Capabilities

Success criteria:

- At least two security emulation frameworks are identified and reviewed
- A matrix comparing the specified properties of the selected frameworks is created
- The USP of the NSAK framework is identified

A.1.2. AI Integration Research (must)

The goal of this research is to investigate how AI (artificial intelligence) can support automated security testing and adversary emulation. The research evaluates potential integration points where AI could enhance the scenario execution, automation, decision-making, and analysis capabilities of the NSAK framework.

Success criteria:

- How is AI currently used in cybersecurity automation?
- Which tasks in adversary emulation can be supported or automated by AI?
- How could AI interact with the NSAK scenarios and drills (e.g., Agents, MCP)?
- What architectural changes would be required to integrate AI into NSAK?

A.2. Framework

A.2.1. Configuration Management (must)

The goal is to extend the NSAK framework, so that scenarios and drills can expose configurable parameters to increase the flexibility of scenarios and reusability of drills. Parameters can be provided via command-line arguments or YAML files and are resolved by the framework into a unified run configuration used for scenario execution.

Success criteria:

- Scenarios and drills can expose configurable parameters
- The available parameters can be listed by the CLI with the --help option
- Parameters can be provided via CLI arguments or YAML files

- The framework resolves the inputs into a run configuration (internal data structure).

A.2.2. REST API (could)

Adding a REST API to the NSAK framework was already proposed in the predecessor project, as it greatly enhances interoperability with other tools. The priority of this goal depends heavily on the outcome of the goal A.1.2 and on whether we want to fulfill the goal A.2.4, which is prioritized as “could”. If the evaluated AI integration requires a REST API (e.g., MCP), this goal needs to be prioritized as “must”; it will be prioritized as “could”.

Success criteria:

- A REST API specification with feature parity to the CLI is created.
- The REST API is implemented according to the specification.
- The API is documented (OpenAPI) and usable by external clients.
- The REST API remains optional and does not replace the CLI-based interaction.
- Authentication and Authorization are explicitly out of scope.

A.2.3. MCP Server (could)

The MCP Protocol allows the AI to interact with applications in a standardized way - and could therefore be necessary for implementing AI-based scenarios. If the outcome of the research task A.1.2 highlights the necessity to specify/expose an MCP-Server to interact seamlessly with the NSAK framework, the implementation is a (must), otherwise a (could).

- A MCP-Server is specified and implemented
- Documentation how to connect to the MCP is provided.
- A form of authentication is provided.

A.2.4. Web GUI (could)

The goal is to design and implement a web-based graphical user interface that allows users to interact with the NSAK framework in a user-friendly way. The Web GUI will enable management of scenarios and drills, triggering scenario execution, and visualization of execution results and system status.

Success criteria:

- A simple Web GUI that interacts with the REST API has been implemented.
- The Web GUI displays a list of available scenarios and their states.
- The scenario's results can be viewed on the Web GUI
- Scenarios can be configured and executed via the Web GUI
- Authentication and authorization are explicitly out of scope.

A.3. Scenarios

A.3.1. AI Scenarios (must)

The goal is to implement artificial intelligence according to the result of A.1.2 and to explore the use of AI to support red, blue, and purple team exercises in the NSAK framework. The AI will be evaluated for its potential to assist in executing attacks in a scenario and analyzing detection results. The findings will help assess how AI could enhance and coordinate red and blue team activities across a network via the NSAK device.

Success criteria:

- The AI is integrated into NSAK as a scenario or as part of the framework, depending on the result of A.1.2.
- The AI can execute CLI or Python tools to directly or indirectly interact with the attached network.
- An operator can interact with or stop the running AI.
- The result artifacts are stored and can be presented or reused for further scenarios.

A.3.2. Reproducible Test Environment (must)

The goal is to design and implement a reproducible and safe test environment for the NSAK framework that enables consistent execution and evaluation of scenarios and drills. The environment should provide a stable basis for testing and comparison of manual and AI-assisted scenarios.

Success criteria:

- The test environment can be set up reproducibly using a defined configuration or setup procedure with Infrastructure as code (e.g., Ansible)
- NSAK scenarios and drills can be executed consistently within the environment.
- The environment supports both manual and AI-assisted scenarios.
- Scenario results can be collected and analyzed for evaluation purposes.

A.3.3. Network Discovery – Red Team (must)

The goal is to implement a reference red team scenario for network discovery within the NSAK framework. The scenario should simulate reconnaissance activities to identify hosts, services, and network characteristics, and the results should be comparable to the AI-Agent approach.

Success criteria:

- A red team network discovery scenario is implemented in the NSAK framework.
- Discovered network information is collected as artifacts.
- The scenario can be executed reproducibly within the test environment.
- The results can be compared with those generated by the AI-assisted approach.

A.3.4. Intrusion Detection – Blue Team (should)

The goal is to define and implement a blue team reference scenario for intrusion detection within the NSAK framework. The scenario should allow observation and evaluation of suspicious network activity to assess detection capabilities in the test environment and compare the results with those of the AI approach.

Success criteria:

- A blue-team intrusion-detection scenario is implemented in the NSAK framework.
- Detection events or alerts can be observed and documented during scenario execution.
- Detection results can be compared with results from AI-assisted scenario execution.

A.4. Evaluation Goals

A.4.1. AI Scenario Evaluation (must)

The objective is to determine the effectiveness of the AI-based scenario introduced by the goal A.3.1 within the network environment defined by the goal A.3.2 The evaluation will assess whether the AI scenario successfully executed the expected red and blue team activities for the provided prompts and the quality of the results.

Success criteria:

- A methodical approach for the assessment is evaluated and documented.
- The results of the AI scenario for red and blue team activity are assessed.
- There is a conclusion on the effectiveness of the AI scenario for red and blue team activities.

A.4.2. AI vs Engineered Scenarios (must/should)

The goal of this evaluation is to compare the quality of AI-based red (network reconnaissance) (must) and blue (network intrusion detection) (should) team scenarios with that of manually engineered scenarios.

The comparison focuses on criteria such as execution reliability, reproducibility, and coverage. The results will help determine the potential benefits and limitations of AI-supported scenarios and provide insights into how AI could enhance future versions of the NSAK framework.

Success criteria:

- A methodical approach for the comparison is evaluated and documented.
- The results of the AI scenario are compared against the respective results of the red team scenario (must)
- The results of the AI scenario are compared against the respective results of the blue team scenario (should)
- There is a conclusion on the effectiveness of the AI scenario compared to manually engineered scenarios.

A.5. Out of Scope / Optional Feature

At the project management meeting on 05.03.26 (see Table C.2 in Appendix A.7), the initial project scope was discussed. During planning, the team proposed using AI-based scenarios and comparing them with the implemented versions, which required reassessing certain milestones.

Running AI-based scenarios does not require a REST API or GUI. A reproducible test setup and further research are needed to properly evaluate this approach.

B. Project Management

B.1. Stakeholders

Hj. Wenger: Instructor and Network Specialist
Urs Keller: Expert and Entrepreneur

B.2. Agile Management Process

The project followed an interactive development approach using Gitlab for issue tracking and planning. We defined our project management in a Scrum-like manner, but we individualized the process according to the projects scope and our needs. Work items where organized into epics, milestones and issues. Each iteration lasts two weeks. We decided to hold the following Scrum ceremonies after each sprint:

Sprint Planning	**Sprint Review**	**Sprint Retro**	**Checkpoint**
Select milestone with sprint goal	Close open tickets	Reflect on sprint	Present sprint results
Check issues for DoR and obstacles	Gather team feedback	Identify improvements	Get stakeholder feedback
Ensure issues are clear	Validate delivered features	Discuss what went well	Define next tasks
Create sprint backlog	Review progress	Discuss issues/problems	Discuss issues/problems

Table B.1.: Key Scrum Meetings

The workflow used for ticket tracking consisted of the following stages:

B.2.1. Issue Lifecycle:

Buckets	Description
Backlog	List of all tasks and features planned for the project.
Todo	Tasks selected for the current sprint that are ready to be worked on.
Q&A	Tasks waiting for clarification, review, or testing.
Done	Completed tasks that meet the defined acceptance criteria.

Table B.2.: Issue Board

B.2.2. Definition of Ready (DoR)

Category	Criteria
Clarity	▶ Clear and precise title ▶ Objective or problem described in two to three sentences ▶ Scope clearly defined
Scope	▶ Affected module specified (drill, scenario, core, container, frontend) ▶ External dependencies identified
Validation	▶ Testable acceptance criteria defined ▶ Expected behavior clearly specified

Table B.3.: Definition of Ready Criteria for Tickets

B.2.3. Definition of Done (DoD)

Category	Criteria
Implementation	▶ All acceptance criteria fulfilled ▶ Container execution verified ▶ Pre-commit hooks pass ▶ Logging and error handling consistent
Testing	▶ Feature or bug fix tested ▶ Acceptance criteria verifiable (automated or manual) ▶ Relevant edge cases considered ▶ Execution reproducible
Documentation	▶ README updated if required ▶ Architecture changes documented ▶ Thesis relevance briefly noted ▶ Limitations documented ▶ Required documentation stored for thesis or appendix

Table B.4.: Definition of Done Criteria for Tickets

B.3. Project Structure

This section describes the projects structure and list the goals as milestones linked to their respective page in GitLab.

B.3.1. Milestones

A milestone represent crucial steps toward the overall project goal. Per our definition a milestone has at least one epic and the mandatory milestones are listed in TableB.5

GitLab Milestones
Project Management
Research AI Integration
Research: Compare Frameworks
NSAK Framework: Configuration Management
NSAK Scenario: Network Discovery - Red Team
NSAK Scenario: AI Scenarios
Reproducible Test Environment

Table B.5.: Project Milestone Overview in GitLab

Project Planning

19.02.2026 – 10.03.2026

This is the initial milestone which is used to plan the whole project and bachelor thesis.

- ✓ The bachelor thesis is planned as an agile (scrum-like) project
- ✓ A risk analysis for the project was conducted and the identified risks are assessed and addressed
- ✓ All meetings, deadlines, scrum ceremonies and sprints are added to the Outlook calendar
- ✓ The goals of the thesis are defined and categorized as “must”, “should” and “could”
- ✓ The goals are created as milestones and divided into epics, categorized by the labels: Project Management, Research, Framework, Implementations, Documentation
- ✓ We know how we coordinate the project with the stakeholders (instructor and expert)

Research: AI Integration

10.03.2026 – 22.03.2026

The goal of this research milestone was to investigate how AI can support automated security testing and adversary emulation. The research evaluated potential integration points where AI could enhance scenario execution, automation, decision-making, and analysis capabilities of the NSAK framework. The milestone was ex-

tended by three days following input from the expert and instructor to clarify the intended interaction between LLMs and NSAK.

- ✓ How is AI currently used in cybersecurity automation?
- ✓ Which tasks in adversary emulation can be supported or automated by AI?
- ✓ How could AI interact with the NSAK scenarios and drills (e.g. Agents, MCP)?
- ✓ What architectural changes would be required to integrate AI into NSAK?

Research: Compare Frameworks

10.03.2026 – 19.03.2026

This milestone focused on the research and comparison of existing security automation frameworks that could potentially support or influence the NSAK scenario and drill execution model. The evaluation included frameworks such as MITRE CALDERA and Atomic Red Team, examining their execution models, configuration and parameter handling, extensibility mechanisms, and the structure of reusable test definitions.

The expected outcomes were:

- ✓ A short comparison of selected frameworks
- ✓ A summary of relevant architectural concepts
- ✓ An initial assessment of how these frameworks relate to NSAK

NSAK Framework: Configuration Management

10.03.2026 – 19.03.2026

This milestone defined the configuration model and execution framework for NSAK scenarios. It introduced a structured approach for configuring scenarios and drills, including typed argument declarations, and the generation of a resolved run configuration. The goal was to provide a flexible and reproducible configuration system that allows scenarios and drills to expose parameters and execute in a defined order.

- ✓ Drills and scenarios can expose runtime parameters as configuration objects
- ✓ The configuration objects can be set via CLI arguments
- ✓ A configuration management system is implemented in the NSAK framework

NSAK Scenario: Network Discovery – Red Team

02.04.2026 – 16.04.2026

Network discovery is one of the most essential Red Team activities in the reconnaissance phase. This milestone covered the implementation of a scenario that identifies network participants and services, producing a structured network map that can be consumed by subsequent drills such as Address Resolution Protocol spoofing.

Goals (Must):

- ✓ The scenario identifies the available ports and interfaces on the device
- ✓ The scenario discovers the available subnets and obtains valid addresses
- ✓ The scenario discovers addresses, addresses and services (port scan)
- ✓ The scenario supports at least a fast full-search mode
- ✓ The scenario returns a network map as a structured data type

Goals (Could):

- ☐ A human interaction hook allows the operator to select the discovery mode
- ☐ Additional modes: slow full search, distinct search

NSAK Scenario: AI Scenarios

19.03.2026 – 16.04.2026

This milestone explored what happens when an AI agent is given full access to the NSAK framework for both Red and Blue Team activities. The scenario is intentionally open-ended, reflecting the research nature of the milestone.

Goals (Must):

- ✓ The operator can provide a custom prompt or select a predefined Red or Blue Team prompt
- ✓ The scenario runs an AI agent with a remote connection
- ✓ An operator can interact with the agent via CLI at all times
- ✓ The scenario logs all relevant actions and network traffic locally
- ✓ The AI can trigger an e-mail alert to notify an operator
- ✓ The scenario has a kill switch to stop execution at any time

Goals (Should):

- ☐ Human interaction hook via Web GUI

- ☐ Remote logging and sniffing
- ☐ Kill switch with full device shutoff

Reproducible Test Environment

19.03.2026 – 02.04.2026

The goal of this milestone was to design and implement a reproducible and safe test environment for the NSAK framework, providing a stable basis for consistent execution and evaluation of both manual and AI-assisted scenarios.

- ✓ The test environment can be set up reproducibly using Infrastructure as Code (e.g. Ansible).
- ✓ NSAK scenarios and drills can be executed consistently within the environment.
- ✓ The environment supports both manual and AI-assisted scenarios.
- ✓ Scenario results can be collected and analyzed for evaluation purposes.

B.3.2. Epics

consist of multiple stories and contribute to fulfilling milestones.

B.3.3. Issues

Each Story is assigned a weight, sprint, milestone and epic and are usually represented by an “Issue” in gitlab.

B.4. Sprints / Iterations

B.4.1. Planning

“Task” and “Issue”, the terms are often used interchangeably with “Stories”. Stories must meet the B.2.2 before implementation and B.2.3 before closing.

B.5. Planning and Estimation

The following estimation strategy was used:

- 1 hour corresponds roughly to 1 weight point
- Iteration length: 2 weeks
- Weights per iteration: 40h per person = 80 weight per iteration
- Sprint planning duration: 1.5 hours per iteration
- Sprint review duration: 1 h per iteration
- Sprint retro duration: 1 h per iteration

B.6. Project Roadmap

To keep track of the overall progress in the project, the project team uses the “Roadmap” feature in GitLab. After each sprint planning meeting we create a new screenshot of the roadmap and add it to the according sprint section in B.8. The first sprint planning is an exception, because at this point the diagram would have been empty.

B.7. Risk Management

Every project is exposed to uncertainties and risks that can influence the outcome of objectives. A risk assessment makes them explicit and addressable, and even defines countermeasures that significantly lower the risk factor. Systematically identifying roadblocks early on can be game-changing for a project.

B.7.1. Risk Assessment

To methodically conduct the risk assessment the section “Risk assessment” from an online guide published by Advisera [30], which describes the risk assessment process according to ISO 27001, was used as a reference.

Risk Assessment Process:

1. Risk categories and criteria: Define which dimensions and taxonomies are important for the assessment.
2. Risk identification: Collect possible risks in a brainstorming session.
3. Risk analysis: Assess the risks criteria on a risk matrix.
4. Risk evaluation: Create a list of risks ordered by the resulting risk level and evaluate if they are acceptable or need treatment.
5. Risk treatment: Describe the strategy and how to treat the risks.

Risk Categories & Criteria:

The table B.6 shows four commonly used categories, which where used to group the risks.

Category	**Description**
Strategic Risk	Risks related to business decisions (e.g., wrong prioritization).
Operational Risk	Risks related to internal processes (e.g., Scrum, QA process).
Financial Risk	Risks related to costs and expenses (e.g., hardware costs).
External Risk	Risks related to uncontrollable sources (e.g., force majeure).

Table B.6.: Risk categories used for the project risk assessment

The criteria for analysing the risks are summarized in the table B.7

Criterion	Description
Probability	Likelihood that the risk will occur.
Impact	Severity of consequences if the risk materializes.
Level	Risk level calculated as Probability + Impact.

Table B.7.: Risk evaluation criteria

Level	Weight	Description
Low	1	Unlikely
Medium	2	Possible
High	3	Likely

Table B.8.: Risk probability scale

The levels of the criteria and their respective numerical weights are listed in the tables B.8 and B.9.

The following table B.10 defines the calculation method and the resulting risk levels, used for the assessment.

Risk Identification

The identification process was conducted with the help of a digital brainstorming, where the risks are already grouped by category. The identified risks are represented by the cards shown in B.1. The respective risk categories are visualized by the color coding of the cards and described in the legend.

Level	Weight	Description
Low	1	Minor inconveniences
Medium	2	Delays milestones or reduces scope
High	3	Threat to the thesis

Table B.9.: Risk impact scale

Risk Level	Weight	Calculation
Very Low	2	Low + Low
Low	3	Low + Medium
Medium	4	Medium + Medium / Low + High
High	5	Medium + High
Critical	6	High + High

Table B.10.: Risk level classification (probability and impact are interchangeable)

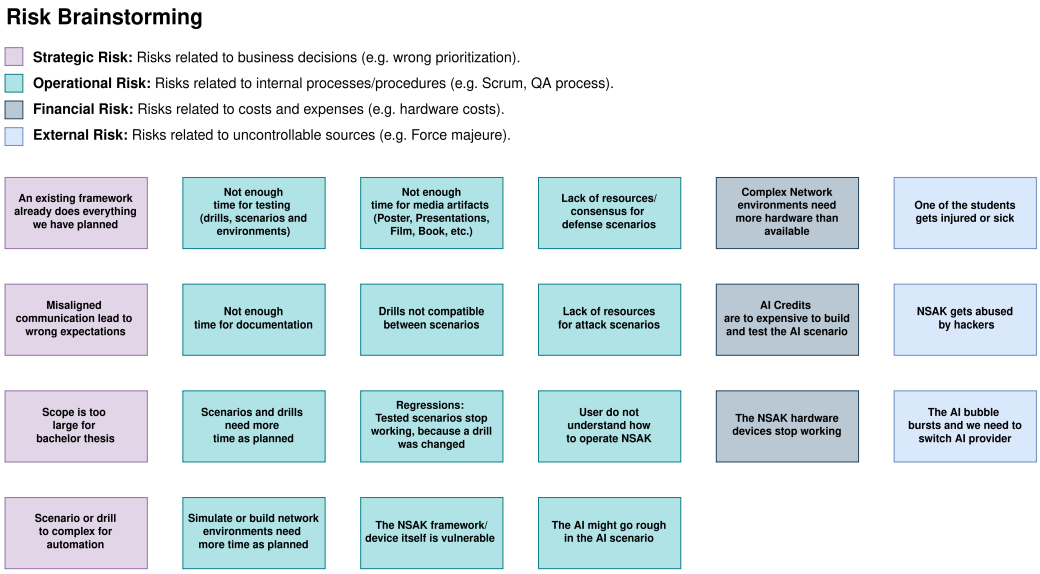


Figure B.1.: Risks Brainstorming

B.7.2. Risk Analysis

For analysing the risks a matrix was prepared, where the criteria “Impact” and “Probability” are representing the x- and y-axis respectively. The project team then assessed the criteria for each individual risk and placed them on the matrix accordingly. In a second step, the risks where moved if they did not fit relatively to each other. The resulting “Risk Analysis Matrix” is illustrated in the following figure B.2.

The background color of the cells is representing the resulting risk level of the tasks.

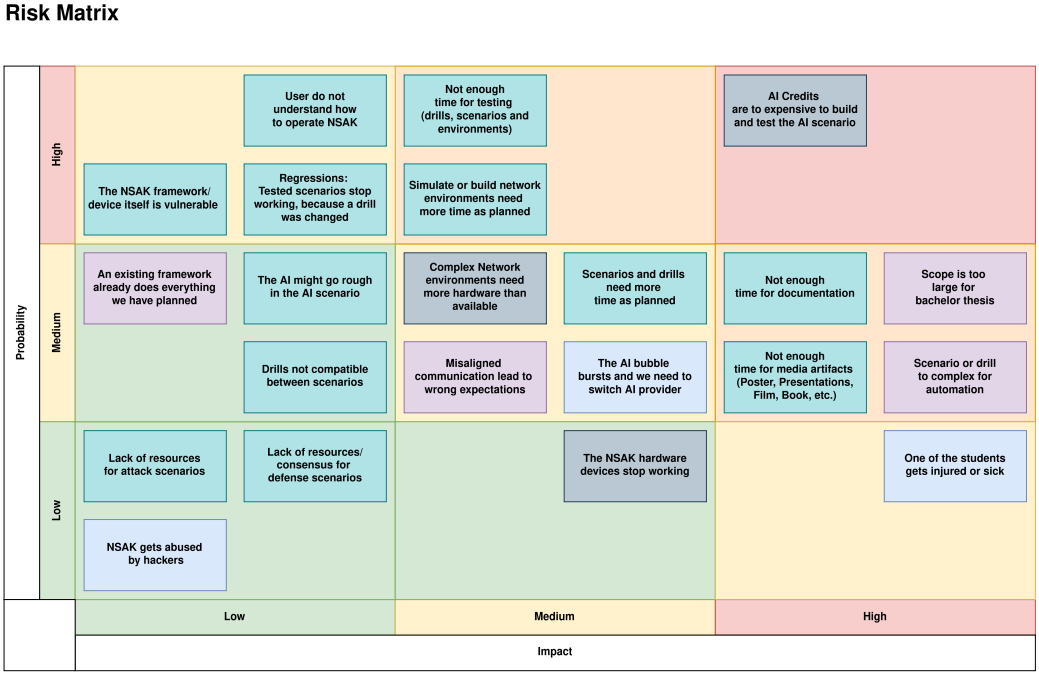


Figure B.2.: Risks Brainstorming

B.7.3. Risk Evaluation & Treatment

In a first step a table with the required columns was created. Then the risks and weights for the criteria where added with an ID in the format “R-000”, to easier reference them in the thesis later on. After sorting the table by the calculated risk level, the risk evaluation was conducted by deciding which treatment strategy described in B.11 we want to apply.

Strategy	Description
Mitigate	Reduce the likelihood or impact of a risk.
Transfer	Move the risk to a third party.
Accept	Acknowledge the risk and take no further action.
Avoid	Eliminate or circumvent the cause of the risk.

Table B.11.: Risk Treatment Strategies

Finally, the project team went through the risks and discussed how to treat them effectively. The resulting “Risk Assessment” is illustrated in the following table B.3.

Risk Assessment

Criteria	Level
1 = Very Low	2 = Low
3, 4 = Medium	6 = High
9 = Critical	

Category
Strategic
Operational
Financial
External

Risk ID	Description	Risk Category	Probability	Impact	Level	Strategy	Treatment
R-001	AI credits are too expensive to build and test the AI scenario	Financial	3	3	9	Avoid	Use a self-hosted LLM, check if the BRH can provide AI services or if there are free plans for education
R-006	Scope is too large for bachelor thesis	Strategic	2	3	6	Mitigate	We have to plan and evaluate the goals carefully. The use of "User Story Maps" also helps to mitigate this risk.
R-007	Scenario or drill too complex for automation	Strategic	2	3	6	Mitigate	Create "User Story Maps" for epics, so that we have a good understanding of the scenarios and their drills.
R-002	Not enough time for documentation	Operational	2	3	6	Mitigate	Add a dedicated documentation issue with an according weight to each epic.
R-003	Not enough time for testing (drills, scenarios and environments)	Operational	3	2	6	Mitigate	Add a dedicated testing issue with an according weight to each epic.
R-004	Simulating or building network environments needs more time than planned	Operational	3	2	6	Mitigate	Add dedicated issues with according weight to each scenario epic.
R-005	User does not understand how to operate NSAK	Operational	3	2	6	Mitigate	Provide a Web GUI and/or user guides
R-012	Misaligned communication leads to wrong expectations	Strategic	2	2	4	Mitigate	Multiple channels and Bi-Weekly Meetings should keep the communication aligned.
R-010	Scenarios and drills need more time than planned	Operational	2	2	4	Mitigate	Use "User Story Maps" to get a good idea of the required steps to build the scenario, it's drills and estimate generously.
R-011	Not enough time for media artifacts (poster, presentations, film, book, etc.)	Operational	2	2	4	Mitigate	Include the creation of the media artifacts as milestones/epics in the project planning.
R-009	Complex network environments need more hardware than available	Financial	2	2	4	Avoid	We can simulate network environments instead of real hardware.
R-008	The AI bubble bursts and we need to switch AI provider	External	2	2	4	Avoid	Use vendor agnostic tools and protocolls, so that we can switch provider easily.
R-014	Regressions: tested scenarios stop working because a drill was changed	Operational	3	1	3	Mitigate	Add automated test cases for detecting regressions early.
R-015	The NSAK framework/device itself is vulnerable	Operational	3	1	3	Transfer	Clearly state that the NSAK framework/device is still a POC and not ready for production use.
R-013	One of the students gets injured or sick	External	1	3	3	Accept	We accept this risk.
R-019	An existing framework already does everything we have planned	Strategic	2	1	2	Accept	We accept the risk, but will compare well known frameworks with NSAK and highlight our USP (HW and AI).
R-017	Drills not compatible between scenarios	Operational	2	1	2	Mitigate	The implementation of the configuration management should provide a way that small deviations can be branched.
R-018	The AI might go rogue in the AI scenario	Operational	2	1	2	Mitigate	Besides the fact that we have physical access to the device we add a killswitch to the AI scenario.
R-016	The NSAK hardware devices stop working	Financial	1	2	2	Accept	The fact that we have two devices already reduces the impact, further we could also use other HW.
R-021	Lack of resources for attack scenarios	Operational	1	1	1	Mitigate	Get familiar and use the MITRE ATT&CK framework as reference and source.
R-022	Lack of resources / consensus for defense scenarios	Operational	1	1	1	Mitigate	Get familiar and use well known sources such as NIST as source.
R-020	NSAK gets abused by hackers	External	1	1	1	Transfer	Clearly state that the NSAK framework/device is built only for educational and white hate purposes.

Figure B.3.: Risk Assessment

B.8. Sprint Ceremonies

B.8.1. Sprint 1: 19.02 –10.03.2026

Sprint Planning 1: 19.02.2026

For the planning of sprint 1 we derived all issues from the milestone project management B.5 and had an overall weight from 89.

Sprint Review 1: 10.03.2026

The milestone was successfully reached, and the board was cleared in preparation for the next sprint. All issues were reviewed and moved to the “Done” column.

Milestone Review: Project Management As shown in burndown and burnup charts B.4 of the Project Planning B.3.1 milestone, all deliverables were fulfilled. However, the deadlines of the milestone had to be moved to the 10.03.2026, because the project planning required more time than initially expected. It should also be noted that the charts abruptly go down or up respectively, as we closed alot of the tickets after we reviewd them together just before we held this meeting.

Figure B.4.: Checklist and Burndown Sprint1

Sprint Retro 1: 10.03.2026

This sprint retro was held in Microsoft Whiteboard as it provides simple templates, the result is illustrated in B.5.

Key Takeaways:

- Maintain clear and consistent communication with the instructor.
- Be aware of maintaining a healthy work–life balance during the project.
- Apply lean project management practices to keep the process efficient and focused.

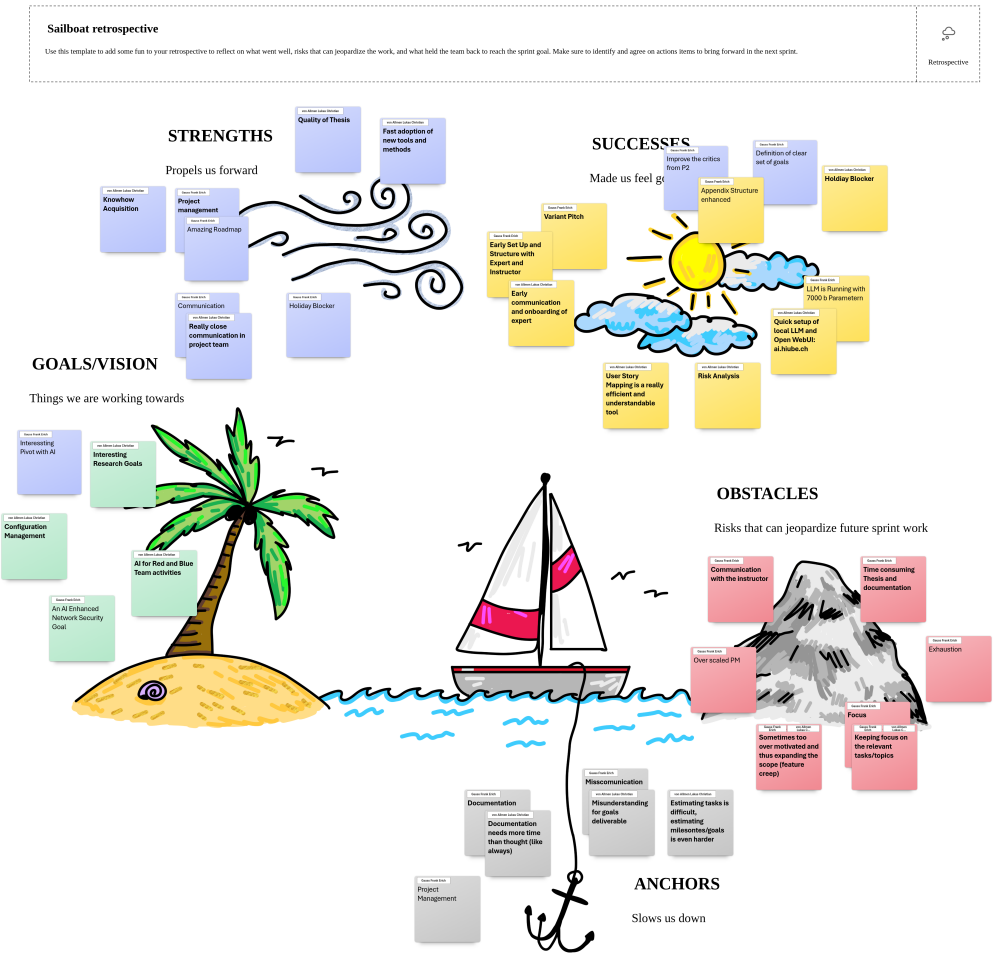


Figure B.5.: Sprint Retro 1

B.8.2. Sprint 2: 05.03 –18.03.2026

Sprint Planning 2: 05.03.2026

The current state of the project roadmap is shown in the following screenshot B.6.

Week	Responsible	Task
Week 1	Lukas	Configuration management implementation
Week 1	Frank	Research and comparison of frameworks
Week 1	Team	AI workshop on Thursday and writing of the related issues
Week 2	Team	AI research and evaluation
Week 2	Team	Preparation and pitch of the project deliverables

Table B.12.: Sprint Planning 2

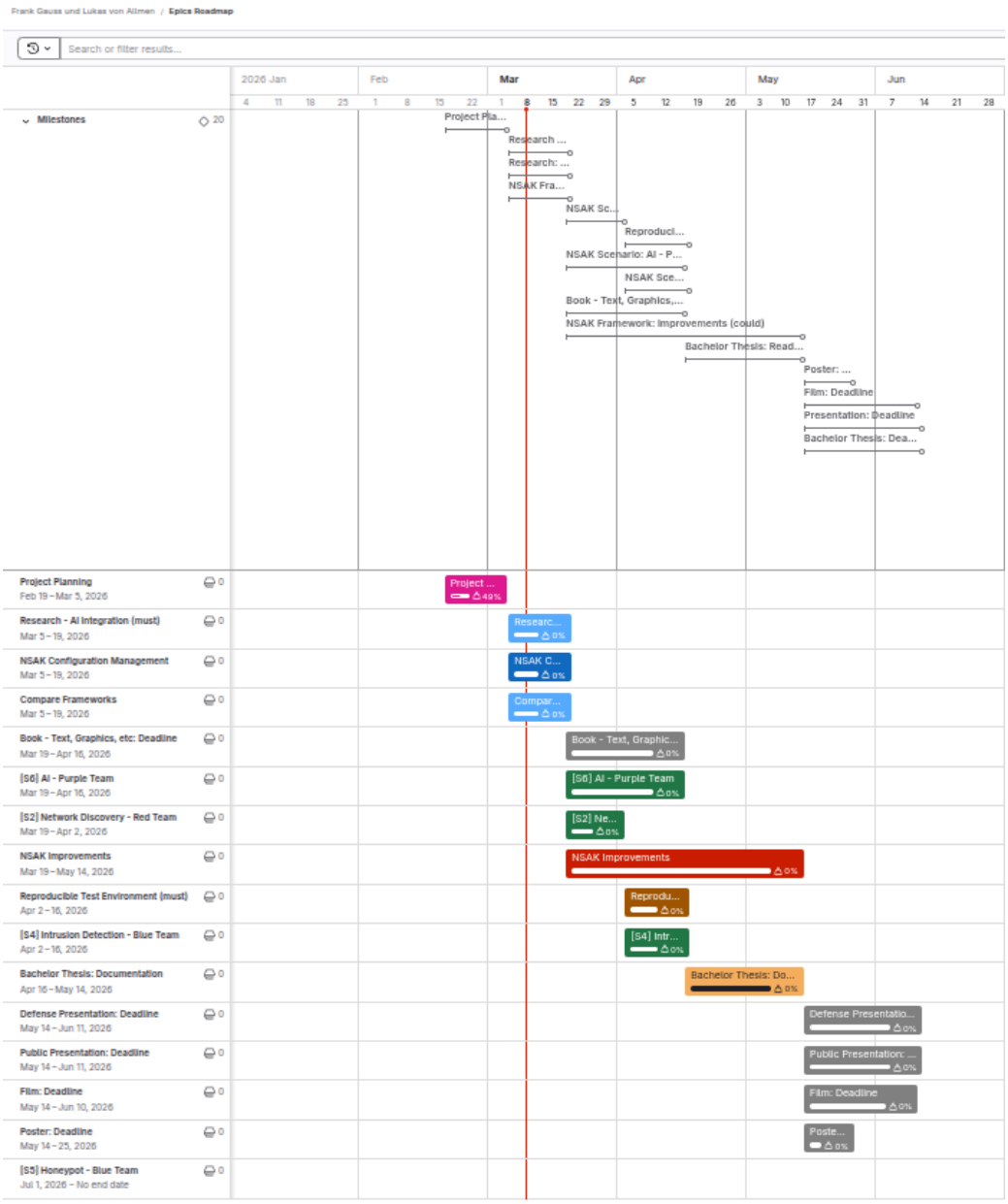


Figure B.6.: Roadmap Sprint Planning 2

Sprint Review 2: 19.03.2026

We got some helpful inputs related to both milestones from the tutor and expert in the checkpoint meeting C.0.3 earlier this day. We decided to still consider the milestone to be successfully reached as the planned work package and deliverables were finalized and presented. The resulting follow-up tasks are planned for the next week

and as a consequence we will hold the next sprint planning B.8.3 a week later than planned. All issues were reviewed and moved to the “Done” column in preparation for the next sprint. It must be noted, that because we usually review and close the tickets at the end of the sprint, the charts are not representative in most cases.

Milestone Review: Compare Frameworks The Compare Frameworks B.3.1 milestone is completed as illustrated in B.7.

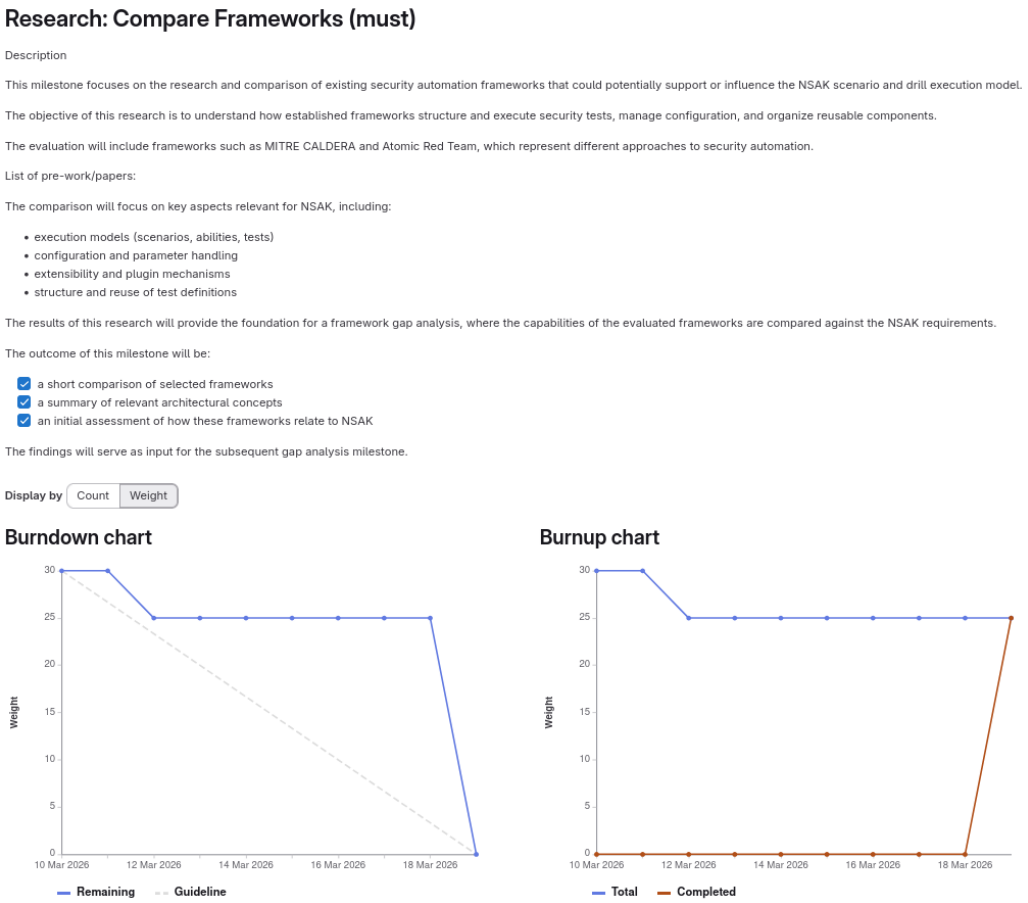


Figure B.7.: Milestone Description and Charts: Compare Frameworks

Milestone Review: Research AI-Integration As shown in B.8, the milestone Research AI-Integration B.3.1 is completed.

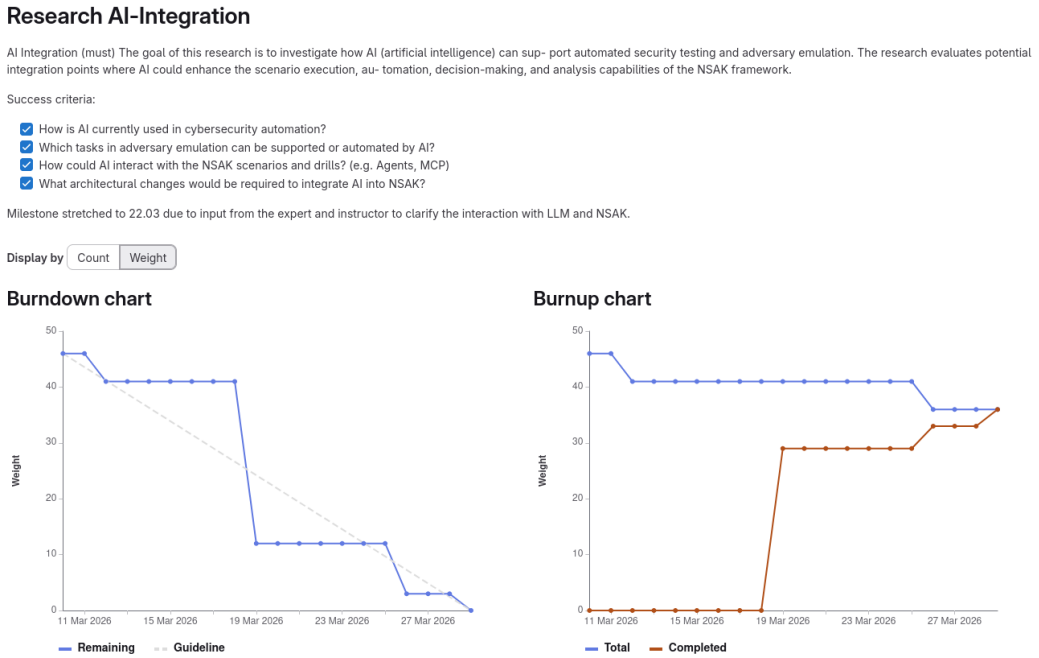


Figure B.8.: Milestone Description and Charts: Research AI-Integration

Milestone Review: Configuration Management The burnup and burndown charts below B.9 are showing that the milestone Configuration Management B.3.1 is implemented as planned.

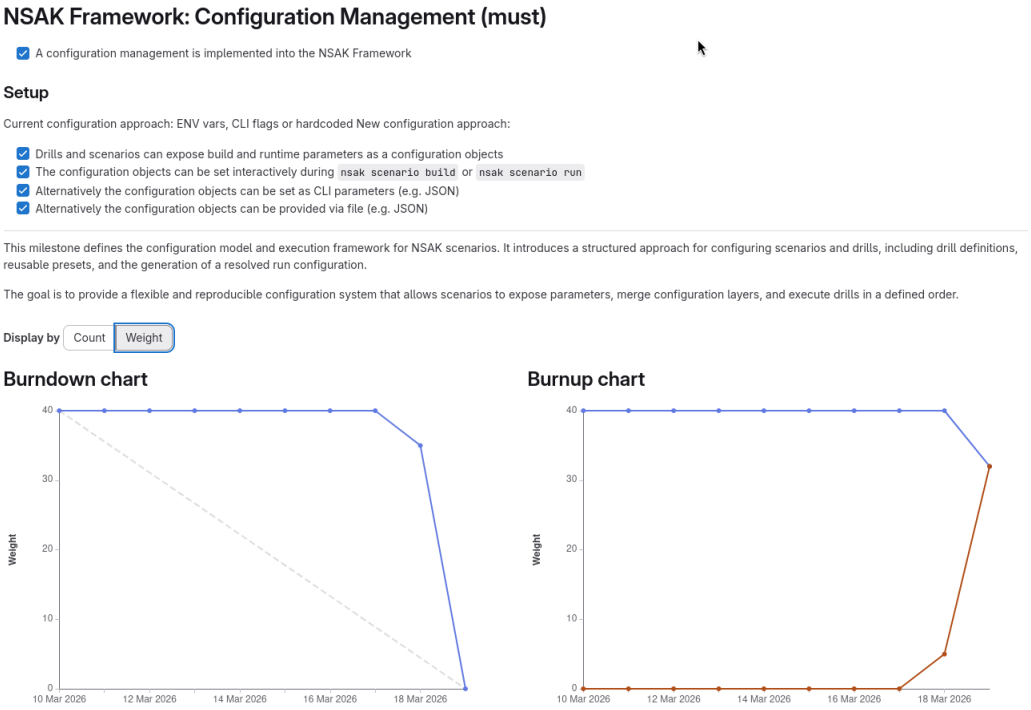


Figure B.9.: Milestone Description and Charts: Configuration Management

Sprint Retro 2: 19.03.2026

This sprint retro was held in Microsoft Whiteboard as it provides simple templates, the result is illustrated in B.10.

Key Takeaways:

- The long project planning phase greatly helped us with the projects overview and minimizes the planning needed for each sprint.
- The “focus weeks” really boosted the projects progress, and we are looking forward to the next “focus weeks”.
- Because we did not review tasks (Quality Assurance (QA)) on a regular basis, all this work needed to be done at the end of an iteration.
- Reviewing on a more regular basis would also lead to more accurate burnup and burndown charts.
- We decided to not include the charts in the future, because it makes no sense to adjust our process only for the sake of good-looking charts.

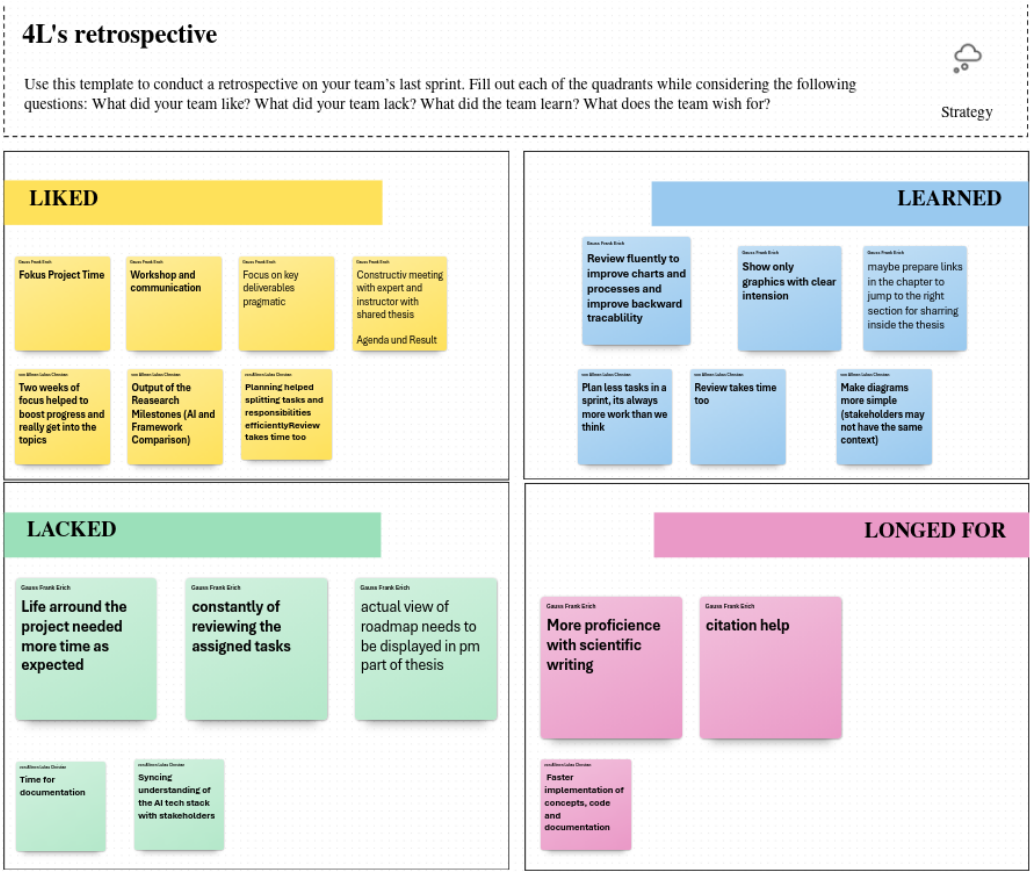


Figure B.10.: Sprint Retro 2

B.8.3. Sprint 3: 26.03 –09.04.2026

Sprint Planning 3: 26.03.2026

Because we got some important input regarding variant decision for AI-integration, framework comparison, and configuration management C.3 which led to additional work. As consequence of this, we decided to shorten the third sprint to one week, which is reflected in the following table B.13 and postponed the sprint planning from the 19.03 to the 26.03. To account for the delayed start in the overall planning we adjusted the roadmap as shown in the screenshot below B.11.

Week	Responsible	Task
Week 1 & 2	Frank	Reproducible Test Environment implementation
Week 1 & 2	Lukas	NSAK Scenario: AI Scenarios (first part): AI-Agent implementation

Table B.13.: Sprint Planning 3

Sprint Review 3: 09.04.2026

The completion of the following two milestones in this sprint are marking a very important point in our bachelor thesis, as we are now able to run the AI based reconnaissance scenario against the reproducible test environment. This enables us to reproducibly test and analyze the actual capabilities of the AI-agent in a realistic scenario. Further we can now demonstrate the full interplay of the technologies to the projects stakeholders and possibly other interested persons.

Milestone Review: Reproducible Test Environment The success criteria of the Reproducible Test Environment B.3.1 milestone are completed as illustrated in B.12.

As a result of this milestone, the test environment can now be set up reproducibly through Infrastructure as Code, which helps in the setup steps and ensures a consistent baseline across runs B.12.

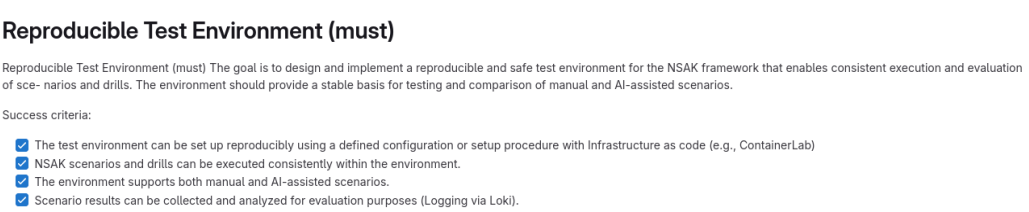


Figure B.12.: Milestone Description: Reproducible Test Environment

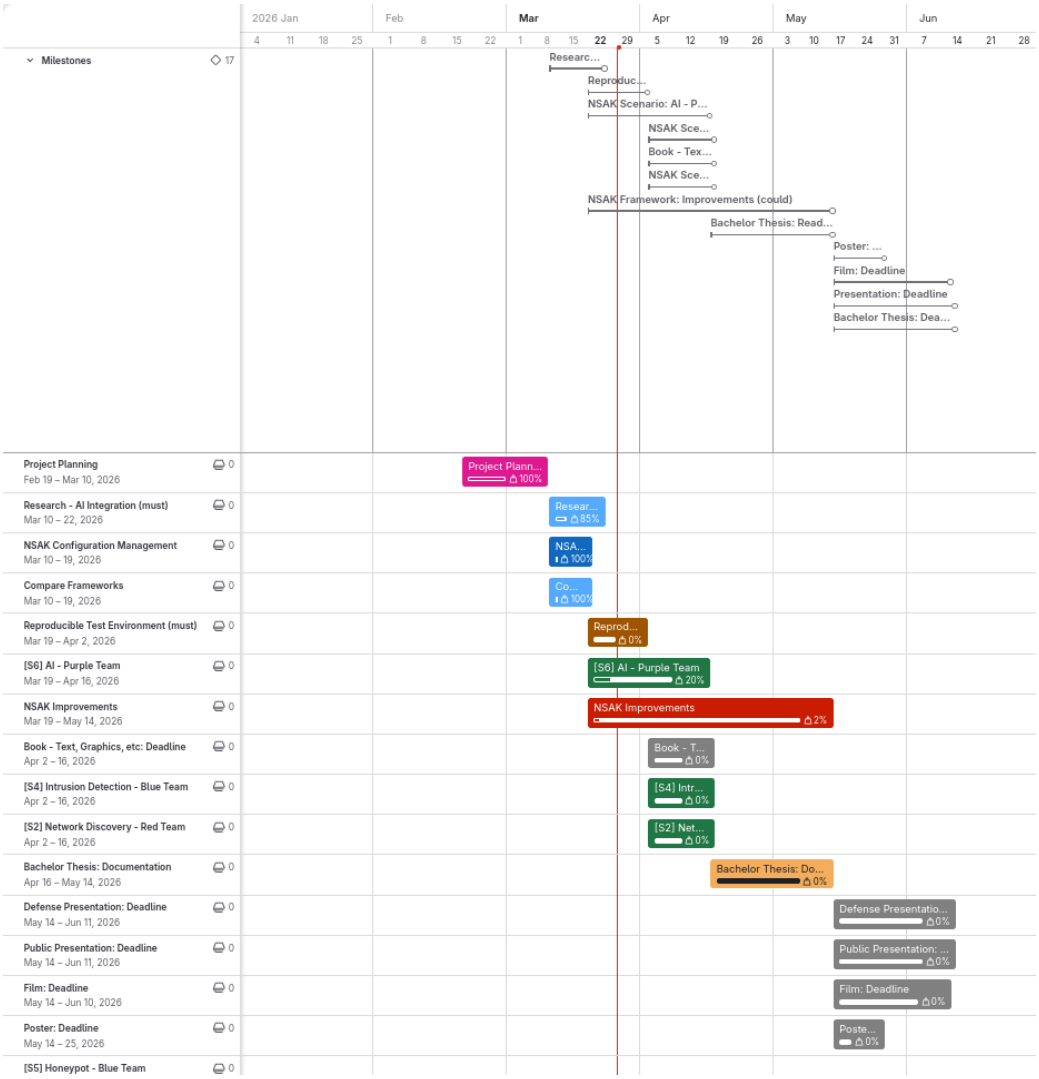


Figure B.11.: Roadmap Sprint Planning 3

Milestone Review: Implement AI Scenarios (first part): AI-Agent implementation As shown in B.13, all "must" goals of the milestone Implement AI Scenarios B.3.1 where completed. But there is still some work todo, to document the implementation and the learnings we made during the development. Further we could improve the implementation in several aspects such as tool-calling and external logging, where we can better analyze the AI-agents behavior. Because this was expected, we planned two sprints for the realization of the AI Scenarios.

NSAK Scenario: AI Scenarios (must)

Description

The usage of AI in network security is still subject of recent research and with the advent of agentic AI we asked ourself: What happens and what are the possibilities when we give an AI full access to the NSAK framework. This scenario is not very well defined yet.

Goals

- [x] The operator can provide an initial prompt or use a predefined Red or Blue Team prompt
- [x] The scenario runs an AI agent with a remote connection
- [x] An operator can interact with the agent, ideally at all times
- [x] The scenario logs all relevant actions and network traffic
- [x] The AI is able to trigger an alert to notify an operator depending on the prompt
- [x] The scenario has a kill switch, so that the operator can stop it at all times

Tasks / Drills

Backbone:

- Initial Prompt
- Run
- Human Interaction Hook
- Logging / Sniffing
- Alerting
- Kill Switch

Must:

- [x] Initial Prompt: Custom Prompt
- [x] Initial Prompt: Red Team Prompt
- [x] Initial Prompt: Blue Team Prompt
- [x] Run: AI Agent with remote connection
- [x] Human Interaction Hook: CLI
- [x] Logging / Sniffing: Local
- [x] Alerting: E-Mail
- [x] Kill Switch: Stop Scenario

Should:

- [ ] Human Interaction Hook: Web GUI
- [ ] Logging / Sniffing: Remote
- [ ] Kill Switch: Shutoff

Could:

- [ ] Human Interaction Hook: Messenger (WhatsApp, Threema, etc.)
- [ ] Human Interaction Hook: Chats (Slack, RocketChat, etc.)
- [ ] Alerting: SMS
- [ ] Alerting: Phone
- [ ] Alerting: Pager
- [ ] Alerting: Messenger (WhatsApp, Threema, etc.)
- [ ] Alerting: Chats (Slack, RocketChat, etc.)
- [ ] Alerting: Scheduling
- [ ] Logging / Sniffing: Dashboard

Workshop Ticket:

Environment

This scenario would be really special, as it should be able to operate in all environments. For running an AI Agent we would need additional setup steps.

Figure B.13.: Milestone Description and Charts: Research AI-Integration

Sprint Retro 3: 09.04.2026

This time we decided to conduct the sprint retro with post-its and a sheet of paper. The key takeaway of the retro B.14 is to estimate the milestones more realistically. We spend a lot of time with documentation which hinders the development. The upside of our continuous approach is a natural growing thesis which helps in the long run. Overall, we are quite happy with the outcome of sprint 3.

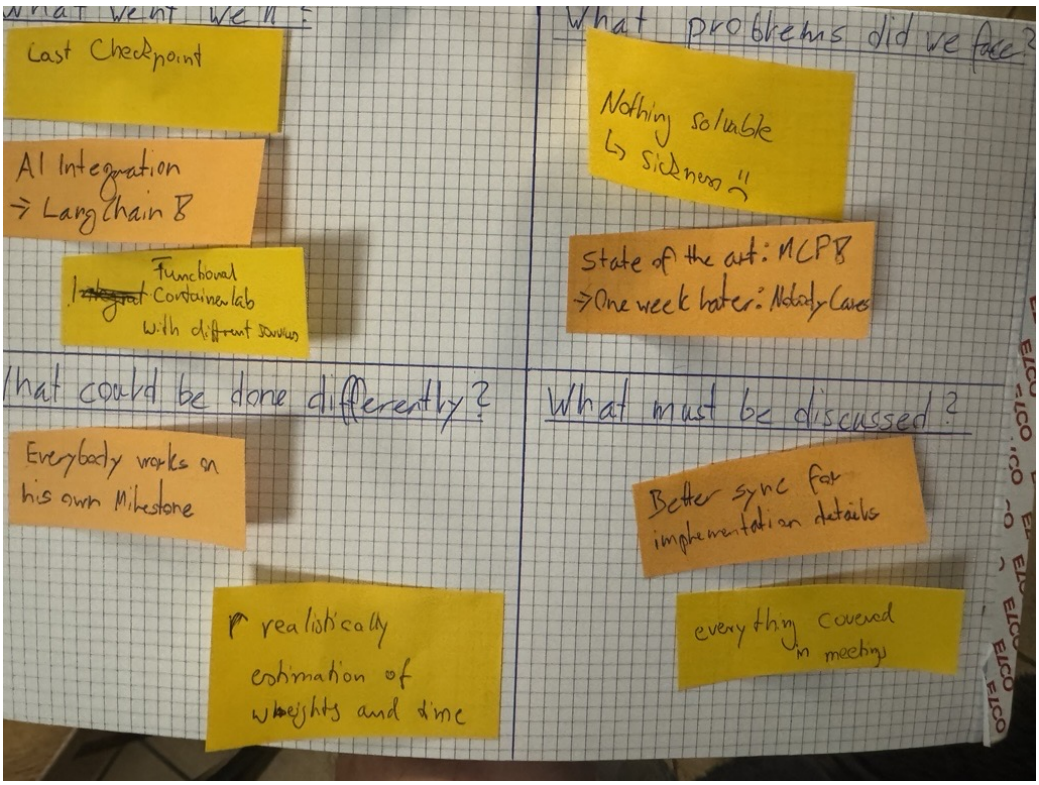


Figure B.14.: Sprint Retro 3

B.8.4. Sprint 4: 10.04 –02.05.2026

Sprint Planning 4: 09.04.2026

Week	Responsible	Task
Week 1	Frank & Lukas	Bring together and test reproducible test environment and AI Reconnaissance
Week 2 & 3	Frank	Network Discovery - Red Team
Week 2	Lukas	NSAK Scenario: AI Scenarios (second part): Improve tool calling and documentation
Week 3	Lukas	External Logging with Grafana / Loki

Table B.14.: Sprint Planning 4

No photograph of the sprint planning roadmap was taken during this sprint, so no visual reference is available for this planning session.

Sprint Review 4: 02.05.2026

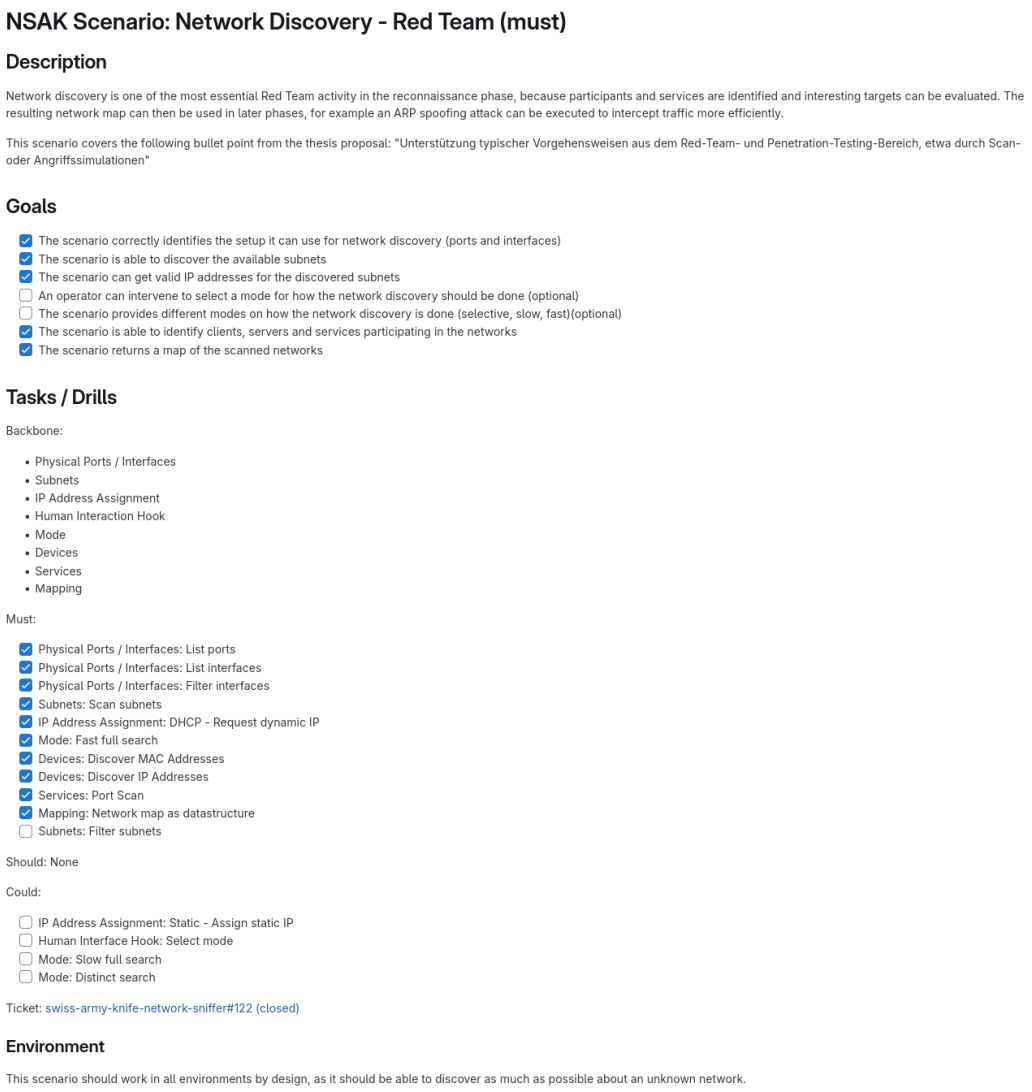
In this sprint could finally bring together the reproducible network environment and the AI scenario, we realized the last scenario categorized as “Must” goal, extended the logging capabilities and could refine the AI agent. Now we can focus on refining the documentation and finalizing the last preparations before moving to the last phase of our thesis, the evaluation and discussion.

Test run of the AI scenario in the reproducible network environment After finishing the previous sprint we were excited to test the AI scenario in the reproducible network environment, as it’s marking a big milestone of our bachelor thesis. We spent more or less a whole day using different models and make fine adjustments to the setup, to see what’s possible. In the end we could instruct the AI agent to conduct network discovery, write a report in markdown, create a network diagram of the findings and send the artifacts via E-Mail. The results of this test run where added to the implementation section of the thesis 5.3.8.

Milestone Review: Network Discovery - Red Team In B.15, not all “must” goals of the milestone where completed. During planning, a human interaction mode was discussed in which an operator could choose how fast or noisy the reconnaissance should be executed. This feature was not implemented, as it does not contribute to answering the thesis question. The scenario is able to discover MAC and IP addresses using arp-scan on Layer 2, and protocols and services using nmap within the

local subnet. All results are stored in a *NetworkDiscoveryResultMap* and displayed to the operator. Service enumeration using targeted NSE scripts was implemented as a dedicated drill, which runs service-specific scripts such as *http-title*, *dns-zone-transfer*, and *smb-security-mode* on all discovered ports.

The scenario could be extended with an optional static IP address, which would be useful in networks where DHCP is not available. Host discovery in remote networks is currently limited to IP addresses only, as Media Access Control addresses are not available across Layer 3 boundaries.



NSAK Scenario: Network Discovery - Red Team (must)

Description

Network discovery is one of the most essential Red Team activity in the reconnaissance phase, because participants and services are identified and interesting targets can be evaluated. The resulting network map can then be used in later phases, for example an ARP spoofing attack can be executed to intercept traffic more efficiently.

This scenario covers the following bullet point from the thesis proposal: "Unterstützung typischer Vorgehensweisen aus dem Red-Team- und Penetration-Testing-Bereich, etwa durch Scan- oder Angriffssimulationen"

Goals

- [x] The scenario correctly identifies the setup it can use for network discovery (ports and interfaces)
- [x] The scenario is able to discover the available subnets
- [x] The scenario can get valid IP addresses for the discovered subnets
- [ ] An operator can intervene to select a mode for how the network discovery should be done (optional)
- [ ] The scenario provides different modes on how the network discovery is done (selective, slow, fast)(optional)
- [x] The scenario is able to identify clients, servers and services participating in the networks
- [x] The scenario returns a map of the scanned networks

Tasks / Drills

Backbone:

- Physical Ports / Interfaces
- Subnets
- IP Address Assignment
- Human Interaction Hook
- Mode
- Devices
- Services
- Mapping

Must:

- [x] Physical Ports / Interfaces: List ports
- [x] Physical Ports / Interfaces: List interfaces
- [x] Physical Ports / Interfaces: Filter interfaces
- [x] Subnets: Scan subnets
- [x] IP Address Assignment: DHCP - Request dynamic IP
- [x] Mode: Fast full search
- [x] Devices: Discover MAC Addresses
- [x] Devices: Discover IP Addresses
- [x] Services: Port Scan
- [x] Mapping: Network map as datastructure
- [ ] Subnets: Filter subnets

Should: None

Could:

- [ ] IP Address Assignment: Static - Assign static IP
- [ ] Human Interface Hook: Select mode
- [ ] Mode: Slow full search
- [ ] Mode: Distinct search

Ticket: swiss-army-knife-network-sniffer#122 (closed)

Environment

This scenario should work in all environments by design, as it should be able to discover as much as possible about an unknown network.

Figure B.15.: Milestone Description: Reproducible Test Environment

Milestone Review: Implement AI Scenarios (second part): Improve tool calling and documentation

As already shown in B.13 from the previous sprint review B.8.3, all "must" goals of the milestone Implement AI Scenarios B.3.1 where completed. In this second part the AI scenarios where tested, tool calling was improved and the documentation was added to the implementation section of the thesis 5.3. Further we could already improve the AI scenarios with the findings of the extensive test run described above B.8.4.

External Logging with Grafana / Loki

In the fifth checkpoint meeting C.0.5 which as in this sprint, we received the feedback that we should improve the logging for the AI-agent and especially which tools it is calling. We decided to set up a self-hosted logging system with Grafana and Loki, where we can collect all logs of the runs and create custom queries to make it easier to analyze the behavior of the AI-agent. The usage is described in the implementation section of the AI Scenarios 5.3.4. The self-hosted setup is documented in the addendum F.5.7.

Sprint Retro 4: 02.05.2026

We held the sprint retro again in Microsoft Whiteboard. We listed the key takeaways below and added a screenshot of the whiteboard B.16.

Key Takeaways:

- Overall good mood and good progress
- Use 80/20 rule for documentation during the implementation phase, so that the base structure is already there.
- Work more continuously on the Bachelor Thesis during the Week (1-2h every day)

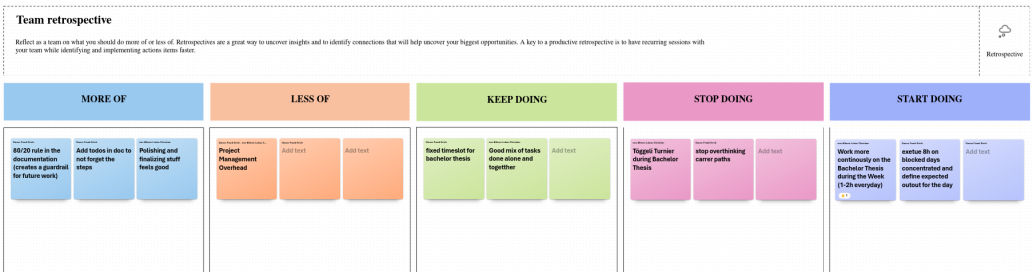


Figure B.16.: Retro Sprint 4

B.8.5. Sprint 5: 03.05 –28.05.2026

Sprint Planning 5: 30.04.2026

We decided on the spot to extend this sprint to four weeks because Frank has become a father, and we hadn't completed all the tickets after the first two weeks. In return, we have expanded the scope of the sprint, updated the sprint planning accordingly B.15. We will catch up on the delay over the following two weeks, in which we both have vacation for conducting our second "focus weeks".

Week	Responsible	Task
Week 1 & 2	Frank & Lukas	Focus on thesis document and prepare evaluation
Week 3	Lukas	Build benchmarking suite
Week 3	Frank	Investigate if we can replace the "Human Interaction Hook" with the "Human In The Loop" from LangChain
Week 3	Lukas & Frank	Evaluate the scenarios in containerlab and test if they are sufficient to test in bfh lab
Week 3 & 4	Frank	Write the evaluation for the benchmarking runs in the thesis, including diagrams
Week 4	Lukas	Build physical test environment
Week 4	Lukas & Frank	Finalize and hand in the poster
Week 4	Lukas & Frank	Finalize and hand in the booklet
Week 4	Lukas & Frank	Evaluate the scenarios in the BFH network security lab
Week 4	Lukas & Frank	Evaluate the scenarios in the physical setup

Table B.15.: Sprint Planning 5

The roadmap B.17 shows the acual state of the project. Most of the epics and milestones are done but partly need a quality check and peer review to mark the issues as "DONE". As we did not change or remove any milestones or tasks, we leave the roadmap as it is.

Sprint Review 5: 28.05.2026

Milestone Review: Network Discovery The milestone was implemented as planned B.3.1.

Implementation Review: Benchmarking Suite After defining the quantitative and qualitative criteria for the evaluation 4.1.1, we realized that it makes sense to develop a benchmarking harness to standardize the results of multiple scenario runs.

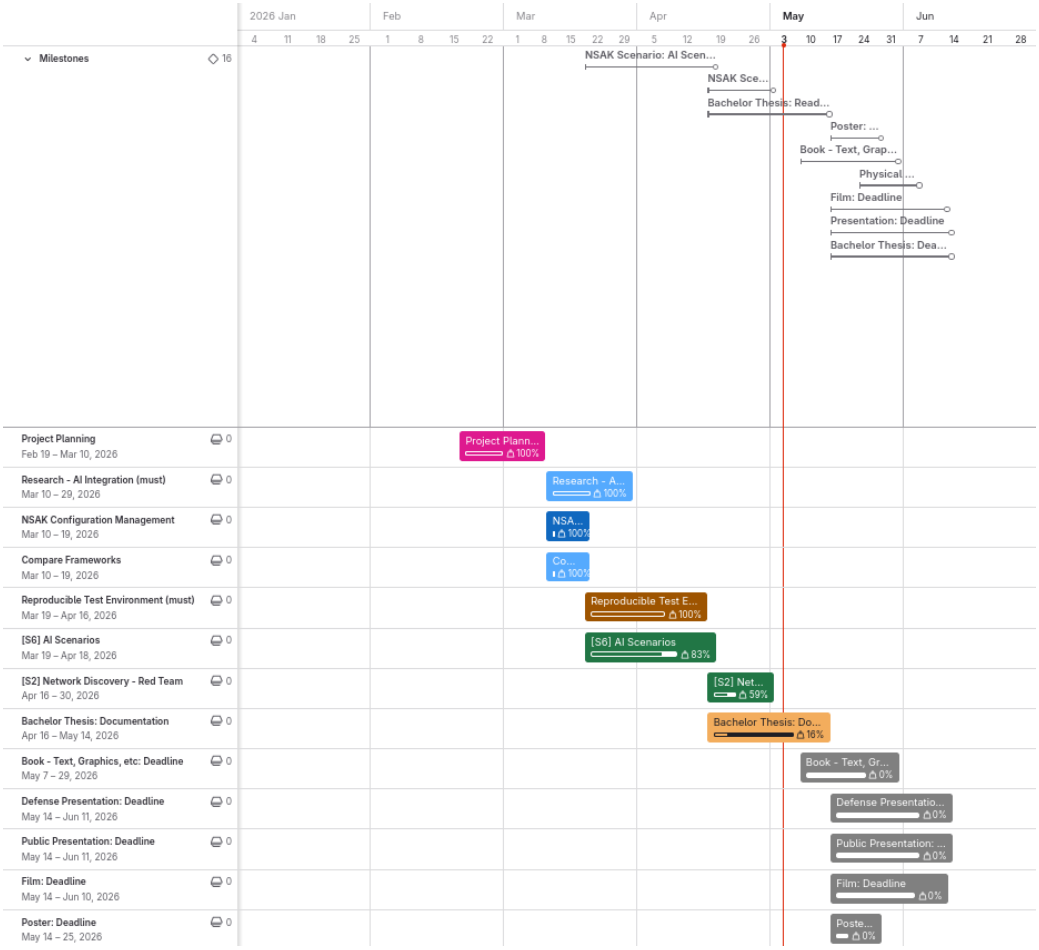


Figure B.17.: Roadmap Sprint Planning 5

Besides standardization the benchmarking harness also greatly reduces the repetitive work for collecting reports required for assessing the qualitative criteria, automates the aggregation of the quantitative criteria and provides the results as JSON for easy diagram generation. As we identified the need for this feature on the spot we did not plan an according milestone. The development is described in the implementation chapter of the thesis 5.5

Implementation Review: LangChain - Human In The Loop After implementing the human interaction hook tool for allowing an operator to interact with the agent, we found out that LangChain already provides its own implementation as a middleware. We then chose to implement a proof-of-concept, which did not bring the improvements we hoped for and decided to stick with the current implementation. More details are documented in the implementation chapter 5.3.3.

Deliverable Review: Poster We decided to only use the header and footer of the template provided by BFH, as we wanted to have a more promotional character with big images and diagrams and less text. The rationale behind this decision is, that the people probably would not read the text anyway, and we rather want to catch attention and then explain the technical details in person. The poster was completed and uploaded to Moodle in time on 25.05.2026 as attached in the media deliverables section G.1. During the design phase we also invented our mascot the “NSAK Lama”, which is now used on all other media.

Deliverable Review: Book In the book we reused the whole abstract and added some additional context, which might be interesting to the reader. For example, we set an emphasis on the conclusion and mentioned LangChain and that we tested a single- and multi-agent with structured output. We added two diagrams from the evaluation to highlight the results and a picture of the physical lab setup with labels, indicating the devices, their roles and in which networks they are. We submitted the book entry on 29.05.2026 within the required deadline as illustrated in the media deliverables section G.2.

Sprint Retro 5: 28.05.2026

As usual, we held the sprint retro in Microsoft Whiteboard. The result is shown in B.18 and the key takeaways are summarized below.

Key Takeaways:

- We are both really happy about the progress we made in the “focus weeks”

- We are a bit stressed as the finishing line is in sight, but that's expected and normal
- This project was a crazy cross-disciplinary ride, and we could learn so much in each area

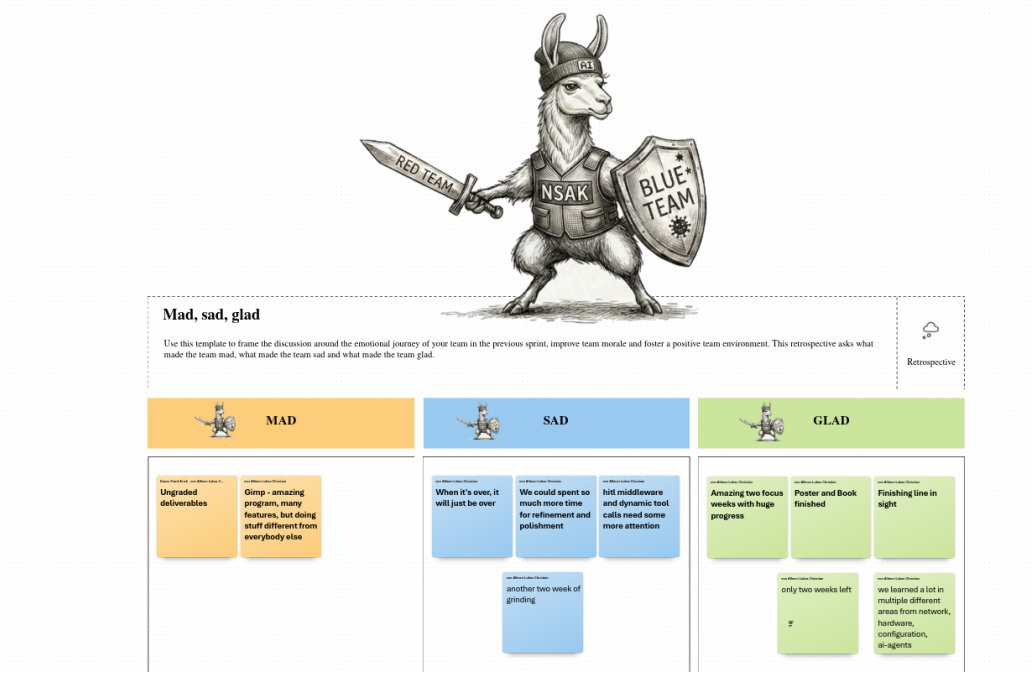


Figure B.18.: Sprint Retro 5

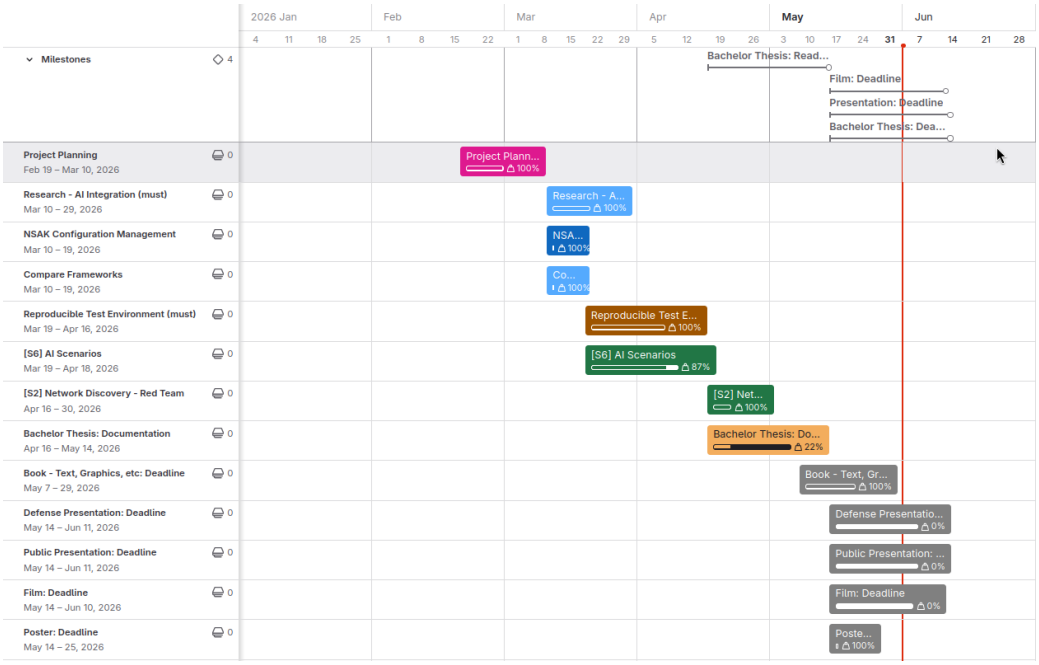


Figure B.19.: Roadmap Sprint Planning 6

B.8.6. Sprint 6: 28.05 –11.06.2026

Sprint Planning 6: 28.05.2026

Week	Responsible	Task
Week 1	Frank & Lukas	Finalize chapters: Methods, Evaluation, Discussion and Conclusion
Week 1 & 2	Frank & Lukas	Polish documentation
Week 2	Frank & Lukas	Create video script, record and cut video
Week 2	Frank & Lukas	Create public presentation
Week 2	Frank & Lukas	Create defense presentation

Table B.16.: Sprint Planning 6

Sprint Review 6: 11.06.2026

Deliverable Review: Video

Deliverable Review: Public Presentation

Deliverable Review: Defence Presentation

Sprint Retro 6: 11.06.2026

C. Meeting Notes

C.0.1. Checkpoint Meeting 1: 26.02.26

Participants	Hj. Wenger (Instructor), Frank Gauss, Lukas von Allmen
Agenda	- Goals presentation via slides - Clarification whether there is a deadline or submission required for the presentation (besides adding it to the addendum). - Clarification whether the defense presentation should be an enhanced version of the public presentation or a separate, more technical presentation. - Proposal to establish a recurring meeting series every Thursday at 15:00 during the semester.
Tasks	- Hand in a set of success criteria for the project goals: We thought we can delivery the success criteria and the project goals as milestones in gitlab, we corrected this later on. - Conduct a risk assessment: B.7.1 - Contact expert via e-mail: Contacted at 21.02.26, response 03.03.26, invited to next checkpoint meeting 05.03.26, the expert takes part at the meeting every two weeks

Table C.1.: Meeting Notes: Checkpoint 1

C.0.2. Checkpoint Meeting 2: 05.03.26

Participants	Hj. Wenger (Instructor), Urs Keller (Expert), Frank Gauss, Lukas von Allmen
Agenda	- Review of the project goals as milestones and project planning. - Review of risk assessment. - Discussion on the investigation and analysis of network communication between end devices and external networks (RED scenario: sniffing, filtering, and potential data exfiltration).
Decisions	- Variant 1 was selected for further development. - Open questions remain regarding the use and management of AI tokens.
Notes	- We misunderstood how to deliver the project goals and success criteria: Instead of defining the goals as milestone we added a goal section in the thesis. - Discussion on the rationale behind selecting Variant 1. Consideration of whether a user interface is required when MCP functionality is available - Open Research Question. - NSAK integrates AI capabilities. - The development of a Web GUI is currently considered lower priority - suggestion of Urs Keller.
Tasks	- Send the slides of the final presentation and documentation of “Project 2” to Urs Keller (Expert): “Done” (via E-Mail). - Delivery the projects goals with a detailed description as part of the thesis by Saturday, 07.03.26: “Done” A - Finalize the project planning: “Done” B.6 - Write the first few chapters of the thesis: “Done”

Table C.2.: Meeting Notes: Checkpoint 2

C.0.3. Checkpoint Meeting 3: 19.03.26

Participants	Hj. Wenger (Instructor), Urs Keller (Expert), Frank Gauss, Lukas von Allmen
Agenda	- Review Project Management and Roadmap - Review of the active milestones (Framework Comparison, AI-Research, Configuration Management) - Discuss Variant Decisions: AI-Integration - Ask about AI token usage - Ask about repository access
Decisions	- We will proceed using NSAK as the basis for our thesis. - Open questions remain regarding the use and management of AI tokens.
Notes	- -
Tasks	- Document decision that we will continue to work with the NSAK framework: 2.2.3 - Expand on variant decision for AI-Integration: - Split diagram into 2 variants: - Expand in detail how the variants are working (e.g., sequence diagram): D.13 and D.15 - Configuration management: Move metadata and others below the device node: "Done" 5.1.2 - BFH TI: Has a cluster with studio macs for AI usage -> Ask if we can use them (https://mlmp.ti.bfh.ch/ via VPN)

Table C.3.: Meeting Notes: Checkpoint 3

C.0.4. Checkpoint Meeting 4: 02.04.26

Participants	Hj. Wenger (Instructor), Frank Gauss, Lukas von Allmen
Agenda	- Overview Roadmap and active milestones - Reproducible Test Environment: Introduce ContainerLab, Show diagram, Physical Implementation (could) - AI Scenario: Demo AI-agent connection to self-hosted, LangChain Research LLM - Review MCP Sequence Diagrams - Discuss what needs to be added to the main document and what to the appendix - Ask if it's possible to lend more HW for physical test setup - Ask for access to BFH AI Cluster
Decisions	- -
Notes	- - Alternative to container LAB: Cisco - DNS3 (not open source) - It would be interesting to add more services to the DMZ in the reproducible test environment - Tutor: The current structure is preferred, we should also ask the expert -
Tasks	- Ask for access to BFH AI: https://mlmp.ti.bfh.ch (Requires VPN) - If we are not allowed, ask tutor for guidance - Variant decisions: Add pros/cons and elaborate why we choose a variant - AI-agent: Design and implement logging - Ask the expert about the preferred document structure: Thesis vs Appendix - Create a HW list for tutor, date and configuration (must be planned, device configuration could be done by assistants) -

Table C.4.: Meeting Notes: Checkpoint 4

C.0.5. Checkpoint Meeting 5: 16.04.26

Participants	Hj. Wenger (Instructor), Urs Keller (Expert), Frank Gauss, Lukas von Allmen
Agenda	- Demo: Reproducible test environment - Demo: AI Agent - Show current state of the project goals and roadmap - Fix the date for bachelor thesis defense: Ideally 17or 18.04.2026
Decisions	- Defense Date: 18.06.2026, 13:30 - 15:30
Tasks	- Add a system prompt to tell the LLM to create a human-readable report of what it did, parallel to the logs: - Export logs to a remote server like Zabbix or Grafana:

Table C.5.: Meeting Notes: Checkpoint 5

C.0.6. Checkpoint Meeting 6: 30.04.26

Participants	Hj. Wenger (Instructor), Frank Gauss, Lukas von Allmen
Agenda	- Demo: Reconnaissance scenario - Demo: Grafana / Loki (Authentication Issue) - Show Roadmap and next two weeks - How can we compare and evaluate the AI with the Reconnaissance scenario?
Notes	- Take a look at nmap plugins - To compare and evaluate the AI with the Reconnaissance scenario, we could run both scenarios in the BFH LAB (physical or virtual). For a physical test run we would need to fix a date with HJ. Wenger. - Interesting metrics are: Speed, Correctness, Initial knowledge (pre-conditions), Creativity (3 - 5 interesting criteria) - Only configure Grafana / Loki to the point where we can easily analyze the steps and tools calls.
Decisions	-
Tasks	-

Table C.6.: Meeting Notes: Checkpoint 6

C.0.7. Checkpoint Meeting 7: 14.05.26

We decided to cancel this meeting because Frank has become a father, and we did not have to show any results worth discussing.

C.0.8. Checkpoint Meeting 8: 28.05.26

Participants	Hj. Wenger (Instructor), Urs Keller (Expert), Frank Gauss, Lukas von Allmen
Agenda	- Demonstrate Benchmarking reports - Present Results & evaluation in thesis - Show picture of physical lab - Discuss current state of the thesis and last steps
Notes	- -
Decisions	- -
Tasks	- Add one technical and one personal part in the conclusion - Move code snippets from the main part to the appendix, unless they are critical

Table C.7.: Meeting Notes: Checkpoint 8

D. Workshops

In the Project Management phase, several workshops were conducted, including a brainstorming session using Post-it notes and a User Story Mapping exercise on a whiteboard. The goal of these workshops was to clarify the scenarios a network edge device should support.

Following these workshops, the insights gathered were used to derive issues and goals, which were then mapped to detailed milestones in GitLab. All deliverables were categorized using the MoSCoW prioritization method into must, should, and could requirements. An example of this prioritization and scenario mapping is illustrated in Figure D.3. All Milestones links are provided in the URL at the top of the section.

D.1. Brainstorm Scenario Selection

In this workshop, participants brainstorm and cluster potential red and blue team scenarios for practical cybersecurity exercises.

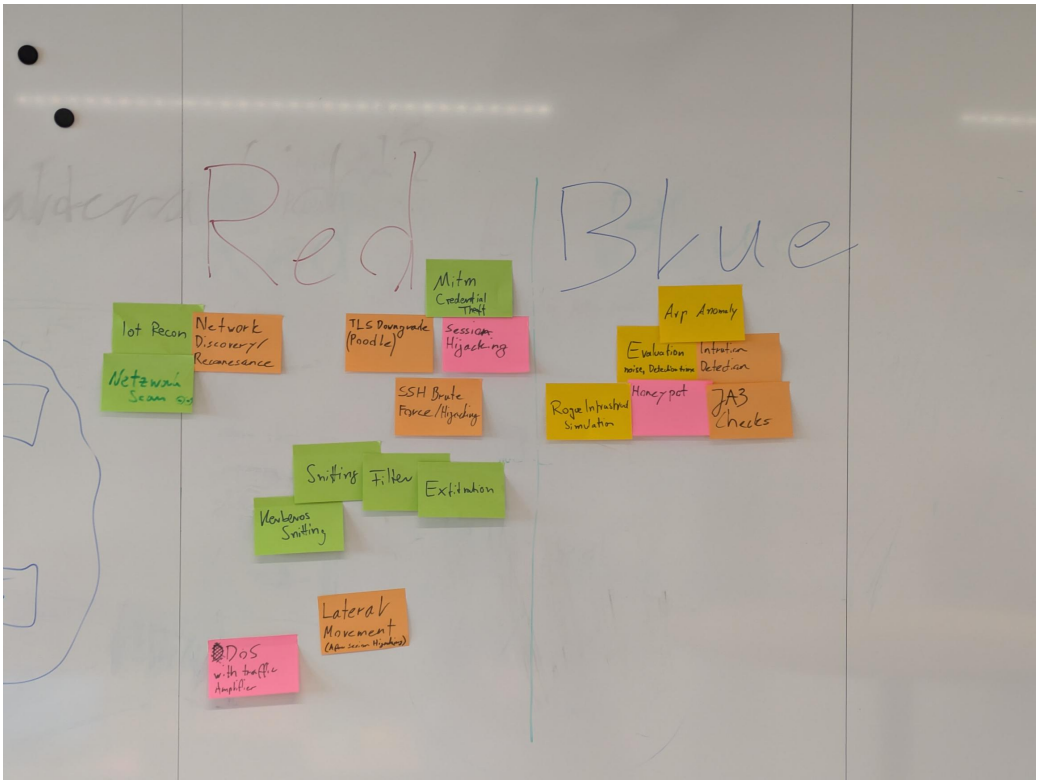


Figure D.1.: Brainstorm Scenarios

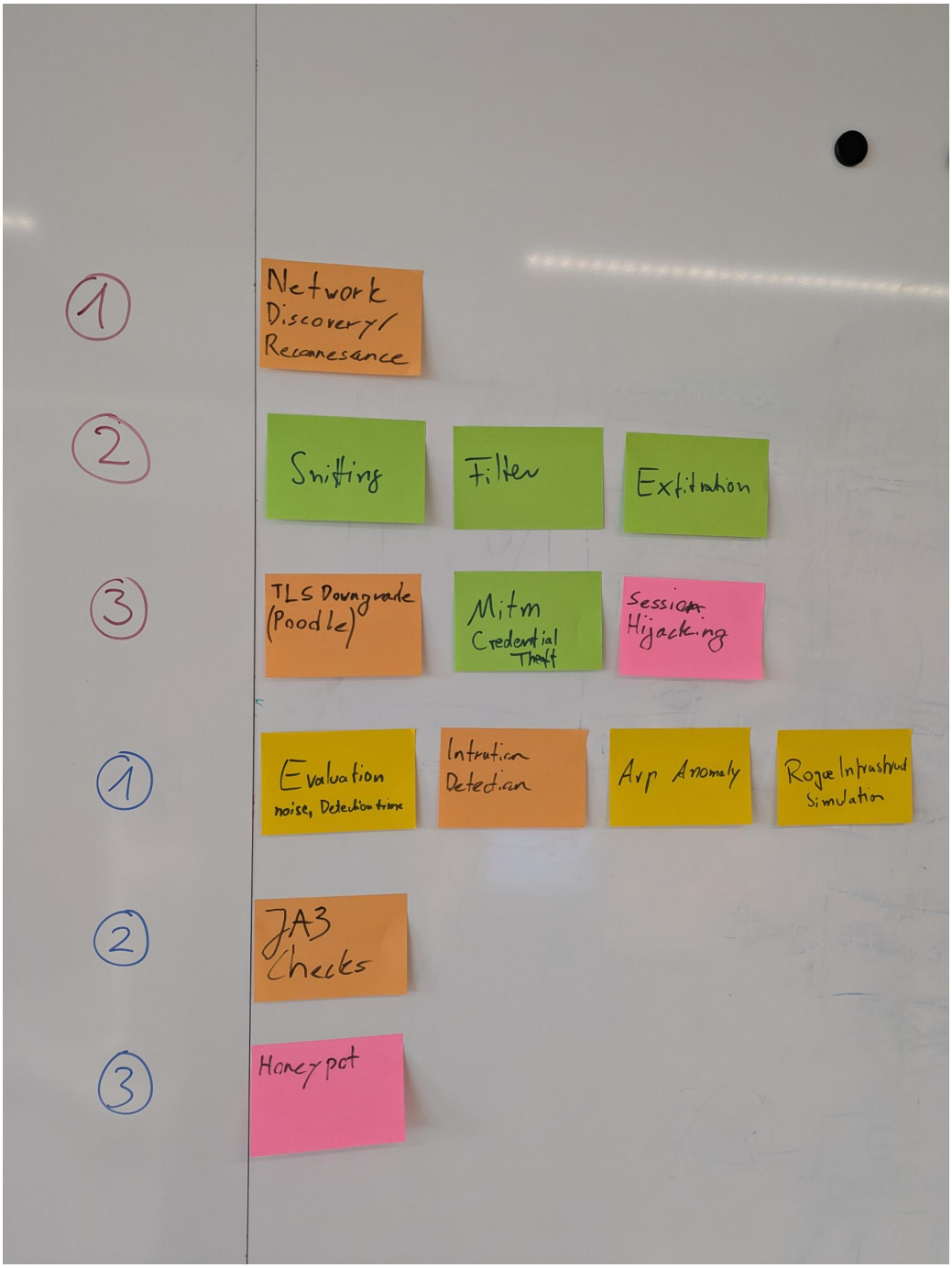


Figure D.2.: Clustering possible Scenarios

The Brainstorm session is used to derive further red/blue team scenarios and led to the pivot of AI-enabled Nsak scenario.

D.2. NSAK Scenario: Network Discovery - Red Team

Network Discovery Scenario

D.3. Objectives

This workshop aims to evaluate the different steps of the network discovery process and to help participants understand how attackers identify and map potential targets within a network.

Building on this objective, Red Teams consider network discovery one of the most essential activities during the reconnaissance phase, as it helps them identify hosts and services and evaluate potential targets. Once a network map is created, the team can use it in later phases. For example, they might execute an ARP spoofing attack, which allows them to intercept network traffic more efficiently.

In relation to these activities, this scenario addresses the following point from the thesis proposal:
‘Unterstützung typischer Vorgehensweisen aus dem Red-Team- und Penetration-Testing-Bereich, etwa durch Scan- oder Angriffssimulationen.’

The workshop led to the milestone *NSAK Scenario: Network Discovery – Red Team (must)*, in which each step for discovering and mapping a network is documented and structured for later implementation.

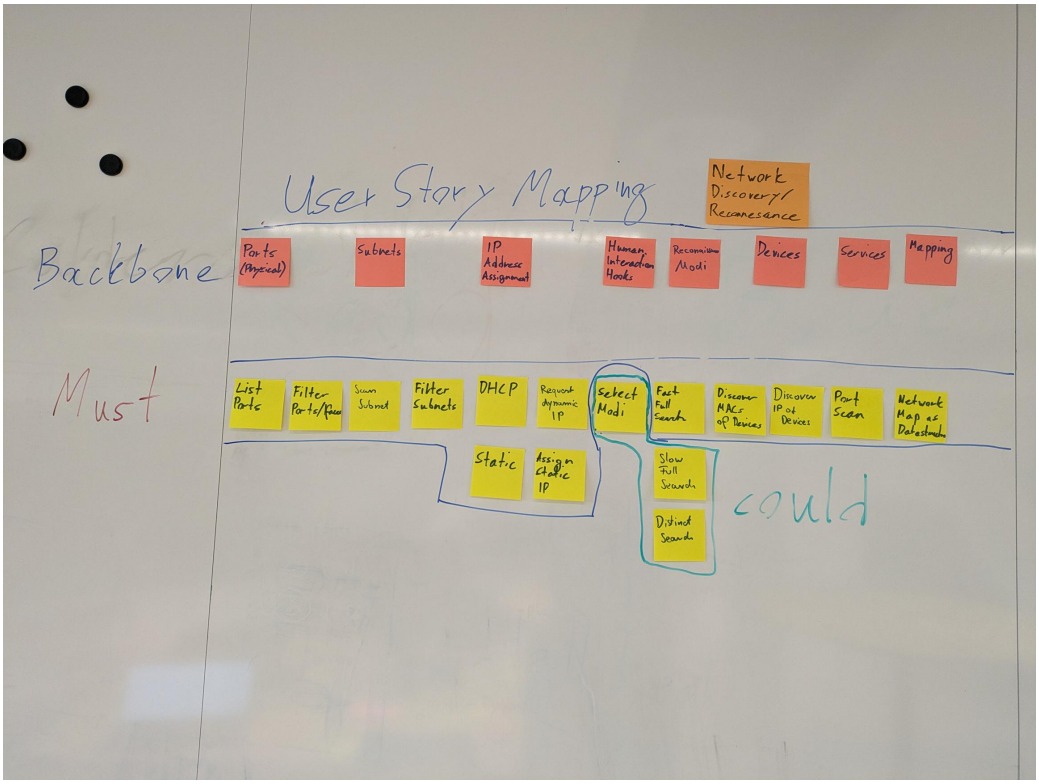


Figure D.3.: NSAK Scenario: Network Discovery - Red Team

D.4. NSAK Scenario: TLS Downgrade Poodle - Red Team

TLS Downgrade Poodle Milestone

D.5. Objectives

This workshop demonstrates how attackers can exploit TLS downgrade vulnerabilities, such as POODLE, to intercept encrypted communication.

In this scenario, the infamous POODLE attack is implemented, in which a malicious actor tries to convince a client and a server to use the vulnerable TLS version 1.1 instead of 1.2 or 1.3 to encrypt HTTP traffic. This attack requires the attacker to sit between the client and the server (a man-in-the-middle, or MITM), allowing them to intercept and control the traffic. When the client initiates a TLS handshake, the attacker drops packets to the server and sends TCP reset packets to the client, eventually trying older TLS versions. Because TLS 1.1 is vulnerable to a CBC Padding

attack, it is possible to create a padding oracle that gradually decrypts the cookie and hijacks the user's session.

Building on this, the scenario directly addresses the following aspect from the thesis proposal:
'Unterstützung typischer Vorgehensweisen aus dem Red-Team- und Penetration-Testing-Bereich, etwa durch Scan- oder Angriffssimulationen'

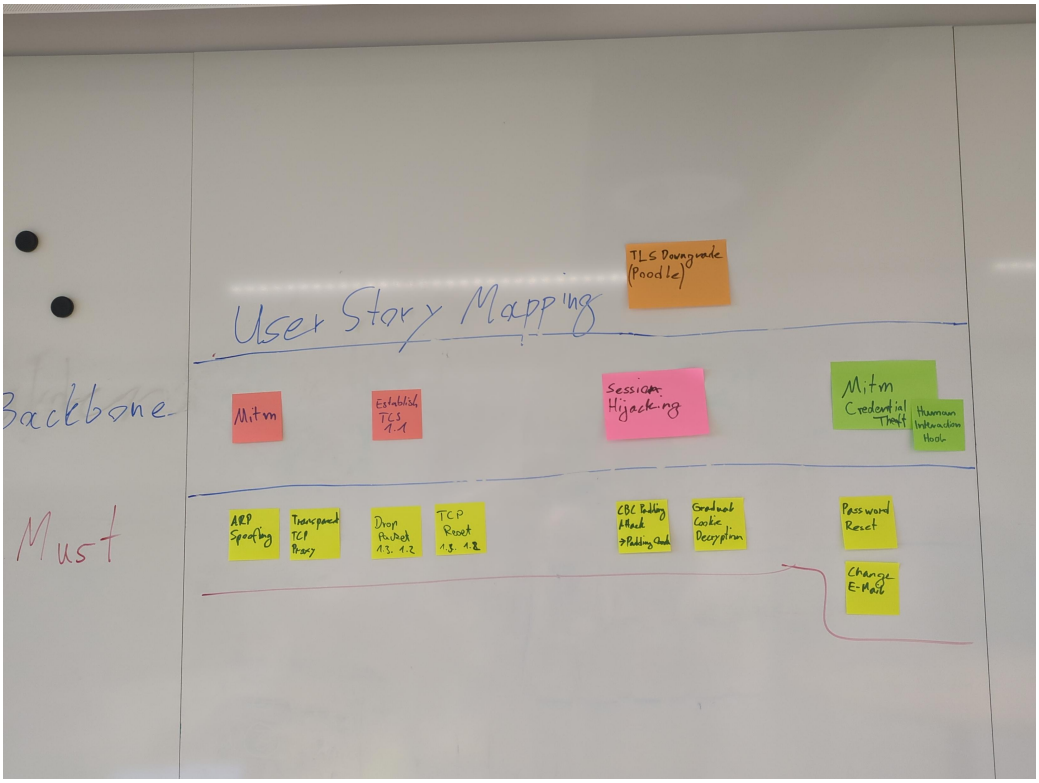


Figure D.4.: NSAK Scenario: TLS Downgrade Poodle - Red Team

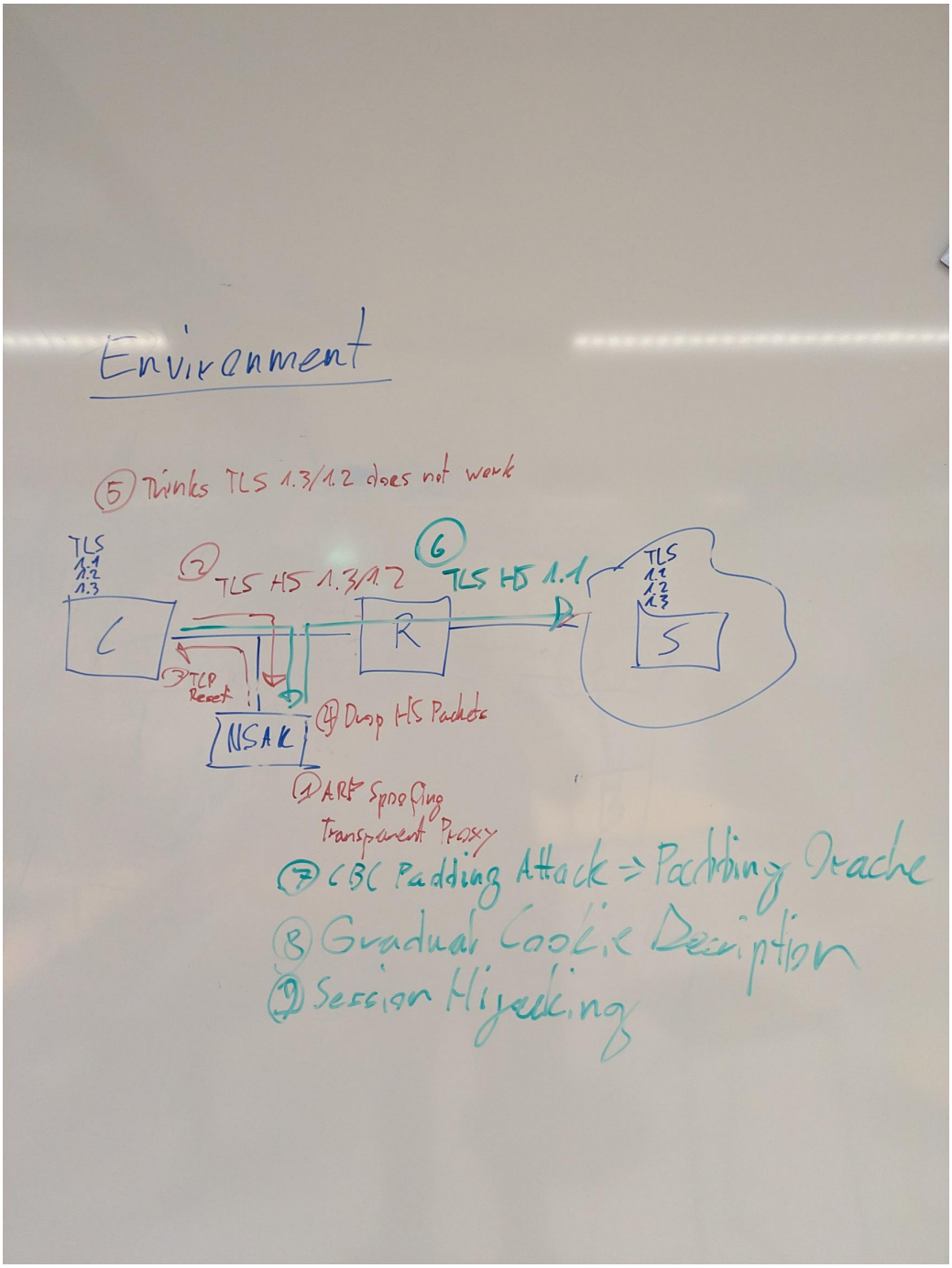


Figure D.5.: NSAK Scenario: TLS Downgrade Poodle -Topology

D.6. NSAK Scenario: Honeypot - Blue Team

Honeypot Milestone

Honeypots can serve as an additional form of intrusion detection and help Blue Teams detect attackers in a network. The system tries to attract malicious actors by providing an attack surface as large as possible; a simple way to achieve this is to open ports. After an attacker tries to connect to any of the open ports, the system notifies an operator and may close all open ports to prevent further attempts to compromise the Honeypot. The operator can then manually intervene to identify and isolate the attacker and analyze the captured payload.

This scenario covers the following bullet point from the thesis proposal:
'Bereitstellung von Funktionen zur Erkennung, Protokollierung und Analyse verdächtiger Aktivitäten als Teil defensiver Massnahmen'

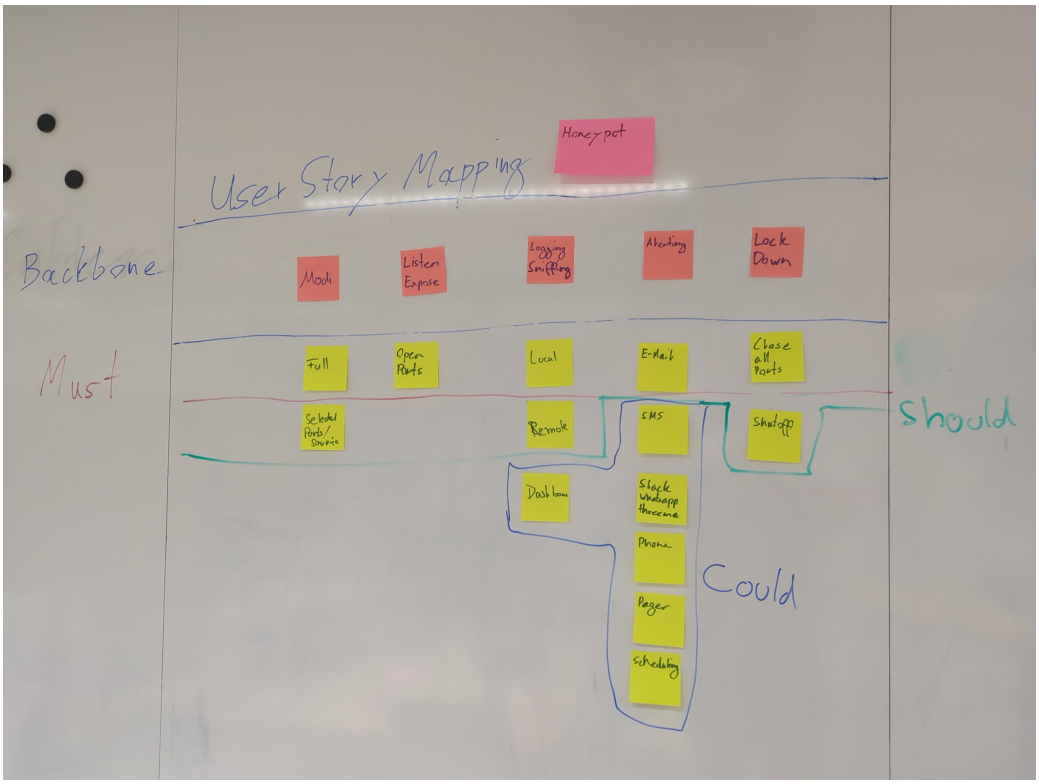


Figure D.6.: NSAK Scenario: Honeypot - Blue Team

D.7. NSAK Scenario: Traffic Capture - Red Team

Traffic Capture Milestone

Capturing, exfiltrating, and analyzing traffic from a targeted network is a key Red Team activity and crucial for initial reconnaissance and planning the next phases of the attack. Based on the gathered intelligence, network participants and services can be identified, and possible attack vectors may be evaluated. If unencrypted traffic is present on the network, session data and credentials could already have been extracted.

This scenario addresses the following bullet point from the thesis proposal: 'Untersuchung und Auswertung der Netzwerkkommunikation zwischen Endgeräten und externen Netzen.'

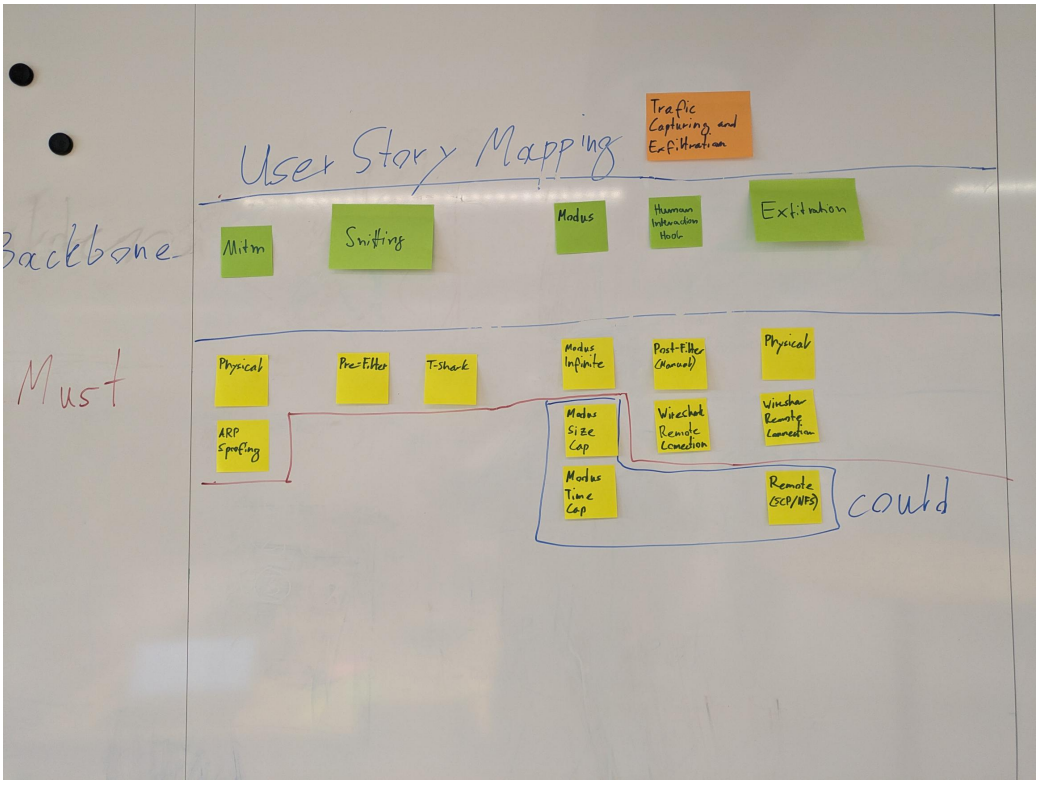


Figure D.7.: NSAK Scenario: Traffic Capture - Red Team

D.8. NSAK Scenario: Intrusion Detection - Blue Team

Intrusion Detection Milestone

Intrusion detection is one of the core Blue Team activities and aims to identify malicious actors and misconfigured clients in production network environments. This can be achieved by introducing a system at a traffic mirroring port or as a central point in a network. The intrusion detection system can leverage a wide array of

security policies, reference traffic, and other markers to effectively identify anomalies. As soon as suspicious activity is detected or a policy is violated, automated measures may be initiated, and security engineers will be automatically alerted for manual intervention.

This scenario covers the following bullet point from the thesis proposal: “Bereitstellung von Funktionen zur Erkennung, Protokollierung und Analyze verdächtiger Aktivitäten als Teil defensiver Massnahmen”

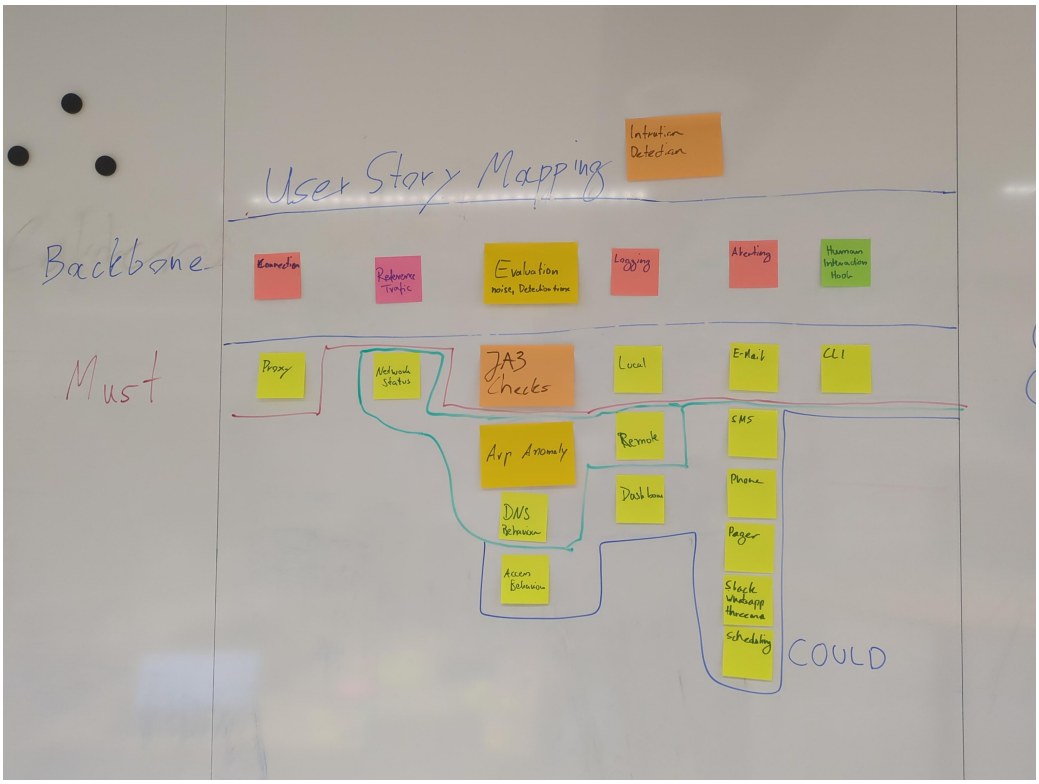


Figure D.8.: NSAK Scenario: Intrusion Detection - Blue Team

D.9. NSAK Scenario: AI - Purple Team

AI Purple Team Milestone

Even tough we were not quite hooked by the current hype around AI during the predecessor project 3.1, we now see a huge potential in integrating it into the NSAK-framework, especially in because of the tool-calling capabilities of agentic AI. In our point of view, the possibility that common blue and red team activities, which require a lot of know how and time, can be simplified or speed up is really promising. This scenarios would explore if it’s possible that intrusion detection or network

reconnaissance tasks can be assisted or maybe even fully automated by an AI-agent.

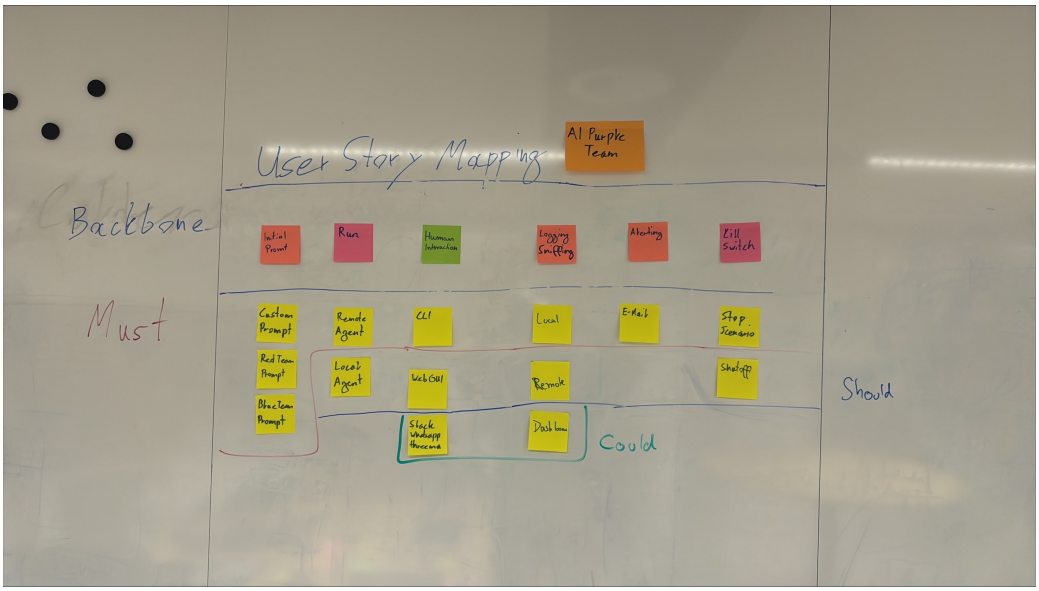


Figure D.9.: NSAK Scenario: AI - Purple Team

Variant 1: AI Purple Team

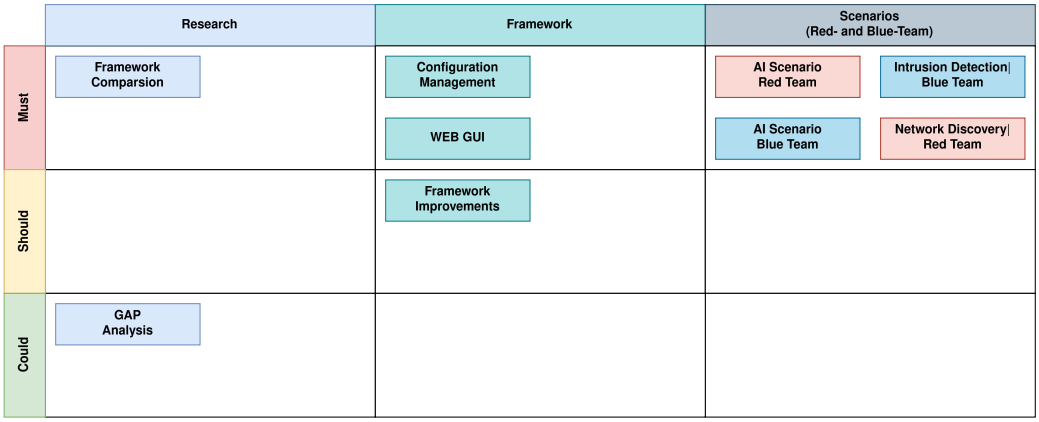


Figure D.10.: Variant 1: AI vs. Manually Built Scenarios

D.10. Variant Decision: Thesis Goals

During the first sprint of the project, which was focused on project management and planning, we identified the very promising hypothesis that red and blue team activities could be automated and/or enhanced by AI. Pursuing this hypothesis would change the projects focus and goals and thus requires the agreement and approval of the stakeholder. To communicate this circumstance efficiently and leave the option to go with the status quo, we prepared two options to conduct a “Variant Decision”. In the following two subsections the two variants presented in the second checkpoint meeting C.2 are described and illustrated. It must be noted, that the variants and their visualizations D.10 D.11 are momentary snapshots and will not be updated for traceability reasons. For example, the Web GUI was already deprioritized during the discussion of the variants, which is reflected in the project goals A but not here.

D.10.1. Variant 1: AI vs. Manually Built Scenarios

Figure D.10 shows the project goals adjusted for the AI focused hypothesis, which is favored variant of the project team. In this option most of the evaluated scenarios will not be realized in favor of AI based scenarios, which also requires additional research A.1.2. The basic idea is to implement scenarios for network discovery A.3.3, intrusion detection A.3.4 and an AI based scenario with the same capabilities A.3.1. In the evaluation of the thesis we can then assess the quality of the AI based scenarios itself A.4.1 and in comparison to the manually built scenarios A.4.2.

Variant 2

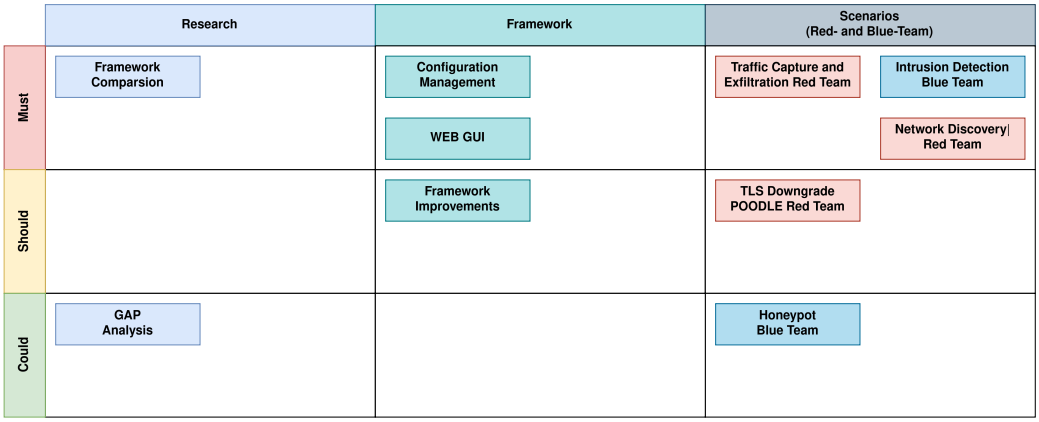


Figure D.11.: Variant 2: Multiple Blue and Red Team Scenarios

D.10.2. Variant 2: Multiple Blue and Red Team Scenarios (Status Quo)

This is the fallback if the other option D.10.1 is not approved by the stakeholders. Even if the project team does not favor this more conservative variant, it still represents a valid way to conduct an interesting project. As illustrated below D.11, in this variant we propose to proceed with the previously planned scenarios for which we already conducted "User Story Mappings" documented under the workshops section**??**. The focus would lie in more conventional scenarios, overall framework improvements and an in depth comparison with another framework in form of a GAP Analysis.

D.10.3. Decision

The stakeholders and project team decided in favor of **Variant 1: AI vs. Manually Built Scenarios** D.10.1 in the checkpoint meeting C.2.

D.11. Variant Decision: AI Integration

During the AI-research 2.2 we identified two interesting ways to allow NSAK to interact with AI based technologies. As illustrated in D.12, the first variant purposes to implement an AI-agent directly into the framework and use it in scenarios and drills. In the second variant shown in D.14, the capabilities of NSAK and other relevant tools on the NSAK device would be exposed via MCP.

D.11.1. Variant 1: AI-agent in NSAK

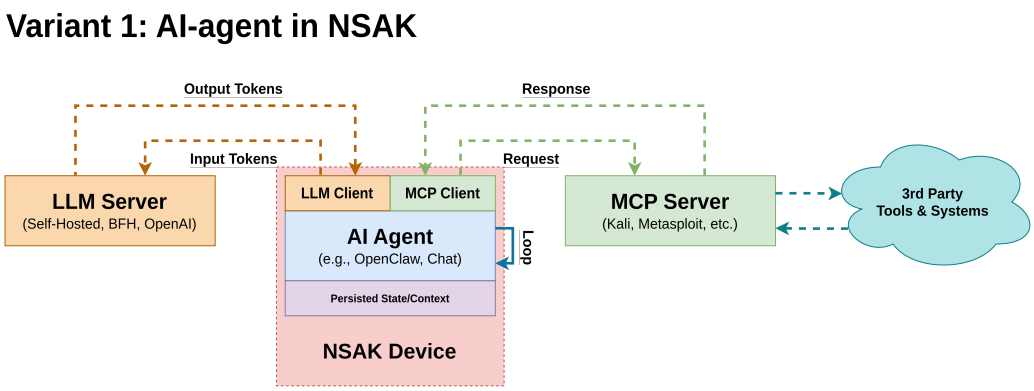


Figure D.12.: AI Integration Variant 1: AI-agent in NSAK

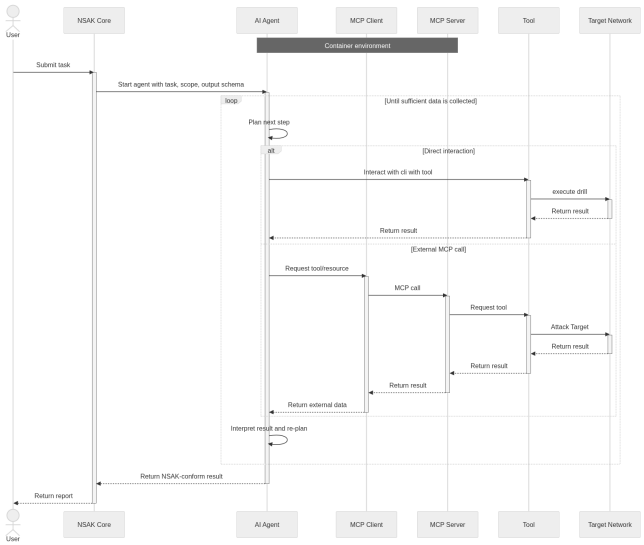


Figure D.13.: Variant 1, AI Agent in NSAK

In this architecture, the AI agent is embedded within the NSAK device. The NSAK core establishes the operational guardrails and schema, and provides the initial prompt to the AI Agent. The agent uses a local or remote LLM, which is accessible over the management network. It operates within an iterative loop, in which it may either invoke external tools via a command-line interface (e.g., nmap, Metasploit) or interact with them through an MCP server interface. The results of these operations are subsequently processed and returned to the NSAK core in a structured schema-compliant format.

D.11.2. Variant 2: NSAK as MCP Server

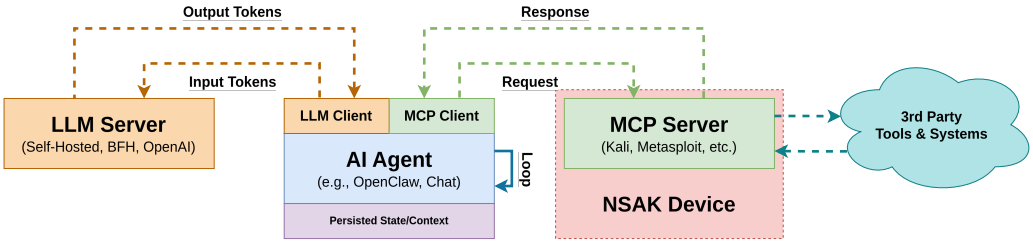


Figure D.14.: AI Integration Variant 2: NSAK as MCP server

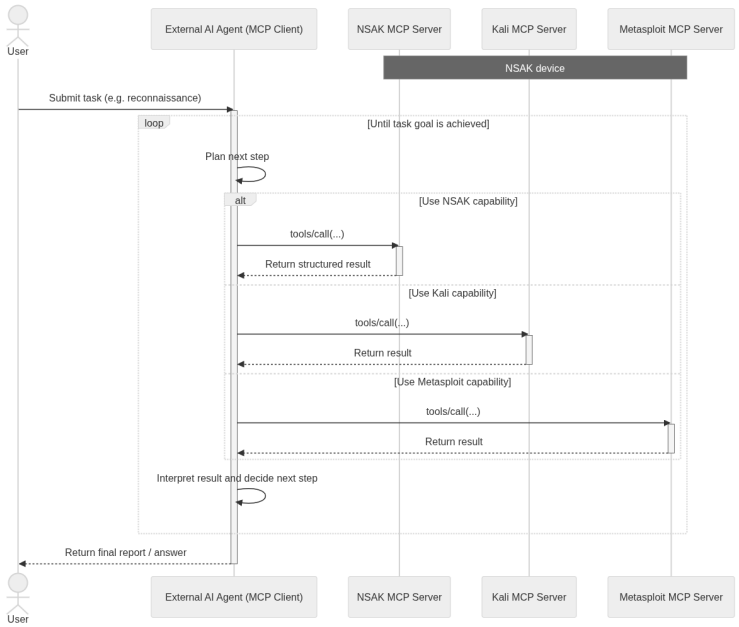


Figure D.15.: Variant 2 NSAK MCP Server

Variant 1, in which the Agentic-AI is embedded within the NSAK device, enables the definition and enforcement of schemas and security policies within a controlled environment. Furthermore, it allows direct interaction with tools via the command-line interface and integration with MCP servers. This results in a tightly coupled architecture with high control and consistency.

In contrast, Variant 2 relies on an external agent that accesses the NSAK device through an exposed MCP server interface. While this approach provides increased modularity, it also introduces limitations in flexibility, as the agent is restricted to the set of tools exposed by the NSAK device.

From a holistic and multi-layered architectural perspective, Variant 1 is preferred. It enables deeper system integration, higher security, and a wider range of executable tools.

D.11.3. Decision

The project team selected **Variant 1: AI Agent in NSAK** D.11.1, as it significantly enhances the autonomy and automation capabilities of the NSAK framework.

From a technical perspective, embedding the agent directly within the NSAK device enabled the framework to serve as an Agentic-AI harness. Furthermore, the agent can leverage existing drills and scenarios as reusable building blocks.

For tool calling, the team decided against relying directly on MCP. A large number of tools bloat the context and pollute the quality of the result. Instead, *LangChain Tools* were used. This approach preserved modularity while allowing the agent's toolset to be integrated with the NSAK framework.

E. Listings

E.1. AI Scenarios: Source Listings

E.1.1. AI Agent Core

```python
class TrackToolCallMiddleware(AgentMiddleware):  # type: ignore
    """
    Tool tracking middleware.
    """

    def __init__(self, tools_available: Iterable[str]) -> None:
        """
        Init tool tracking middleware.
        """
        self.tools_called: dict[str, list[str]] = {
            tool: [] for tool in set(tools_available)
        }

    async def awrap_tool_call(
        self,
        request: ToolCallRequest,
        handler: Callable[[ToolCallRequest], Awaitable[ToolMessage | Command[Any]]],
    ) -> ToolMessage | Command[Any]:
        """
        Track async tool calls.
        """
        tool_name = request.tool_call["name"]
        tool_args = request.tool_call["args"]

        self.tools_called.setdefault(tool_name, [])
        self.tools_called[tool_name].append(str(tool_args))

        return await handler(request)

    def wrap_tool_call(
        self,
        request: ToolCallRequest,
        handler: Callable[[ToolCallRequest], ToolMessage | Command[Any]],
    ) -> ToolMessage | Command[Any]:
        """
        Track sync tool calls.
        """
        tool_name = request.tool_call["name"]
        tool_args = request.tool_call["args"]

        self.tools_called.setdefault(tool_name, [])
        self.tools_called[tool_name].append(str(tool_args))

        return handler(request)
```

Listing 1: LangChain: AI Agent - TrackToolCallMiddleware

```python
class UsageCallback(AsyncCallbackHandler):  # type: ignore
    """
    Track token usage and tool calls.
    """

    def __init__(self) -> None:
        """
        Init token usage.
        """
        self.prompt_tokens = 0
        self.completion_tokens = 0
        self.total_tokens = 0

    async def on_llm_end(self, response: LLMResult, **kwargs: Any) -> None:  # noqa: ANN401
        """
        Capture the token usage.
        """
        if response.llm_output is None:
            # Fallback: sum usage from generations (present when streaming)
            for generations in response.generations:
                for gen in generations:
                    if not hasattr(gen, "message"):
                        continue
                    _usage = getattr(gen.message, "usage_metadata", None)
                    if _usage is None:
                        logger.warning("No usage metadata in on_llm_end!")
                        return
                    self.prompt_tokens += _usage.get("input_tokens", 0)
                    self.completion_tokens += _usage.get("output_tokens", 0)
                    self.total_tokens += _usage.get("total_tokens", 0)
            return

        usage: dict[str, int] = (
            response.llm_output.get("token_usage")
            or response.llm_output.get("usage")
            or {}
        )
        if usage is None:
            logger.warning("No token usage in on_llm_end!")
            return
        prompt = usage.get("prompt_tokens") or usage.get("input_tokens", 0)
        completion = usage.get("completion_tokens") or usage.get("output_tokens", 0)
        self.prompt_tokens += prompt
        self.completion_tokens += completion
        self.total_tokens += usage.get("total_tokens") or (prompt + completion)
```

Listing 2: LangChain: AI Agent - UsageCallback

```python
def create_chat_ollama(
    model: str, base_url: str, api_key: str | None, kwargs: dict[str, Any]
) -> BaseChatModel:
    """
    Create ChatOllama instance.
    """
    if api_key is not None:
        kwargs.update({"Authorization": f"Bearer {api_key}"})

    return ChatOllama(model=model, base_url=base_url, **kwargs)


PROVIDER_MAP: dict[
    str, Callable[[str, str, str | None, dict[str, Any]], BaseChatModel]
] = {
    "ollama": create_chat_ollama,
    "openai": lambda model, base_url, api_key, kwargs: ChatOpenAI(
        model=model, base_url=base_url, api_key=api_key, **kwargs
    ),
    "openwebui": lambda model, base_url, api_key, kwargs: ChatOpenAI(
        model=model, base_url=base_url, api_key=api_key, **kwargs
    ),
    "anthropic": lambda model, base_url, api_key, kwargs: ChatAnthropic(
        model_name=model,
        base_url=base_url,
        api_key=api_key,
        **kwargs,
    ),
}
```

Listing 3: LangChain: AI Agent - Provider Map

```python
class AiAgent:
    """
    Abstraction for an AiAgent.
    """

    system_prompt: str = """
    You are in a cybersecurity simulation and act as the purple team.

    Tool calling rules:
    - Working directory: %(working_directory)s
    - Only send emails if requested
    """ % {"working_directory": str(config.work_path.absolute())}

    def __init__(
        self,
        provider: str,
        model: str,
        base_url: str,
        api_key: str | None,
        tools: Sequence[BaseTool | Callable[..., Any] | dict[str, Any]] | None = None,
        middleware: Sequence[AgentMiddleware] | None = None,
        debug: bool = False,
        system_prompt: str | None = None,
        response_format: Any = None,  # noqa: ANN401
    ) -> None:
        """
        Initializes the AiAgent and the connection to its backend model.
        """
        self.usage = UsageCallback()

        if system_prompt is not None:
            self.system_prompt = system_prompt

        if tools is None:
            tools = []

        self.track_tool_call_middleware = TrackToolCallMiddleware(
            tools_available=[tool.name for tool in tools]  # type: ignore
        )

        middlewares: list[AgentMiddleware] = [self.track_tool_call_middleware]
        if middleware is not None:
            middlewares.extend(list(middleware))

        if model == "gpt-oss:120b" and response_format is not None:
            # Hack for gpt-oss:120b as langchain uses ProviderStrategy for this model, which fails.
            response_format = ToolStrategy(response_format)

        self.model = AiAgent.create_model(
            provider,
            model,
            base_url,
            api_key,
            callbacks=[self.usage],
        )
        self.agent = create_agent(
            self.model,
            tools=tools,
            middleware=middlewares,
            system_prompt=self.system_prompt,
            debug=debug,
            response_format=response_format,
        )
```

Listing 4: LangChain: AI Agent - Init

```python
    @staticmethod
    def create_model(
        provider: str,
        model: str,
        base_url: str,
        api_key: str | None = None,
        **kwargs: Any,  # noqa: ANN401
    ) -> BaseChatModel:
        """
        Create a provider specific ChatModel from a model string.
        """
        # Temperature is not supported by opus 4.7, so we removed it for convenience
        # Temperature means something like "creativity" and usually leads to less predictable and
        ↪  consistent results.
        # In the context of our bachelor thesis we want the agent to behave as consistent as possible.
        # kwargs.update({"temperature": 0})

        create_model = PROVIDER_MAP.get(provider)

        if create_model is None:
            supported_providers = ", ".join(PROVIDER_MAP.keys())
            message = ValueError(
                f"Provider {provider} is not supported yet, valid options {supported_providers}."
            )
            raise ValueError(message)

        return create_model(model, base_url, api_key, kwargs)
```

Listing 5: LangChain: AI Agent - create_model

```python
    async def ainvoke(self, prompt: str, role: str = "user") -> dict[str, Any] | Any:  # noqa: ANN401
        """
        Invoke a prompt.

        :param role:
        :param prompt:
        :return:
        """
        return await self.agent.ainvoke(
            {"messages": [{"role": role, "content": prompt}]}, reasoning=True
        )

    def invoke(self, prompt: str, role: str = "user") -> dict[str, Any] | Any:  # noqa: ANN401
        """
        Invoke a prompt.
        """
        return asyncio.run(self.ainvoke(prompt, role))
```

Listing 6: LangChain: AI Agent - ainvoke / invoke

```python
    async def run(self, prompt: str) -> AsyncGenerator[str, None]:
        """
        Run the AI-agent with the given prompt.
        """
        result = await self.ainvoke(prompt)
        for message in result.get("messages", []):
            role = type(message).__name__.replace("Message", "")

            if isinstance(message.content, str):
                content = message.content
            else:
                content = pformat(message.content)

            yield f"[{role}]\n{content}\n"

    async def run_interactive(
        self,
        prompt: str,
    ) -> AsyncGenerator[str, None]:
        """
        Run the agent in a continuous interactive loop, maintaining conversation history.

        After each agent response the human operator is prompted for the next instruction.
        The loop ends when the operator enters an empty line or the 'exit' string.

        :param prompt: The initial prompt to start the session.
        """
        message = f"ai_agent:[Prompt] {prompt}"
        logger.info(message)
        stop_command = "exit"
        messages: list[Any] = [{"role": "user", "content": prompt}]

        while True:
            prev_count = len(messages)
            result = await self.agent.ainvoke({"messages": messages})
            messages = result.get("messages", messages)

            for message in messages[prev_count:]:
                role = type(message).__name__.replace("Message", "")
                if isinstance(message.content, str):
                    content = message.content
                else:
                    content = pformat(message.content)
                if content:
                    yield f"[{role}]\n{content}\n"

            next_instruction = input(
                f"\n[Next instruction (empty or '{stop_command}' to stop)]\n> "
            ).strip()
            if not next_instruction or next_instruction.lower() == stop_command.lower():
                break

            message = f"ai_agent:[Prompt] {next_instruction}"
            logger.info(message)
            messages.append({"role": "user", "content": next_instruction})
```

Listing 7: LangChain: AI Agent - run / run_interactive

```python
async def get_mcp_tools(config: Configuration) -> list[BaseTool]:
    """
    Setup MCP Tools.
    """
    mcp_tools = []
    mcp_configs = {}

    if config.drawio_mcp is not None:
        mcp_configs.update(
            {
                "drawio": {
                    "transport": "streamable_http",
                    "url": config.drawio_mcp.url,
                    "headers": {
                        "Authorization": f"Basic {config.drawio_mcp.basic_auth}",
                    },
                }
            }
        )

    mcp_client = MultiServerMCPClient(mcp_configs)

    try:
        mcp_tools = await mcp_client.get_tools()
    except Exception as e:
        logger.warning("Failed to load MCP tools; continuing without them.", exc_info=e)

    return mcp_tools
```

Listing 8: LangChain: AI Agent - get_mcp_tools

```python
async def create_ai_agent(
    interactive: bool = False,
    tools: list[BaseTool | Callable[..., Any] | dict[str, Any]] | None = None,
    system_prompt: str | None = None,
    response_format: Any = None,  # noqa: ANN401
    load_drill_tools: bool = False,
    load_mcp_tools: bool = False,
) -> AiAgent:
    """
    Creates an AI-agent from the runtime configuration.
    """
    if config.ai is None:
        message = "The AI is not configured. Run:`nsak config set ai` for the interactive setup."
        raise RuntimeError(message)

    if tools is None:
        tools = [
            cli_tool,
            host_configuration,
            send_email,
        ]

        if load_drill_tools:
            tools.extend([*generate_drill_tools()])

        if load_mcp_tools:
            tools.extend([*await get_mcp_tools(config)])

    middleware: list[AgentMiddleware] = []

    if interactive:
        tools.append(human_interaction_hook)

    return AiAgent(
        config.ai.provider,
        config.ai.model,
        config.ai.base_url,
        config.ai.api_key,
        tools,
        middleware,
        debug=False,
        system_prompt=system_prompt,
        response_format=response_format,
    )
```

Listing 9: LangChain: AI Agent - create_ai_agent

E.1.2. AI Configuration

```python
from __future__ import annotations

import dataclasses


@dataclasses.dataclass(kw_only=True)
class AiConfiguration:
    """
    Configuration for the AI agent backend.
    """

    provider: str
    model: str
    base_url: str
    api_key: str | None = dataclasses.field(default=None)
```

Listing 10: AI Configuration Dataclass

```python
@dataclasses.dataclass(kw_only=True)
class LokiConfiguration:
    """
    Configuration for logging to loki.
    """

    url: str
    username: str
    password: str
```

Listing 11: Loki Logging Configuration

```python
@dataclasses.dataclass(kw_only=True)
class EmailConfiguration:
    """
    SMTP Configuration for sending emails.
    """

    host: str
    port: int
    encryption: EmailEncryptionType = EmailEncryptionType.NONE
    username: str
    password: str
    sender: str
    receiver: str
```

Listing 12: E-Mail Alerting Configuration

```
class EmailBackend:
    """
    Email Backend used for sending mails via SMTP.
    """

    backend: smtplib.SMTP | LoggerEmailBackend

    def __init__(self, email_config: EmailConfiguration | None) -> None:
        """
        Set up the SMTP client with the correct configuration for encryption.
        """
        self.config = email_config
        self._init_backend()
```

Listing 13: E-Mail Alerting Implementation

E.1.3. LangChain Tools

```
@tool  # type: ignore[misc]
def host_configuration() -> dict[str, Any]:
    """
    Returns the host network configuration as a dictionary.

    Use this tool first to understand the local network setup before running any scans or attacks.

    The returned dictionary has the following structure:
    {
        "debug": bool,
        "device": {
            "id": str,
            "name": str,
            "description": str,
            "configuration": {
                "network": {
                    "ethernets": {
                        "<interface_name>": {
                            "name": str,
                            "is_up": bool,        # False means the interface is DOWN, skip it
                            "is_target": bool,    # True means this interface is used for attacks/scans
                            "is_management": bool # True means this interface is used for device access
                            "addresses": {
                                "<cidr>": {
                                    "ip": str,        # e.g. "10.10.10.5/24" - use this as nmap source IP
                                    "is_target": bool,
                                    "is_management": bool,
                                }
                            },
                        }
                    }
                }
            }
        }
    }

    Guidance:
    - Use `is_target=True` interfaces/IPs when running nmap scans or attacks
    - Skip interfaces where `is_up=False`
    - Use `is_management=True` interfaces for device access or data extraction
    - `configuration` may be None if the device has not been configured yet
    """
    return asdict(config.scrub())
```

Listing 14: LangChain Tool: Host Configuration

```python
@tool  # type: ignore[misc]
def cli_tool(command: str, timeout: int = 120) -> tuple[int, str, str]:
    """
    Run an arbitrary CLI command and return its output.

    Use nmap for network scanning, e.g.:
    - "nmap -sV 10.10.10.1" (service version detection)
    - "nmap -sC -sV -oN output.txt 10.10.10.1" (default scripts + versions)
    - "nmap -p 1-1000 10.10.10.1" (port range)

    Use nmap NSE scripts for targeted service enumeration with --script <name> or --script <cat>:
    - Single script:   "nmap --script http-title -p 80 10.10.10.1"
    - Multiple:        "nmap --script http-title,http-headers -p 80,443 10.10.10.1"
    - With args:       "nmap --script smb-security-mode --script-args smbuser=guest -p 445 10.10.10.1"

    Service-to-script mapping — run these when the corresponding service is detected:
      HTTP / HTTPS / http-alt (80, 443, 8080…):
          http-title, http-headers, http-robots.txt
          e.g. "nmap --script http-title,http-headers,http-robots.txt -p 80 10.10.10.1"

      DNS / domain (53):
          dns-zone-transfer, dns-brute
          e.g. "nmap --script dns-zone-transfer,dns-brute -p 53 10.10.10.1"

      SMTP / SMTPS (25, 465, 587):
          smtp-commands, smtp-enum-users
          e.g. "nmap --script smtp-commands,smtp-enum-users -p 25 10.10.10.1"

      FTP (21):
          ftp-anon, ftp-ls
          e.g. "nmap --script ftp-anon,ftp-ls -p 21 10.10.10.1"

      SMB / NetBIOS (139, 445):
          smb-security-mode, smb2-security-mode
          e.g. "nmap --script smb-security-mode,smb2-security-mode -p 139,445 10.10.10.1"

      LDAP / LDAPS (389, 636):
          ldap-rootdse
          e.g. "nmap --script ldap-rootdse -p 389 10.10.10.1"

    Workflow: first run -sV to detect services, then re-scan open ports with the
    matching scripts above. Combine with -sV to keep version info:
      "nmap -sV --script http-title,http-headers -p 80,443 10.10.10.1"

    Use `apt install` if a cli tool is missing.
    Args:
        command: Full CLI command string to execute.
        timeout: Max seconds to wait (default 120, increase for large scans).

    Returns
    -------
        Tuple of (exit_code, stdout, stderr).
        Exit code 0 means success. Check stderr on failure.
    """
    try:
        completed_process = subprocess.run(  # noqa: S603
            shlex.split(command),
            capture_output=True,
            text=True,
            check=True,
            timeout=timeout,
```

Listing 15: LangChain Tool: CLI Tool

```python
@tool  # type: ignore[misc]
def human_interaction_hook(question: str) -> str:
    """
    Ask the human operator a question and return their answer.

    Use this tool when you need information, confirmation, or guidance that
    cannot be determined from the available tools or the current context.

    Args:
        question: The question or request to present to the human operator.

    Returns
    -------
        The human operator's response as a string.
    """
    return input(f"[Human Input Required]\n{question}\n> ")
```

Listing 16: LangChain Tool: Human Interaction Hook

```python
# Hardcoded for now!
grafana_link_template = "https://grafana.hiube.ch/a/grafana-lokiexplore-app/explore/nsak_run_uuid/{run_uuid}/logs?var-filters=nsak_run_uuid|%3D|{run_uu
template = """
Scenario: %(scenario)s\n
Run ID: %(run_uuid)s\n
Datetime: %(timestamp)s\n
\n
See logs in Grafana: %(grafana_link)s\n
\n
-------------------------------------------\n
\n
%(message)s\n
\n
"""


@tool  # type: ignore[misc]
def send_email(
    subject: str,
    message: str,
    cc_recipients: list[str] | None = None,
```

Listing 17: LangChain Tool: E-Mail Alerting

E.1.4. Scenario Implementations

```python
async def run(prompt: str, interactive: bool = True) -> None:
    """
    Scenario, which runs an AI agent with a custom prompt in an interactive loop.

    After each agent response the operator is prompted for the next instruction.
    Enter an empty line or 'exit' to stop the session.

    :return: None
    """

    logger.info("Starting scenario: AI Agent")

    ai_agent = await create_ai_agent(interactive)
    result = ai_agent.run_interactive(prompt)

    async for line in result:
        logger.warning(line)
```

Listing 18: Scenario: AI Agent - scenario.py

```yaml
scenarios:
  ai_agent:
    id: ai_agent
    name: AI Agent (custom prompt)
    author: vonal3@bfh.ch
    repository: https://gitlab.ti.bfh.ch/gausf1-vonal3/swiss-army-knife-network-sniffer

    drills:

    environments:

    dependencies:
      system:
      python:

    interface:
      arguments:
        prompt:
          type: str
        interactive:
          type: bool
          default: True
      return_type: None
```

Listing 19: Scenario: AI Agent - scenario.yaml

```python
reconnaissance_prompt_template = """
Steps:
1. Retrieve the network configuration with the `host_configuration` tool.
2. Discover all subnets, hosts and services with nmap on the following interface: %(interface)s
3. Enumerate all services based on the result of the network discovery result with service-specific nmap NSE scripts
4. Write a markdown formated assessment of your findings
5. Return the result as structured output: AIReconnaissanceAssessmentResult

"""

@dataclass(frozen=True, kw_only=True)
class AIReconnaissanceAssessmentResult:
    """
    Adhoc result type.
    """
    result: ReconnaissanceScenarioResult
    assessment: str

async def run_reconnaissance_agent(
    interface: str,
    interactive: bool = False,
) -> tuple[AIReconnaissanceAssessmentResult, AiAgent]:
    """
    Run an agent which returns a NetworkDiscoveryResultMap.
    """
    prompt = reconnaissance_prompt_template % {"interface": interface}

    if interactive:
        prompt += "\n5. Use the `human_interaction_hook` tool to allow the operator to execute followup steps."

    agent = await create_ai_agent(
        interactive,
        response_format=AIReconnaissanceAssessmentResult,
    )
    result = await agent.ainvoke(prompt)

    structured_response = result.get("structured_response")

    if structured_response is None:
        response = result.get("messages", [])[-1].content
        logger.warning("The value `result.structured_response` in `run_reconnaissance_agent` was None. Last message was %s (%s)." % (response, type(response)))
        logger.info("Try parsing the last response with `json.loads`.")
        try:
            data = json.loads(response)
            rows = [NetworkDiscoveryRow(**row) for row in data["result"]["network_discovery_table"]["rows"]]
            results = [EnumerateServicesResultEntry(**result) for result in data["result"]["enumerate_services_result"]["results"]]
            structured_response = AIReconnaissanceAssessmentResult(
                result=ReconnaissanceScenarioResult(
                    network_discovery_table=NetworkDiscoveryTable(rows=rows),
                    enumerate_services_result=EnumerateServicesResult(results=results)
                ),
                assessment=data["assessment"]
            )
            logger.info("Successfully parsed the last message with `json.loads`.")
        except Exception as e:
            logger.exception("Error while trying to parse the last message with `json.loads`.", exc_info=e)
            raise ValueError("Could get structured output from response!")

    return structured_response, agent
```

Listing 20: Scenario: AI Reconnaissance - scenario.py

```yaml
scenarios:
  ai_reconnaissance:
    id: ai_reconnaissance
    name: AI Reconnaissance
    author: vonal3@bfh.ch
    repository: https://gitlab.ti.bfh.ch/gausf1-vonal3/swiss-army-knife-network-sniffer

    drills:

    environments:
      - simple_tcp_client_server

    dependencies:
      system:
      python:

    interface:
      arguments:
        interface:
          type: str
        interactive:
          type: bool
          default: False
      return_type: AIReconnaissanceScenarioResult
```

Listing 21: Scenario: AI Reconnaissance - scenario.yaml

```
PACKET_COUNT = 200
CAPTURE_DURATION = 60  # seconds
PROMPT_TEMPLATE = """
You are analyzing network traffic for intrusion detection on interface %(interface)s.

Network Discovery Result Map:
%(host_discovery_result_map)s

Captured packets (CSV):
%(csv_summary)s

Identify at most 5 suspicious events if any. For each, output:
- src/dst IP and MAC
- protocol
- reason it is suspicious

Identify malicious hosts (MAC and/or IP) if any.

Be concise. Do not repeat packet data.
"""

async def run(interface: str, interactive: bool = False) -> None:
    """
    Scenario, which listens to network traffic and tries to detect intruders.

    :return: None
    """
    logger.info("Starting scenario: AI Intrusion Detection")

    host_discovery_result_map: NetworkDiscoveryResultMap = DrillManager.execute("discover_hosts",
    ↪  interface=interface)

    # Capture traffic and extract fields including MAC addresses
    result = subprocess.run(
        [
            "tshark",
            "-i", interface,
            "-c", str(PACKET_COUNT),
            "-a", f"duration:{CAPTURE_DURATION}",
            "-n",
            "-T", "fields",
            "-e", "frame.number",
            "-e", "eth.src",
            "-e", "eth.dst",
            "-e", "ip.src",
            "-e", "ip.dst",
            "-e", "tcp.dstport",
            "-e", "udp.dstport",
            "-e", "frame.protocols",
            "-e", "frame.len",
            "-E", "header=y",
            "-E", "separator=,",
        ],
        capture_output=True,
        text=True,
        timeout=CAPTURE_DURATION + 10,
    )

    csv_summary = result.stdout.strip()

    if not csv_summary:
        logger.info("No relevant traffic captured.")
        return

    prompt = PROMPT_TEMPLATE % {
        "interface": interface,
        "host_discovery_result_map": host_discovery_result_map.as_table(),
        "csv_summary": csv_summary,
    }

    ai_agent = await create_ai_agent(interactive)
    result = ai_agent.run(prompt)

    async for line in result:
        logger.warning(line)
```

Listing 22: Scenario: AI Intrusion Detection - scenario.py

```yaml
scenarios:
  ai_intrusion_detection:
    id: ai_intrusion_detection
    name: AI Intrusion Detection
    author: vonal3@bfh.ch
    repository: https://gitlab.ti.bfh.ch/gausf1-vonal3/swiss-army-knife-network-sniffer

    drills:

    environments:
      - simple_tcp_client_server

    dependencies:
      system: [ tshark ]
      python:

    interface:
      arguments:
        interface:
          type: str
        interactive:
          type: bool
          default: False
      return_type: None
```

Listing 23: Scenario: AI Intrusion Detection - scenario.yaml

E.2. Configuration Management: Source Listings

E.2.1. NSAK Core Configuration

```python
import logging
import os
from pathlib import Path

from dotenv import load_dotenv

load_dotenv(os.getenv("NSAK_ENV_FILE", None))

logger = logging.getLogger()


def get_base_path() -> Path:
    """
    Returns the default base path.

    :return:
    """
    base_path = os.getenv("NSAK_BASE_PATH", None)

    if base_path is not None:
        return Path(base_path).absolute()

    return Path(__file__).resolve().parents[3].absolute()


def get_run_path() -> Path:
    """
    Returns the run path.

    :return:
    """
    run_path = os.getenv("NSAK_RUN_PATH", None)

    if run_path is not None:
        return Path(run_path).absolute()

    return (get_base_path() / "run").absolute()


def get_library_paths() -> set[Path]:
    """
    Returns a list of library paths.

    :return:
    """
    library_paths = set()

    default_path = (BASE_PATH / "lib").absolute()
    if default_path.exists():
        library_paths.add(default_path)

    library_path = os.getenv("NSAK_LIBRARY_PATH", None)
    if library_path is not None:
        _library_path = Path(library_path).absolute()
        if not _library_path.exists():
            message = (
                f"Library path '{library_path}' in NSAK_LIBRARY_PATH does not exist!"
            )
            logger.warning(message)
        library_paths.add(_library_path)

    return library_paths


BASE_PATH = get_base_path()
RUN_PATH = get_run_path()
BENCHMARK_PATH = RUN_PATH.joinpath("benchmarks")
LIBRARY_PATHS = get_library_paths()
CONFIG_FILE = (RUN_PATH / "config.yaml").absolute()
DOCKER_CONTEXT = BASE_PATH
```


Listing 24: NSAK Static Configuration

```python
from __future__ import annotations

import logging
import uuid
from copy import deepcopy
from dataclasses import dataclass, field
from datetime import datetime
from pathlib import Path
from zoneinfo import ZoneInfo

from nsak.core.configuration.ai_configuration import AiConfiguration
from nsak.core.configuration.container_info import ContainerInfo
from nsak.core.configuration.drawio_mcp_configuration import DrawioMCPConfiguration
from nsak.core.configuration.email_configuration import EmailConfiguration
from nsak.core.configuration.loki_configuration import LokiConfiguration
from nsak.core.device import Device
from nsak.core.settings import RUN_PATH

logger = logging.getLogger(__name__)


def get_default_timezone() -> ZoneInfo:
    """
    Return the default timezone.
    """
    return ZoneInfo("Europe/Zurich")


@dataclass(kw_only=True)
class Configuration:
    """
    Class for loading and saving the configuration.
    """

    from ..device.device_manager import DeviceManager

    debug: bool = field(default=True)
    device: Device = field(
        default_factory=DeviceManager.get_default,
    )
    container: ContainerInfo = field(
        default_factory=ContainerInfo.load,
    )
    run_uuid: uuid.UUID = field(default_factory=uuid.uuid4)
    email: EmailConfiguration | None = field(
        default=None,
    )
    ai: AiConfiguration | None = field(default=None)
    loki: LokiConfiguration | None = field(default=None)
    drawio_mcp: DrawioMCPConfiguration | None = field(default=None)

    timezone: ZoneInfo = field(default_factory=get_default_timezone)
    timestamp: datetime = field(init=False)
    running_scenario: str | None = field(default=None)

    def __post_init__(self) -> None:
        """
        Initialize fields, which have dependencies to other fields.
        """
        self.timestamp = self.now()

    def set_running_scenario(self, scenario: str) -> None:
        """
        Set the name of the currently running scenario for logging.
        """
        from nsak.core.configuration import ConfigurationManager

        self.running_scenario = scenario
        ConfigurationManager.update_loki_handler(self)

    def reset_running_scenario(self) -> None:
        """
        Reset the running scenario name to none.
        """
        from nsak.core.configuration import ConfigurationManager

        self.running_scenario = None
        ConfigurationManager.update_loki_handler(self)

    def reset_run_uuid(self) -> uuid.UUID:
        """
        Set a new run uuid and return it.

        Used when multiple runs are executed in the same cli command.
        """
        self.run_uuid = uuid.uuid4()
        return self.run_uuid

    def save(self) -> None:
        """
        Shorthand proxy for ConfigManager save.
        """
        from .configuration_manager import ConfigurationManager
```

```python
import click

from .ai_agent import ai_agent_group
from .benchmark import benchmark_group
from .config import config_group
from .device import device_group
from .drill import drill_group
from .environment import environment_group
from .scenario import kill_all_scenarios, scenario_group


@click.group()
def cli() -> None:
    """
    CLI root.
    """
    # Load the configuration for initialization
    from nsak.core import config  # noqa


cli.add_command(scenario_group)
cli.add_command(drill_group)
cli.add_command(device_group)
cli.add_command(environment_group)
cli.add_command(ai_agent_group)
cli.add_command(config_group)
cli.add_command(benchmark_group)
cli.add_command(kill_all_scenarios, name="killswitch")
```

Listing 26: NSAK CLI Initialisation

```yaml
debug: true
device:
  id: bananapi_r4
  name: BananaPI R4
  path: <base-path>/lib/devices/bananapi_r4
  author: vonal3@bfh.ch
  repository: <repository-url>
  configuration:
    network:
      ethernets:
        lan1@eth0:
          addresses:
            10.10.10.30/24:
              is_target: true
              is_management: false
```

Listing 27: Persisted Runtime Configuration (run/config.yaml)

```
# Show usage, options and subcommands for the device resource
nsak device --list

# List all devices found in the resource library
nsak device list

# Output:
❯ nsak device list
ID           NAME         PATH
bananapi_r4  BananaPI R4  <repository>/lib/devices/bananapi_r4
```

Listing 28: NSAK CLI: List Devices

E.2.2. Device Resource Management

```yaml
devices:
    <str:resource-id>:
        id: <str:device-id>
        name: <str:device-name>
        description: <str:device-description>
        author: <str:author-email>
        repository: <str:resource-repository>
```

Listing 29: Device Resource YAML Specification

```yaml
devices:
    <str:resource-id>:
        # ...
        configuration:
          network:
            ethernets:
              <str:intreface-name>:
                addresses:
                  <str:cidr>:
                    is_target: <boolean>
                    is_management: <boolean>
```

Listing 30: Device Resource Configuration YAML Specification

```
class DeviceLoader(ResourceLoader[Device]):
    """
    Finds and loads devices from the library paths.
    """

    ResourceClass = Device

    @classmethod
    def _parse_device_configuration(
        cls, data: dict[str, Any]
    ) -> DeviceConfigration | None:
        if "configuration" not in data:
            return None
        network = data["configuration"].get("network") or {}
        if network == "auto":
            ethernets = {}
            for interface in get_network_interfaces():
                ethernets[interface.name] = {
                    "addresses": {
                        ip: {
                            "is_target": "lo" != interface.name,
                            "is_management": False,
                        }
                        for ips in interface.ips.values()
                        for ip in ips
                    }
                }
        else:
            ethernets = network.get("ethernets") or {}

        return DeviceConfigration(
            raw=data["configuration"],
            network=NetworkConfiguration(
                ethernets={
                    interface: EthernetConfiguration(
                        name=interface,
                        addresses={
                            ip: IPConfiguration(
                                ip=ip_interface(ip),
                                is_target=address.get("is_target", False),
                                is_management=address.get("is_management", False),
                            )
                            for ip, address in ethernet.get("addresses", {}).items()
                        },
                    )
                    for interface, ethernet in ethernets.items()
                }
            ),
        )

    @classmethod
    def _to_resource(cls, id: str, data: dict[str, Any], path: Path | str) -> Device:
        """
        Creates a Device object from a dict containing the device's metadata.
        """
        if isinstance(path, str):
            path = Path(path)

        return cls.ResourceClass(
            id=id,
            name=str(data.get("name") or ""),
            description=str(data.get("description") or ""),
            path=path,
            author=str(data.get("author") or ""),
            repository=str(data.get("repository") or ""),
            configuration=cls._parse_device_configuration(data),
        )

```

Listing 31: DeviceLoader Configuration Parsing

```python
@dataclasses.dataclass(frozen=True, kw_only=True)
class IPConfiguration:
    """
    Represents an ip configuration on an interface.
    """

    ip: IPInterface
    is_target: bool
    is_management: bool


@dataclasses.dataclass(frozen=True, kw_only=True)
class EthernetConfiguration:
    """
    Represents an ethernet node.
    """

    name: str
    addresses: dict[str, IPConfiguration]
```

```python
@dataclasses.dataclass(frozen=True, kw_only=True, eq=True)
class DeviceConfigration:
    """
    Represents the device configuration.
    """

    raw: object
    network: NetworkConfiguration


@dataclasses.dataclass(frozen=True, kw_only=True, eq=True)
class Device(Resource):
    """
    Represents a device.
    """

    NAME = "device"
    KEY = "devices"

    id: str
    name: str
    description: str
    path: Path
    author: str
    repository: str
    configuration: DeviceConfigration | None = None
```

Listing 32: Device Configuration Datastructures

```
# List all subcommands for the Device resource
nsak device --help

# List all available devices
nsak device list

# Show the device details and configuration
nsak device show <str:device-id>

# Load a device into in the NSAK configuration
nsak device load <str:device-id>

# Show the currently loaded device stored in the NSAK configuration
nsak device loaded

# Reset the loaded device
nsak device unload
```

Listing 33: Device Configuration CLI Management

E.2.3. Resource YAML Specifications

```
# Resource type version was removed, as it was not used anyway

# scenarios|drills|environments|devices
<str:resource-type-key>:
    # unique resource identifier
    <str:resource-id>:
        # The "metadata" key is removed and the children are now nested directly under the resource
        name: <str:name>
        description: <str:description>
        author: <str:email>
        repository: <str:link>
        # ... resource type specific properties, which previously were at the top level
```

Listing 34: Mergeable Resource YAML Specification

```
<str:resource-type-key>:
    <str:resource-id>:
        # Unchanged properties...

        interface:
            arguments:
                <str:argument-name>:
                    type: <str:type-annotation>
                    default: <any:default-value>
            return_type: <str:type-annotation>
```

Listing 35: Updated Interface Specification

```yaml
scenarios:
  mitm:
    id: mitm
    name: MITM Scenario
    author: vonal3@bfh.ch
    repository: https://gitlab.ti.bfh.ch/gausf1-vonal3/swiss-army-knife-network-sniffer

    drills:
      - discover_hosts
      - arp_spoof
      - transparent_tcp_proxy

    environments:
      - simple_tcp_client_server

    dependencies:
      system:
        - arp-scan
        - dsniff
        - iptables
      python:
        - scapy

    interface:
      arguments:
        interface:
          type: str
      return_type: None
```

Listing 36: mitm Scenario YAML

E.3. Reconnaissance Scenario: Source Listings

```python
"""
Scenario entrypoint for Reconnaissance .
"""
import logging

from scapy.arch import get_if_addr

from nsak.core import DrillManager
from nsak.core.network import NetworkDiscoveryResultMap
from nsak.core.network.enumerate_services_result import EnumerateServicesResult
from nsak.core.scenario_results.reconnaissance_scenario_result import ReconnaissanceScenarioResult

logger = logging.getLogger(__name__)


def run(interface: str | None = None, subnet: str | None = None) -> ReconnaissanceScenarioResult:
    """
    Scenario, which runs Reconnaissance attack.
    1. If no interface is specified, scan all active physical interfaces
    2. If no interface is specified, scan all active physical interfaces
    3. Discover network on ifc and subnet
    :return: None
    """
    if interface is None:
        discovered = DrillManager.execute("discover_network_interfaces")
        interfaces_to_scan = [iface.name for iface in discovered]
    else:
        interfaces_to_scan = [interface]

    all_results = {}
    for iface_name in interfaces_to_scan:
        if get_if_addr(iface_name) in ("0.0.0.0", ""):
            DrillManager.execute("dhcp_request", interface=iface_name)

        network_discovery_result_map: NetworkDiscoveryResultMap = DrillManager.execute(
            "discover_hosts",
            interface=iface_name,
            subnet=subnet,
        )
        all_results.update(network_discovery_result_map.results)

    network_discovery_result_map = NetworkDiscoveryResultMap(results=all_results)
    DrillManager.execute("port_scan", discovery_result=network_discovery_result_map)
    enumerate_services_result: EnumerateServicesResult = DrillManager.execute(
        "enumerate_services", discovery_result=network_discovery_result_map
    )

    result = ReconnaissanceScenarioResult(
        network_discovery_table=network_discovery_result_map.to_network_discovery_table(),
        enumerate_services_result=enumerate_services_result
    )
    logger.info(result.display())
    return result
```

Listing 37: Reconnaissance Scenario Implementation

```yaml
drills:
  discover_hosts:
    name: Discover Hosts
    author: vonal3@bfh.ch
    repository: https://gitlab.ti.bfh.ch/gausf1-vonal3/swiss-army-knife-network-sniffer

    dependencies:
      system:
        - arp-scan
        - nmap
      python: [ ]

    interface:
      arguments:
        interface:
          type: str
        subnet:
          type: str
          default: null
      return_type: NetworkDiscoveryResultMap
```

Listing 38: Discover Hosts Drill: Resource Configuration

```python
import logging
import dataclasses
import subprocess
from ipaddress import IPv4Address, IPv6Address

from nsak.core import IPAddress
from nsak.core import config
from nsak.core.network import NetworkDiscoveryResultMap, NetworkDiscoveryResult, NetworkService, NetworkServiceEndpoint
from nsak.core.network.configuration import EthernetConfiguration

logger = logging.getLogger(__name__)


@dataclasses.dataclass(frozen=True, kw_only=True)
class ARPScanResult:
    ip: IPAddress
    mac: str
    vendor: str


def parse_arp_scan_result(arp_scan_result_output: str) -> list[ARPScanResult]:
    """

    :param arp_scan_result_output:
    :return:
    """

    entries = []
    for line in arp_scan_result_output.splitlines():
        parts = line.split(None, 2)
        if len(parts) < 2:
            continue

        raw_ip, mac = parts[:2]
        try:
            ip = IPv4Address(raw_ip)
        except ValueError:
            ip = IPv6Address(raw_ip)
        vendor = parts[2] if len(parts) == 3 else ""
        entries.append(ARPScanResult(ip=ip, mac=mac, vendor=vendor))

    return entries


def parse_nmap_result(nmap_output: str) -> list[ARPScanResult]:
    """
    Parses the output of `nmap -sn` into a list of ARPScanResults.

    Uses a stateful approach since IP and MAC appear on separate lines per host.
    Hosts without a MAC address (e.g. remote subnets) are included with an empty mac field.
    """
    results = []
    current_ip = None
    current_mac = ""
    current_vendor = ""

    for line in nmap_output.splitlines():
        if line.startswith("Nmap scan report for "):
            if current_ip is not None:
                results.append(ARPScanResult(ip=current_ip, mac=current_mac, vendor=current_vendor))
            rest = line.removeprefix("Nmap scan report for ")
            raw_ip = rest.split("(")[1].rstrip(")") if "(" in rest else rest.strip()
            try:
                current_ip = IPv4Address(raw_ip)
            except ValueError:
                try:
                    current_ip = IPv6Address(raw_ip)
                except ValueError:
                    current_ip = None
            current_mac = ""
            current_vendor = ""
        elif line.startswith("MAC Address:") and current_ip is not None:
            parts = line.split(None, 3)
            current_mac = parts[2] if len(parts) > 2 else ""
            current_vendor = parts[3].strip("()") if len(parts) > 3 else ""

    if current_ip is not None:
        results.append(ARPScanResult(ip=current_ip, mac=current_mac, vendor=current_vendor))

    return results


def discover_hosts(network_interface: EthernetConfiguration) -> NetworkDiscoveryResult:
    hosts: list[NetworkService] = []
    # --localnet: Generates addresses from the interface configuration
    # -x: Only prints results to the output (no headers, etc.)
    completed_process = subprocess.run([
        "/usr/sbin/arp-scan",
        "--interface", network_interface.name,
        "--localnet", "-x",
    ], capture_output=True, text=True, check=True)

```

```python
import logging
import re
import subprocess
from typing import Literal

from nsak.core.network import NetworkDiscoveryResultMap, NetworkService, NetworkServiceEndpoint

logger = logging.getLogger(__name__)

# matches: "22/tcp open  ssh     OpenSSH 8.9p1"
_PORT_LINE = re.compile(r"^(\d+)/(tcp|udp)\s+open\s+(\S+)\s*(.*)") #compile regex once in obj (p,prot,name,version,


def parse_nmap(stdout: str) -> list[tuple[int, Literal["tcp", "udp"], str, str]]:
    """Extract open ports from nmap stdout as (port, protocol, name, version) tuples.

    :param stdout: Raw nmap output text.
    :return: List of (port, protocol, service name, version) tuples for open ports.
    """
    results = []
    # compare the output with the regex and skip if output does not match | group(0)=whole line, group(1)=port...)
    for line in stdout.splitlines():
        match = _PORT_LINE.match(line.strip())
        if match:
            port = int(match.group(1))
            protocol = match.group(2)
            name = match.group(3)
            version = match.group(4).strip()
            results.append((port, protocol, name, version))
    return results


def run(discovery_result: NetworkDiscoveryResultMap) -> NetworkDiscoveryResultMap:
    """Run nmap service scan on all target IPs and append open ports to the discovery result.

    :param discovery_result: Existing host discovery result containing target IPs per interface.
    :return: The same result map, with open port/service information.
    """
    for iface_name, result in discovery_result.results.items():
        mac_by_ip = {
            endpoint.ip: endpoint.mac
            for service in result.network_services
            for endpoint in service.endpoints
            if endpoint.ip is not None and endpoint.mac
        }
        for ip in result.target_ips:
            logger.debug("Scanning ports on %s (%s)", ip, iface_name)
            proc = subprocess.run(
                ["nmap", "-sV", "--open", str(ip)],
                capture_output=True, text=True, check=True
            )
            ports = parse_nmap(proc.stdout)
            logger.debug("Found %d open ports on %s", len(ports), ip)
            for port, protocol, name, version in ports:
                service = NetworkService(
                    endpoints=[NetworkServiceEndpoint(ip=ip, mac=mac_by_ip.get(ip), port=port, protocol=protocol)],
                    state="open",
                    name=name,
                    version=version or None,
                )
                result.network_services.append(service)
    return discovery_result
```

Listing 40: Port Scan Drill: Implementation

```python
import logging
import re
import subprocess

from nsak.core.network import NetworkDiscoveryResultMap
from nsak.core.network.enumerate_services_result import EnumerateServicesResult,
↪  EnumerateServicesResultEntry

logger = logging.getLogger(__name__)

_NSE_LINE = re.compile(r"^\|[_ ]?\s*(.*)")  # nmap NSE output: "| text" or "|_ text"

# Known services get targeted scripts; everything else falls back to "banner"
# (banner grabs the initial TCP response — gives software name + version for any unknown service)
_SERVICE_SCRIPTS: dict[str, list[str]] = {
    "http":         ["http-title", "http-headers", "http-robots.txt"],
    "https":        ["http-title", "http-headers", "http-robots.txt"],
    "http-alt":     ["http-title", "http-headers", "http-robots.txt"],
    "domain":       ["dns-zone-transfer", "dns-brute"],
    "smtp":         ["smtp-commands", "smtp-enum-users"],
    "smtps":        ["smtp-commands"],
    "ftp":          ["ftp-anon", "ftp-ls"],
    "netbios-ssn":  ["smb-security-mode", "smb2-security-mode"],
    "microsoft-ds": ["smb-security-mode", "smb2-security-mode"],
    "ldap":         ["ldap-rootdse"],
    "ldapssl":      ["ldap-rootdse"],
}


def _parse_nse_output(stdout: str) -> list[str]:
    """
    Extract script result lines from nmap stdout.

    :param stdout: Raw nmap output.
    :return: Parsed output lines with pipe prefixes stripped.
    """
    findings = []
    for line in stdout.splitlines():
        m = _NSE_LINE.match(line.strip())
        if m:
            text = m.group(1).strip()
            if text:
                findings.append(text)
    return findings


def run(discovery_result: NetworkDiscoveryResultMap) -> EnumerateServicesResult:
    """
    Run service-specific nmap NSE scripts on all discovered services.

    :param discovery_result: Port-scan result from the port_scan drill.
    :return: Mapping of "ip:port" to a list of finding strings.
    """
    results: list[EnumerateServicesResultEntry] = []

    for iface_name, result in discovery_result.results.items():
        for service in result.network_services:
            scripts = _SERVICE_SCRIPTS.get(service.name or "", ["banner"])

            for endpoint in service.endpoints:
                if endpoint.ip is None or endpoint.port is None:
                    continue

                key = f"{endpoint.ip}:{endpoint.port}"
                logger.debug("Running NSE %s on %s (%s)", ",".join(scripts), key, iface_name)

                proc = subprocess.run(
                    [
                        "nmap", "--script", ",".join(scripts),
                        "-p", str(endpoint.port),
                        "-sT", "-Pn", "--host-timeout", "30s",
                        str(endpoint.ip),
                    ],
                    capture_output=True, text=True,
                )

                findings = _parse_nse_output(proc.stdout)
                if findings:
                    entry = EnumerateServicesResultEntry(
                        ip=endpoint.ip,
                        port=endpoint.port,
                        findings="\n".join(findings)
                    )
                    results.append(entry)
                    logger.debug("Found %d findings on %s", len(findings), key)

    return EnumerateServicesResult(results=results)
```


Listing 41: Enumerate Services Drill: Implementation

E.4. Test-Environment-Code

```
name: virtual-enterprise-network-lab

#  ┌──────────────────────────────────────────────────────────────────────────┐
#  │  Virtual Enterprise Network Lab                                          │
#  │                                                                          │
#  │  Authorized Recon & Enumeration Training Lab                             │
#  │  8 Nodes, 3 Zones — physically replicable with Raspberry Pis / old HW    │
#  │                                                                          │
#  │  WAN  10.0.1.0/24                                                        │
#  │                                                                          │
#  │    ┌──────────────────────────────────────────────────────────────┐      │
#  │    │  [external]  10.0.1.10  — neutral outside host (firewall testing)  │
#  │    └──────────────────────┬───────────────────────────────────────┘      │
#  │                           │ wan-br                                       │
#  │  [host: wan-br 10.0.1.254]  ← default gateway / "internet" (host bridge) │
#  │                           │                                              │
#  │                  [firewall]  eth1:10.0.1.1 / eth2:172.16.1.1 / eth3:192.168.10.1
#  │              ┌────────────┴──────────────────────────────┐               │
#  │              │ dmz-br  │                          lan-br                │
#  │  DMZ  172.16.1.0/24                    LAN  192.168.10.0/24              │
#  │    ┌──────────────────────┐      ┌──────────────────────────────────┐    │
#  │    │  dmz-server  .10     │      │  ctrl-server .5  22,139,389,445  │    │
#  │    │    80,443  HTTP/S    │      │  alice   .100  22,445            │    │
#  │    │    53      DNS/AXFR  │      │  bob     .101  22,445            │    │
#  │    │                      │      │  printer  .50  80,161/udp,631    │    │
#  │    └──────────────────────┘      │  nsak-lan .200  (recon node, dual-homed)
#  │                                  └──────────────────────────────────┘    │
#  │                                                                          │
#  │  Firewall policy (nftables):                                             │
#  │    WAN → DMZ  : 80/443 + 53 → dmz-server only                            │
#  │    WAN → LAN  : BLOCKED                                                  │
#  │    DMZ → LAN  : BLOCKED                                                  │
#  │    LAN → DMZ  : ALLOWED (employees browse DMZ services)                  │
#  │    LAN → WAN  : HTTP/HTTPS only via NAT                                  │
#  │    LAN → LAN  : ALLOWED                                                  │
#  │                                                                          │
#  │  Credentials:                                                            │
#  │    SSH alice    asmith  / Password123!                                   │
#  │    SSH bob      bjones  / Password123!                                   │
#  │    SSH ctrl-server  admin   / Password123!                               │
#  │    SNMP printer public  (community string, read-only)                    │
#  │                                                                          │
#  │  Prerequisites:                                                          │
#  │    sudo ip link add wan-br type bridge && sudo ip link set wan-br up     │
#  │    sudo ip link add dmz-br type bridge && sudo ip link set dmz-br up     │
#  │    sudo ip link add lan-br type bridge && sudo ip link set lan-br up     │
#  │    sudo ip addr add 10.0.1.254/24 dev wan-br  # host = WAN gateway       │
#  │    docker build -t nsak:latest .                                         │
#  │                                                                          │
#  │  Deploy:   sudo containerlab deploy -t virtual_enterprise_network_lab.clab.yaml
#  │  Redeploy: sudo containerlab deploy -t virtual_enterprise_network_lab.clab.yaml --reconfigure
#  │  Destroy:  sudo containerlab destroy -t virtual_enterprise_network_lab.clab.yaml
#  │  Shell:    docker exec -it clab-virtual-enterprise-network-lab-nsak bash
#  │  Sniff:    sudo tcpdump -i lan-br -n                                     │
#  └──────────────────────────────────────────────────────────────────────────┘
```

Listing 42: Containerlab Topology YAML: Overview

```yaml
mgmt:
  network: clab-mgmt
  ipv4-subnet: 172.20.20.0/24
  ipv6-subnet: ""

topology:
  defaults:
    kind: linux
    sysctls:
      net.ipv6.conf.all.disable_ipv6: 1
      net.ipv6.conf.default.disable_ipv6: 1
  nodes:

    # ── L2 Bridges ──────────────────────────────────────────────────────
    # Virtual switches that connect nodes within each zone.
    # In a physical setup these would be unmanaged switches.
    wan-br:
      kind: bridge
    dmz-br:
      kind: bridge
    lan-br:
      kind: bridge
    # ── External Segment (WAN 10.0.1.0/24) ──────────────────────────────
    # Simulates the external internet — a neutral outside host, not an attacker.
    # Used to verify firewall rules from the outside perspective:
    #   "What can someone on the internet actually reach?"
    # Attack scenarios are run by nsak (internal, compromised-host model).
    #
    # Reachable through firewall:
    #   172.16.1.10:80,443   dmz-server (HTTP/HTTPS)
    #   172.16.1.10:53       dmz-server (DNS — zone transfer enabled)
    # NOT reachable: anything in LAN (192.168.10.0/24)
```

Listing 43: Containerlab Topology YAML: Network

```
    # ── Firewall (Alpine + nftables) ──────────────────────────────────────
    # Tri-homed router/firewall: eth1=WAN, eth2=DMZ, eth3=LAN.
    # All inter-zone traffic is filtered here.
    # ip_forward enables kernel-level packet routing between interfaces.
    firewall:
      kind: linux
      image: alpine:3.19
      cmd: tail -f /dev/null
      cap-add:
        - NET_ADMIN
      sysctls:
        net.ipv4.ip_forward: 1  # required to forward packets between interfaces
      exec:
        - apk add --no-cache nftables iproute2
        - ip addr add 10.0.1.1/24 dev eth1
        - ip addr add 172.16.1.1/24 dev eth2
        - ip addr add 192.168.10.1/24 dev eth3
        - ip route replace default via 10.0.1.254
        # Default FORWARD policy: drop everything not explicitly allowed
        - nft add table inet filter
        - sh -c "nft add chain inet filter forward '{ type filter hook forward priority 0; policy drop; }'"
        # Allow return traffic for already-established connections (stateful firewall)
        - nft add rule inet filter forward ct state established,related accept
        # WAN → DMZ: only HTTP/HTTPS and DNS to dmz-server
        - sh -c "nft add rule inet filter forward iifname eth1 oifname eth2 ip daddr 172.16.1.10 tcp dport
        ↪  '{ 80, 443 }' accept"
        - nft add rule inet filter forward iifname eth1 oifname eth2 ip daddr 172.16.1.10 udp dport 53
        ↪  accept
        - nft add rule inet filter forward iifname eth1 oifname eth2 ip daddr 172.16.1.10 tcp dport 53
        ↪  accept
        # WAN → LAN: completely blocked (no rules = default DROP applies)
        # DMZ → LAN: completely blocked
        - nft add rule inet filter forward iifname eth2 oifname eth3 drop
        # LAN → DMZ: allowed (internal users browse DMZ services)
        - nft add rule inet filter forward iifname eth3 oifname eth2 accept
        # LAN → WAN: HTTP/HTTPS only, with source NAT (masquerade)
        - sh -c "nft add rule inet filter forward iifname eth3 oifname eth1 tcp dport '{ 80, 443 }' accept"
        # NAT: source masquerade for LAN and DMZ egress to WAN
        - nft add table ip nat
        - sh -c "nft add chain ip nat postrouting '{ type nat hook postrouting priority 100; }'"
        - nft add rule ip nat postrouting ip saddr 192.168.10.0/24 oifname eth1 masquerade
        - nft add rule ip nat postrouting ip saddr 172.16.1.0/24 oifname eth1 masquerade
        # DMZ → WAN: DNS forwarding (dmz-server forwards unknown queries to 8.8.8.8)
        - nft add rule inet filter forward iifname eth2 oifname eth1 udp dport 53 accept
        - nft add rule inet filter forward iifname eth2 oifname eth1 tcp dport 53 accept
```

Listing 44: Containerlab Topology YAML: Firewall

```
    # ──────────────────────────────────────────────────────────────────────
    #  DMZ Nodes  (172.16.1.0/24)
    # ──────────────────────────────────────────────────────────────────────

    # ── DMZ: Combined Web + DNS Server ──────────────────────────────────
    # Single DMZ node running both nginx (web) and BIND9 (DNS).
    # Merging saves a device slot while covering two distinct attack surfaces.
    #
    # Web (nginx):
    #   80/tcp   HTTP  — corporate landing page, /robots.txt, /api/v1/status
    #   443/tcp  HTTPS — self-signed cert: CN=dmz-server.lab.local
    #   Recon: Server header leaks version; /robots.txt disallows /admin /backup
    #
    # DNS (BIND9):
    #   53/udp+tcp  queries and zone transfer
    #   AXFR intentionally open (misconfiguration) — reveals all internal IPs
    #   Recon: dig @172.16.1.10 vlab.local AXFR  → full zone dump
    dmz-server:
      kind: linux
      image: alpine:3.19
      cmd: tail -f /dev/null
      cap-add:
        - NET_ADMIN
      binds:
        - scripts/dmz-web-server:/dmz-web-server:ro
      exec:
        - apk add --no-cache nginx bind bind-tools iproute2 openssl python3
        - ip addr add 172.16.1.10/24 dev eth1
        - ip route replace default via 172.16.1.1
        - sh /dmz-web-server/setup.sh
```

Listing 45: Containerlab Topology YAML: DMZ

```
163     # ──────────────────────────────────────────────────────────────────────
164     #  LAN Nodes  (192.168.10.0/24)
165     # ──────────────────────────────────────────────────────────────────────
166
167     # ── LAN: Internal Server (Samba + LDAP + SSH) ──────────────────────────
168     # Combines Domain Controller and File Server into one node.
169     # Samba provides SMB shares; OpenLDAP provides a lightweight directory.
170     # No full AD provisioning — covers the same recon/attack surface at less cost.
171     #
172     #   22/tcp   SSH   (admin / Password123!)
173     #   139/tcp  NetBIOS (Samba)
174     #   389/tcp  LDAP  — anonymous base read: DC=lab,DC=local (user list exposed)
175     #   445/tcp  SMB   — shares: public (anon read), finance (auth), it (auth)
176     #
177     # Recon:
178     #   smbclient -L //192.168.10.5 -N         → lists all shares
179     #   smbmap -H 192.168.10.5                 → share names + permissions
180     #   ldapsearch -x -H ldap://192.168.10.5   → user list, group memberships
181     ctrl-server:
182       kind: linux
183       image: alpine:3.19
184       cmd: tail -f /dev/null
185       cap-add:
186         - NET_ADMIN
187         - SYS_ADMIN  # needed for Samba AD provisioning
188       binds:
189         - scripts/ctrl-file-server:/ctrl-file-server:ro
190       exec:
191         - apk add --no-cache samba openldap openldap-back-mdb openldap-clients openssh iproute2
192         - ip addr add 192.168.10.5/24 dev eth1
193         - ip route replace default via 192.168.10.1
194         - sh -c 'echo "nameserver 172.16.1.10" > /etc/resolv.conf'
195         - sh /ctrl-file-server/server_setup.sh
```

Listing 46: Containerlab Topology YAML: Control Server

```
    # ── LAN: Workstation alice (Finance dept, user: asmith) ──────────────────
    # Simulates a Finance department workstation.
    # Sensitive documents on disk; drive mappings reveal share paths.
    #
    #   22/tcp   SSH  (asmith / Password123!)
    #   445/tcp  SMB  (Samba standalone)
    #
    # Recon:
    #   SSH banner leaks org name and policy
    #   /home/asmith/Documents/Q3_Finance.txt — confidential data (post-exploit)
    #   /home/asmith/Desktop/drive_mappings.txt — reveals \\ctrl-server\finance path
    alice:
      kind: linux
      image: alpine:3.19
      cmd: tail -f /dev/null
      cap-add:
        - NET_ADMIN
      exec:
        - apk add --no-cache openssh samba iproute2
        - ip addr add 192.168.10.100/24 dev eth1
        - ip route replace default via 192.168.10.1
        - sh -c 'echo "nameserver 172.16.1.10" > /etc/resolv.conf'
        - ssh-keygen -A
        - sh -c 'echo "PasswordAuthentication yes" >> /etc/ssh/sshd_config'
        - sh -c 'printf "NSAK-Enterprise - Authorized Access Only\nThis system is monitored.\n" >
        ↪  /etc/ssh/banner.txt'
        - sh -c 'echo "Banner /etc/ssh/banner.txt" >> /etc/ssh/sshd_config'
        - adduser -D -s /bin/sh asmith
        - sh -c 'echo "asmith:Password123!" | chpasswd'
        - mkdir -p /home/asmith/Documents /home/asmith/Desktop
        - sh -c 'echo "Q3 2024 Finance Report - CONFIDENTIAL" > /home/asmith/Documents/Q3_Finance.txt'
        - sh -c 'printf "DOMAIN=LAB\nUSER=asmith\nSHARE=\\\\\\\\ctrl-server\\\\finance\n" >
        ↪  /home/asmith/Desktop/drive_mappings.txt'
        - /usr/sbin/sshd
```

Listing 47: Containerlab Topology YAML: Alice (Client)

```
230     # ── LAN: Workstation bob (IT dept, user: bjones) ──────────────────────────
231     # Simulates an IT department workstation.
232     # bjones has elevated rights — scripts directory reveals full internal topology.
233     #
234     #   22/tcp   SSH  (bjones / Password123!)
235     #   445/tcp  SMB  (Samba standalone)
236     #
237     # Recon:
238     #   /home/bjones/Scripts/hosts.txt  — lists all internal IPs and roles
239     #   /home/bjones/Scripts/README.txt — hints at admin access
240     bob:
241       kind: linux
242       image: alpine:3.19
243       cmd: tail -f /dev/null
244       cap-add:
245         - NET_ADMIN
246       exec:
247         - apk add --no-cache openssh samba iproute2
248         - ip addr add 192.168.10.101/24 dev eth1
249         - ip route replace default via 192.168.10.1
250         - sh -c 'echo "nameserver 172.16.1.10" > /etc/resolv.conf'
251         - ssh-keygen -A
252         - sh -c 'echo "PasswordAuthentication yes" >> /etc/ssh/sshd_config'
253         - sh -c 'printf "Acme Corp AG - Authorized Access Only\nThis system is monitored.\n" >
            ↪  /etc/ssh/banner.txt'
254         - sh -c 'echo "Banner /etc/ssh/banner.txt" >> /etc/ssh/sshd_config'
255         - adduser -D -s /bin/sh bjones
256         - sh -c 'echo "bjones:Password123!" | chpasswd'
257         - mkdir -p /home/bjones/Documents /home/bjones/Scripts
258         - sh -c 'printf "ctrl-server=192.168.10.5\nalice=192.168.10.100\nprinter=192.168.10.50\n" >
            ↪  /home/bjones/Scripts/hosts.txt'
259         - sh -c 'echo "# admin tooling — see ctrl-server at 192.168.10.5" >
            ↪  /home/bjones/Scripts/README.txt'
260         - /usr/sbin/sshd
```

Listing 48: Containerlab Topology YAML: Bob (Client)

```
    # ── LAN: Network Printer (HP-like, intentionally misconfigured) ──────────────
    # Simulates a typical enterprise network printer — the "forgotten device".
    # Rarely patched, often misconfigured, rich source of OSINT and lateral movement.
    #
    #   80/tcp    HTTP  — unauthenticated web admin interface
    #   161/udp   SNMP  — community string "public" (v1/v2c, full device info)
    #   631/tcp   IPP   — Internet Printing Protocol (job history, no auth)
    #
    # Recon:
    #   snmpwalk -v2c -c public 192.168.10.50  → model, serial, toner, job list, usernames
    #   curl http://192.168.10.50              → unauthenticated admin page, firmware version
    #   curl http://192.168.10.50:631/jobs     → print job history reveals internal usernames
    printer:
      kind: linux
      image: alpine:3.19
      cmd: tail -f /dev/null
      cap-add:
        - NET_ADMIN
        - NET_BIND_SERVICE  # needed to bind to privileged ports 80 and 631
      binds:
        # printer_sim.py serves the web admin UI (port 80) and IPP (port 631)
        - scripts/printer_sim.py:/printer_sim.py:ro
      exec:
        - apk add --no-cache python3 net-snmp iproute2
        - ip addr add 192.168.10.50/24 dev eth1
        - ip route replace default via 192.168.10.1
        - sh -c 'echo "nameserver 172.16.1.10" > /etc/resolv.conf'
        # SNMP with open public community string — intentional misconfiguration
        - sh -c 'printf "rocommunity public\nsysName HP-LaserJet-M428fdw\nsysLocation Server Room
        ↪  B2\nsysContact it@lab.local\n" > /etc/snmp/snmpd.conf'
        - sh -c 'snmpd -f -Lo &'
        # Web admin UI (80) and IPP job listener (631)
        - sh -c 'python3 /printer_sim.py > /var/log/printer_sim.log 2>&1 &'
```

Listing 49: Containerlab Topology YAML: Printer

```
    # ── NSAK — Swiss Army Knife (Reconnaissance Node) ──────────────────────────
    # The nsak tool itself runs here. Dual-homed to observe both LAN and DMZ.
    #   eth1 → LAN 192.168.10.200  (full internal network visibility)
    #   eth2 → DMZ 172.16.1.200    (DMZ visibility — same segment as dmz-server)
    #
    # Quickstart:
    #   docker exec -it clab-virtual-enterprise-network-lab-nsak bash
    #   nsak drill execute discover_hosts --interface eth1
    #   nsak drill execute discover_hosts --interface eth2
    #
    # Prerequisite: docker build -t nsak:latest .  (from project root)
    nsak:
      kind: linux
      image: nsak:latest
      entrypoint: /usr/bin/tail
      cmd: "-f /dev/null"
      cap-add:
        - NET_ADMIN
        - NET_RAW  # needed for raw socket access (Scapy, tcpdump, ping)
      env:
        NSAK_BASE_PATH: /nsak
      exec:
        - apt-get install -y isc-dhcp-client smbclient ldap-utils
        - ip addr add 192.168.10.200/24 dev eth1
        - ip route replace default via 172.20.20.1 dev eth0
        - sh -c 'echo "nameserver 8.8.8.8" > /etc/resolv.conf'
      mgmt-ipv4: 172.20.20.10
```

Listing 50: Containerlab Topology YAML: NSAK

```
  links:
    # WAN segment (WAN 10.0.1.0/24)
    - endpoints: [ "firewall:eth1", "wan-br:fw1" ]
    # DMZ segment (172.16.1.0/24)
    - endpoints: [ "firewall:eth2", "dmz-br:fw2" ]
    - endpoints: [ "dmz-server:eth1", "dmz-br:dmz-serv" ]
    # LAN segment (192.168.10.0/24)
    - endpoints: [ "firewall:eth3", "lan-br:fw3" ]
    - endpoints: [ "ctrl-server:eth1", "lan-br:ctrl" ]
    - endpoints: [ "alice:eth1", "lan-br:A" ]
    - endpoints: [ "bob:eth1", "lan-br:B" ]
    - endpoints: [ "printer:eth1", "lan-br:print" ]
    - endpoints: [ "nsak:eth1", "lan-br:nsak-lan" ]
```

Listing 51: Containerlab Topology YAML: Links

F. Service and Tooling

During the risk assessment **??** we identified that the AI tokens required to use cloud based LLMs might be very expensive, and it's really hard to estimate the costs. One of the possibilities to avoid this risk was the usage of self-hosted LLMs, and luckily we already had existing hardware at home which is suitable for running smaller LLMs. Besides eliminating the costs for AI tokens, at least during the development phase, it also allows the quick evaluation of different AI Models. Depending on the quality and performance of the smaller models running in the self-hosted setup we can consider not using a cloud based LLM at all.

Refactor the section: Move all information in their respective sections, etc.

F.1. Hardware Setup

Component	Specification
CPU	AMD Ryzen 5 3600 (6 cores / 12 threads, up to 4.2 GHz)
GPU	NVIDIA GeForce RTX 3070 (8 GB VRAM)
System Memory	32 GB RAM

Table F.1.: Hardware configuration of the self-hosted LLM inference server.

The 8 GB VRAM of the RTX 3070 allows us to run models with up to 7 billion parameters, depending on the specific model [31].

F.2. System Setup

The system setup described in 54 will configure and expose the Ollama API and OpenWeb UI via NGINX, which provided a chat GUI to interact with Ollama. In contrast to Ollama, which runs directly on the host, the other services are run in containers managed by rootles podman and defined the docker-compose.yml 56. The NGINX default.conf 62 configures the web server to terminate the SSL connection and act as reverse proxy for the "Open WebUI" and the Ollama API under the directory /llm/.

Services:

- Ollama: LLM manager, exposing an API for prompts and other interactions (host).
- Swag: NGINX web server with Certbot for automated SSL certificate generation (containerized).
- Open WebUI: Web based chat interface for LLM managers (containerized).

F.3. System Hardening

It is recommended to put a proper firewall before the services we want to expose to the internet, but for our purposes we can just disable the port forwarding when we don't use the AI. It still makes sense to perform a minimal system hardening as shown in the following listing 52, before exposing the services to the internet.

```
# Install UFW
sudo pacman -Syu ufw
sudo ufw status

# Allow SSH for remote access
sudo ufw allow ssh

# Allow HTTP for certbot HTTP challange
sudo ufw allow http

# Allow HTTPS for Open WebUI
sudo ufw allow https

# Allow default Ollama port
sudo ufw allow 11434/tcp

# Make sure to run this command after adding the "allow" rules!
sudo ufw enable

sudo ufw status
```

Listing 52: UFW configuration

F.4. Expose Open WebUI and Ollama

For exposing the services from the home network via HTTPS to the internet, we need a domain name which points to the home router. To achieve this we can use DynDNS, which is provided for free by Infomaniak where the domain "hiube.ch" was registered, and is supported by the FRITZ!Box used in the home network. The screenshots F.2 and F.1 are showing the respective configurations.

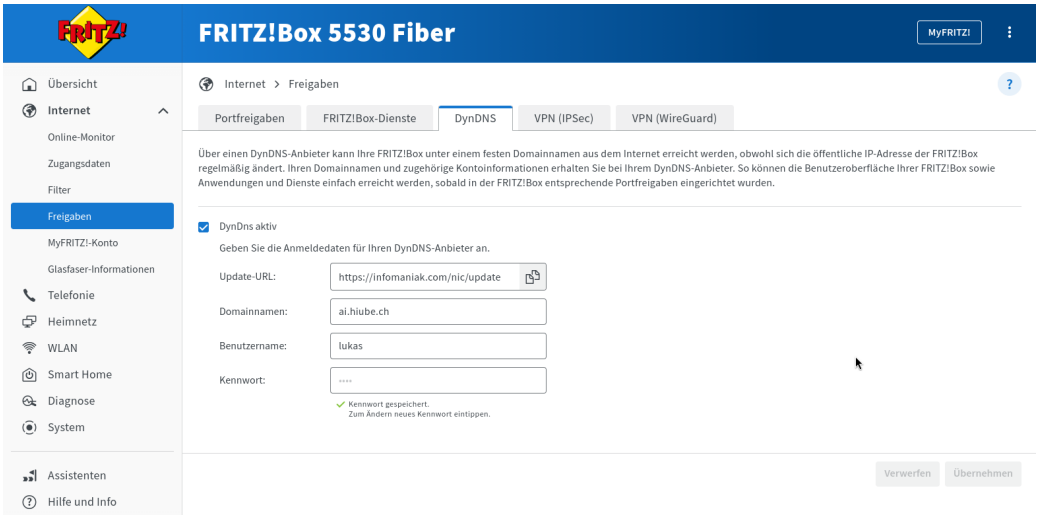


Figure F.1.: FRITZ!Box DynDNS Setup

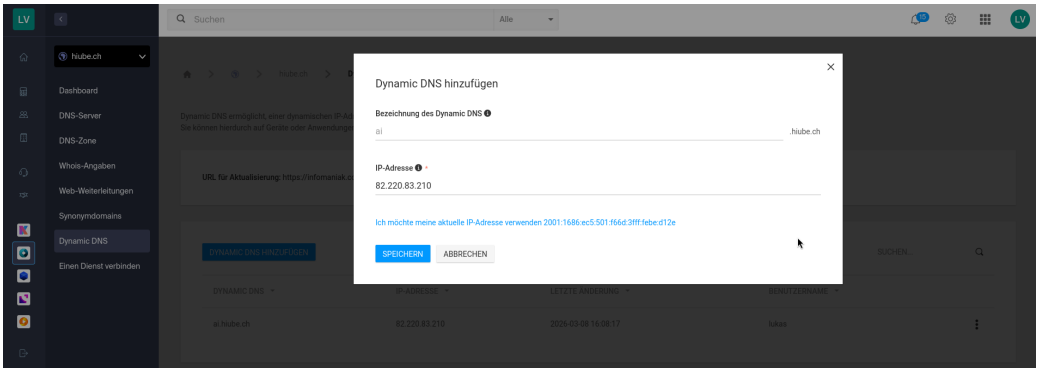


Figure F.2.: Infomaniak DynDNS Setup

Now that the domain name “ai.hiube.ch” points to the router in the home network, we need to redirect the HTTP(S) traffic to the LLM server. This can be achieved with port forwarding rules on the FRITZ!Box, as shown in the screenshot F.3.

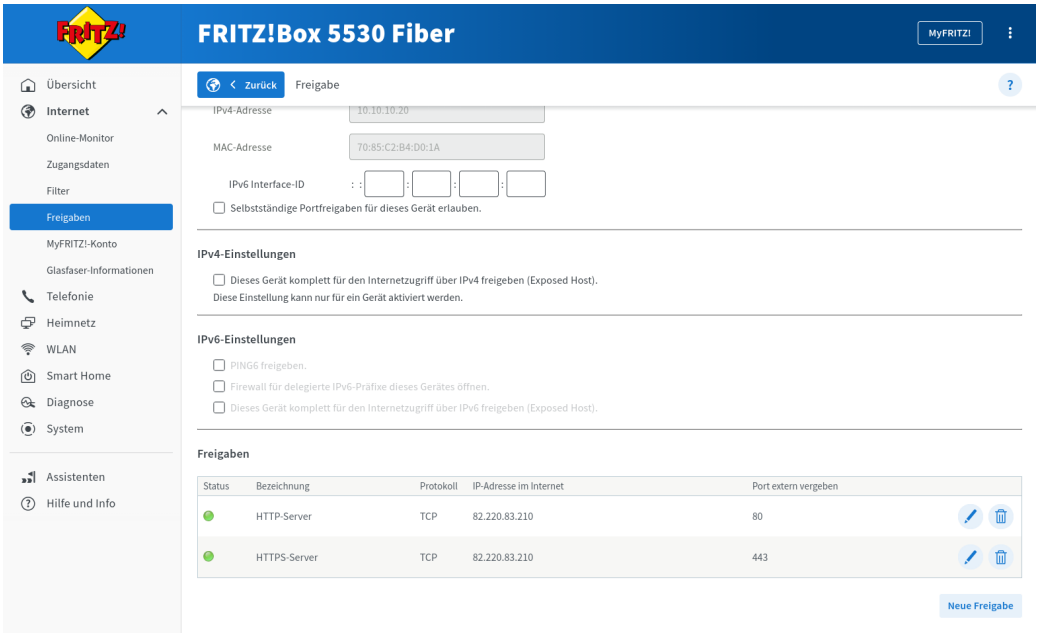


Figure F.3.: FRITZ!Box DynDNS Setup

The Open WebUI and Ollama API should already be reachable via “ai.hiube.ch”, but we will receive a security warning because there is no valid SSL certificate. The easiest way to generate the certificate, is to trigger certbot by restarting the “swag” service as shown in the listing below 53.

```
1 # Login as user ai and restart the swag service
2 podman-compose restart swag
```

Listing 53: Restart SWAG Container

F.5. Evaluation

For quickly evaluate models and assess their capabilities we can now use the Open WebUI served under “ai.hiube.ch”. The screenshot F.4 shows a quick test with “qwen2.5-coder:7B”.

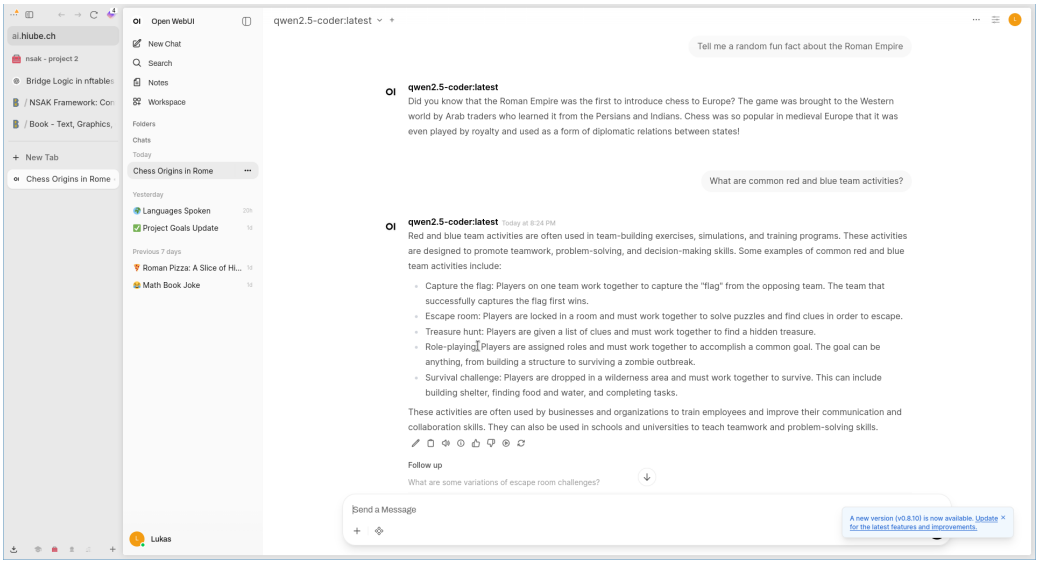


Figure F.4.: FRITZ!Box DynDNS Setup

Later on we can use the Ollama API served under /llm/ as a backend for AI agents.

F.5.1. Self-Hosted Base Setup

```
# Login as ai user
ssh ai@10.10.10.20

# Create a docker compose file and add contents from lst:nginx-docker-compose-yml
vim docker-compose.yml

# Start the services, so that they create the volume mappings etc.
# The containers will most likely fail, but that's fine for now.
podman-compose up -d
podman-compose down
```

Listing 54: Self-Hosted Base Setup

```
# Login as ai user
ssh ai@10.10.10.20

# Create a docker compose file and add contents from lst:nginx-docker-compose-yml
vim docker-compose.yml

# Start the services, so that they create the volume mappings etc.
# The containers will most likely fail, but that's fine for now.
podman-compose up -d
podman-compose down
```

Listing 55: Nginx & Let's Encrypt Instructions

F.5.2. Nginx & Let's Encrypt - Reverse Proxy with SSL Termination

```yaml
services:
  swag:
    image: lscr.io/linuxserver/swag:latest
    container_name: swag
    cap_add:
      - NET_ADMIN
    environment:
      - PUID=1000
      - PGID=1000
      - TZ=Europe/Zurich
      - URL=hiube.ch
      - VALIDATION=http
      - SUBDOMAINS=ai,
      - EMAIL=<my-email>
      - ONLY_SUBDOMAINS=true
      - EXTRA_DOMAINS= #optional
      - STAGING=false #optional
      - DISABLE_F2B= #optional
      - SWAG_AUTORELOAD= #optional
      - SWAG_AUTORELOAD_WATCHLIST= #optional
    volumes:
      - /home/ai/swag:/config
    ports:
      - 443:443
      - 80:80
      - 443:443/udp
    restart: unless-stopped
```

Listing 56: Nginx & Let's Encrypt Compose File

F.5.3. Ollama - LLM Server

```
ssh ai@10.10.10.20

# Check nvidia driver is loaded and the GPU was correctly detected, otherwise install the according drivers
nvidia-smi

# Install Ollama and enable systemd service
sudo pacman -Syu ollama
sudo systemctl enable --now ollama
sudo systemctl status ollama

# Download and run a qwen coder 7b model for testing
# Later used models:
# - `qwen2.5:7b-instruct` (tool calling)
# - `gemma4:e2b` (was hyped, but is not that good in tool calling)
ollama pull qwen2.5-coder:7b
ollama run qwen2.5-coder

# Open the nginx defalult.conf and add the contents from lst:ollama-nginx-conf
vim swag/nginx/site-confs/default.conf

# Start the services, this time they should run successfully.
# For a local setup, certbot will fail to perform the http challenge for issuing the TLS certificate,
# as long as DynDNS and the port forwarding are not set up properly.
podman-compose up -d
```

Listing 57: Ollama Instructions

```
# Redirect all traffic to HTTPS
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    server_name ai.hiube.ch;

    location / {
        return 301 https://$host$request_uri;
    }
}

# Using upstream enables us to restart ollama without restarting the nginx
upstream ollama {
    server host.docker.internal:11434;
}

server {
    listen 43434 ssl;
    listen [::]:43434 ssl;

    server_name ai.hiube.ch;

    include /config/nginx/ssl.conf;

    location / {
        auth_basic "Restricted";
        auth_basic_user_file /config/nginx/.htpasswd;

        proxy_pass http://ollama;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        chunked_transfer_encoding off;
        proxy_buffering off;
        proxy_cache off;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_read_timeout 600s;
        proxy_send_timeout 600s;
    }
}
```

Listing 58: Ollama Nginx Configuration

```
# Enable Ollama debug logs

During the development of the AI intrusion detection scenario, Ollama suddenly sent an empty response terminating the agent.
For investigation, I enabled the debug logs on the AI server.

```bash
sudo systemctl edit --full ollama.service

# Add the following line
Environment="OLLAMA_DEBUG=2"

# Restart the service
sudo systemctl restart ollama.service

# The logs can now be listed with:
journalctl -f -b -u ollama
```

Container logs of the empty Ollama response:

-> The last log indicates a container restart (scenario was terminated by empty response)

```bash
❯ sudo podman logs -f simple_tcp_client_server_ai_intrusion_detection_1
WARNING:nsak.core.scenario.scenario_manager:EXEC SCENARIO: AI Intrusion Detection
INFO:root:Starting scenario: AI Intrusion Detection
WARNING:nsak.core.drill.drill_manager:EXEC DRILL: Discover Hosts
DEBUG:httpcore.connection:connect_tcp.started host='ai.hiube.ch' port=43434 local_address=None timeout=None socket_options=None
DEBUG:httpcore.connection:connect_tcp.complete return_value=<httpcore._backends.sync.SyncStream object at 0x7f6434e86f90>
DEBUG:httpcore.connection:start_tls.started ssl_context=<ssl.SSLContext object at 0x7f6434eff4d0> server_hostname='ai.hiube.ch' timeout=None
DEBUG:httpcore.connection:start_tls.complete return_value=<httpcore._backends.sync.SyncStream object at 0x7f6434eab110>
DEBUG:httpcore.http11:send_request_headers.started request=<Request [b'POST']>
DEBUG:httpcore.http11:send_request_headers.complete
DEBUG:httpcore.http11:send_request_body.started request=<Request [b'POST']>
DEBUG:httpcore.http11:send_request_body.complete
DEBUG:httpcore.http11:receive_response_headers.started request=<Request [b'POST']>
DEBUG:httpcore.http11:receive_response_headers.complete return_value=(b'HTTP/1.1', 200, b'OK', [(b'Server', b'nginx'), (b'Date', b'Mon, 06 Apr 2026 07:22:02 GMT'), (b'Content-Type', b'application/x-ndjson'), (b'Connection', b'close')])
INFO:httpx:HTTP Request: POST https://ai.hiube.ch:43434/api/chat "HTTP/1.1 200 OK"
DEBUG:httpcore.http11:receive_response_body.started request=<Request [b'POST']>
DEBUG:httpcore.http11:receive_response_body.complete
DEBUG:httpcore.http11:response_closed.started
DEBUG:httpcore.http11:response_closed.complete
WARNING:root:[Human]

You are analyzing network traffic for intrusion detection on interface eth1.

Network Discovery Result Map:
| Interface   | MAC               | IP           | Port   | Protocol   | State   | Service   | Product   | Version   |
|-------------|-------------------|--------------|--------|------------|---------|-----------|-----------|-----------|
| eth1        | ea:35:69:e3:1d:1a | 10.10.100.10 |        |            |         |           |           |           |
| eth1        | 72:31:39:c6:67:e6 | 10.10.100.20 |        |            |         |           |           |           |
| eth1        | f2:f7:61:1a:62:f5 | 10.10.100.30 |        |            |         |           |           |           |
| eth1        | f2:f7:61:1a:62:f5 | 10.10.100.20 |        |            |         |           |           |           |
| eth1        | f2:f7:61:1a:62:f5 | 10.10.100.10 |        |            |         |           |           |           |

Captured packets (CSV):
frame.number,eth.src,eth.dst,ip.src,ip.dst,tcp.dstport,udp.dstport,frame.protocols,frame.len
1,f2:f7:61:1a:62:f5,42:0a:f1:34:e7:5b,,,,,eth:ethertype:arp,42
2,f2:f7:61:1a:62:f5,42:0a:f1:34:e7:5b,,,,,eth:ethertype:arp,42
3,42:0a:f1:34:e7:5b,33:33:00:00:00:02,,,,,eth:ethertype:ipv6:icmpv6,70
4,72:31:39:c6:67:e6,33:33:00:00:00:02,,,,,eth:ethertype:ipv6:icmpv6,70
5,f2:f7:61:1a:62:f5,33:33:00:00:00:02,,,,,eth:ethertype:ipv6:icmpv6,70
6,ea:35:69:e3:1d:1a,33:33:00:00:00:02,,,,,eth:ethertype:ipv6:icmpv6,70
7,f2:f7:61:1a:62:f5,42:0a:f1:34:e7:5b,,,,,eth:ethertype:arp,42
8,f2:f7:61:1a:62:f5,42:0a:f1:34:e7:5b,,,,,eth:ethertype:arp,42
9,f2:f7:61:1a:62:f5,42:0a:f1:34:e7:5b,,,,,eth:ethertype:arp,42
10,f2:f7:61:1a:62:f5,42:0a:f1:34:e7:5b,,,,,eth:ethertype:arp,42
11,f2:f7:61:1a:62:f5,42:0a:f1:34:e7:5b,,,,,eth:ethertype:arp,42
12,f2:f7:61:1a:62:f5,42:0a:f1:34:e7:5b,,,,,eth:ethertype:arp,42
13,42:0a:f1:34:e7:5b,33:33:00:00:00:02,,,,,eth:ethertype:ipv6:icmpv6,70
14,72:31:39:c6:67:e6,33:33:00:00:00:02,,,,,eth:ethertype:ipv6:icmpv6,70
15,f2:f7:61:1a:62:f5,42:0a:f1:34:e7:5b,,,,,eth:ethertype:arp,42
16,f2:f7:61:1a:62:f5,42:0a:f1:34:e7:5b,,,,,eth:ethertype:arp,42
17,f2:f7:61:1a:62:f5,33:33:00:00:00:02,,,,,eth:ethertype:ipv6:icmpv6,70
18,f2:f7:61:1a:62:f5,42:0a:f1:34:e7:5b,,,,,eth:ethertype:arp,42
19,f2:f7:61:1a:62:f5,42:0a:f1:34:e7:5b,,,,,eth:ethertype:arp,42
20,ea:35:69:e3:1d:1a,33:33:00:00:00:02,,,,,eth:ethertype:ipv6:icmpv6,70
21,f2:f7:61:1a:62:f5,42:0a:f1:34:e7:5b,,,,,eth:ethertype:arp,42
22,f2:f7:61:1a:62:f5,42:0a:f1:34:e7:5b,,,,,eth:ethertype:arp,42
```
```

F.5.4. Open WebUI - Web Chat for LLM Servers

```
ssh ai@10.10.10.20

# Open the nginx defalult.conf and add the contents from lst:open-webui-nginx-conf
vim swag/nginx/site-confs/default.conf

# Start the services
podman-compose up -d
```

Listing 60: Open WebUI Instructions

```yaml
services:
  # ...

  open-webui:
    image: ghcr.io/open-webui/open-webui:main
    container_name: open-webui
    volumes:
      - /home/ai/open_webui:/app/backend/data # Use a named volume for persistence
    environment:
      OLLAMA_BASE_URL: http://host.docker.internal:11434
    restart: always
```

Listing 61: Open WebUI Compose File

```
# Open WebUI
upstream open_webui {
    server open-webui:8080;
}

server {
    listen 443 ssl default_server;
    listen [::]:443 ssl default_server;

    server_name ai.hiube.ch;

    include /config/nginx/ssl.conf;

    location / {
        proxy_pass http://open_webui;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_read_timeout 900s;
        proxy_send_timeout 900s;
    }
}
```

Listing 62: Open WebUI Nginx Configuration

F.5.5. Drawio MCP - Diagram Tool

```
ssh ai@10.10.10.20

# https://www.drawio.com/blog/diagrams-docker-app
echo "unqualified-search-registries = [\"docker.io\"]" >> /etc/containers/registries.conf

# Add drawio.hiube.ch CNAME entry to ai.hiube.ch
# Add drawio.hiube.ch to linuxserver.io swag config for letsencrypt and add the drawio service from below
vim docker-compose.yaml

# Update nginx conifg
sudo vim swag/nginx/site-confs/default.conf

# Download drawio-mcp
git clone https://github.com/lgazo/drawio-mcp-server.git

# Restart services
podman-compose restart swag
```

Listing 63: Drawio Instructions

```yaml
services:
  # ...

  drawio-mcp-server:
    build: ./drawio-mcp-server
    container_name: drawio-mcp-server
    restart: unless-stopped
    command:
      - "node"
      - "packages/drawio-mcp-server/build/index.js"
      - "--transport"
      - "http"
      - "--editor"
      - "--http-port"
      - "3000"
```

Listing 64: Drawio Compose File

```
# Draw.io
upstream drawio {
    server drawio:8080;
}

upstream drawio_mcp {
    server drawio-mcp-server:3000;
}

upstream drawio_ws {
    server drawio-mcp-server:3333;
}

server {
    listen 443 ssl;
    listen [::]:443 ssl;

    server_name drawio.hiube.ch;

    include /config/nginx/ssl.conf;

    location / {
        auth_basic "Restricted";
        auth_basic_user_file /config/nginx/.htpasswd;

        proxy_pass http://drawio;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }

    location /mcp {
        auth_basic "Restricted";
        auth_basic_user_file /config/nginx/.htpasswd;

        proxy_pass http://drawio_mcp/mcp;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_buffering off;
        proxy_cache off;
        proxy_read_timeout 3600s;
        proxy_send_timeout 3600s;
        chunked_transfer_encoding on;
        proxy_set_header X-Accel-Buffering no;
    }

    location /ws {
        auth_basic "Restricted";
        auth_basic_user_file /config/nginx/.htpasswd;

        proxy_pass http://drawio_ws;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_read_timeout 3600s;
        proxy_send_timeout 3600s;
    }
}
```

Listing 65: Drawio Nginx Configuration

F.5.6. Netbox - Network Inventory System

```
ssh ai@10.10.10.20

# https://learn.srlinux.dev/blog/2024/creating-a-network-digital-twin-with-containerlab-and-netbox/
# https://github.com/netbox-community/netbox-docker
git clone -b release https://github.com/netbox-community/netbox-docker.git
cd netbox-docker/

cp docker-compose.override.yml.example docker-compose.override.yml

podman-compose up -d

podman-compose exec netbox /opt/netbox/netbox/manage.py createsuperuser

cd ..

# Add netbox.hiube.ch CNAME entry to ai.hiube.ch
# Add netbox.hiube.ch to linuxserver.io swag config for letsencrypt
vim docker-compose.yaml

# Update nginx conifg
sudo vim swag/nginx/site-confs/default.conf

# Restart services
podman-compose restart swag
```

Listing 66: Netbox Instructions

```
1
```

Listing 67: Netbox Compose File

```
# Netbox
upstream netbox {
    server host.docker.internal:8000;
}

server {
    listen 443 ssl;
    listen [::]:443 ssl;

    server_name netbox.hiube.ch;

    include /config/nginx/ssl.conf;

    location / {
        proxy_pass http://netbox;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Listing 68: Netbox Nginx Configuration

F.5.7. Grafana / Loki - Logging System

```
ssh ai@10.10.10.20

# https://grafana.com/docs/loki/latest/setup/install/docker/#install-with-docker-compose
mkdir loki
wget https://raw.githubusercontent.com/grafana/loki/v3.7.0/examples/getting-started/docker-compose.yaml -O
↪  loki/docker-compose.yaml
wget https://raw.githubusercontent.com/grafana/loki/v3.7.0/examples/getting-started/alloy-local-config.yaml
↪  -O loki/alloy-local-config.yaml
wget https://raw.githubusercontent.com/grafana/loki/v3.7.0/examples/getting-started/loki-config.yaml -O
↪  loki/loki-config.yaml

# Adjustments needed for rootles podman
systemctl --user enable --now podman.socket

# Adjust the alloy service described in the loki `compose.yaml`
vim loki/docker-compose.yaml

# Add grafana.hiube.ch CNAME entry to ai.hiube.ch
# Add grafana.hiube.ch to linuxserver.io swag config for letsencrypt
vim docker-compose.yaml

# Update nginx conifg
sudo vim swag/nginx/site-confs/default.conf

# Restart services
podman-compose up -d
```

Listing 69: Grafana / Loki Instructions

```yaml
# ~/loki/docker-compose.yaml
networks:
  loki:

services:
  read:
    image: grafana/loki:latest
    command: "-config.file=/etc/loki/config.yaml -target=read"
    ports:
      - 127.0.0.1:3101:3100
      - 7946
      - 9095
    volumes:
      - ./loki/loki-config.yaml:/etc/loki/config.yaml
    depends_on:
      - minio
    healthcheck:
      test: [ "CMD", "/usr/bin/loki", "-health" ]
      start_period: 30s
      interval: 10s
      timeout: 5s
      retries: 5
    networks: &loki-dns
      loki:
        aliases:
          - loki

  write:
    image: grafana/loki:latest
    command: "-config.file=/etc/loki/config.yaml -target=write"
    ports:
      - 127.0.0.1:3102:3100
      - 7946
      - 9095
    volumes:
      - ./loki/loki-config.yaml:/etc/loki/config.yaml
    healthcheck:
      test: [ "CMD", "/usr/bin/loki", "-health" ]
      start_period: 30s
      interval: 10s
      timeout: 5s
      retries: 5
    depends_on:
      - minio
    networks:
      <<: *loki-dns

  alloy:
    image: grafana/alloy:latest
    volumes:
      - ./loki/alloy-local-config.yaml:/etc/alloy/config.alloy:ro
      - /run/user/1001/podman/podman.sock:/var/run/docker.sock
    command:  run --server.http.listen-addr=0.0.0.0:12345 --storage.path=/var/lib/alloy/data
    ↪  /etc/alloy/config.alloy
    ports:
      - 127.0.0.1:12345:12345
    depends_on:
      - swag
    networks:
      - loki

  minio:
    image: minio/minio
    entrypoint:
      - sh
      - -euc
      - |
        mkdir -p /data/loki-data && \
        mkdir -p /data/loki-ruler && \
        minio server /data
    environment:
      - MINIO_ROOT_USER=loki
      - MINIO_ROOT_PASSWORD=supersecret
      - MINIO_PROMETHEUS_AUTH_TYPE=public
      - MINIO_UPDATE=off
    ports:
      - 9000
    volumes:
      - ./loki/.data/minio:/data
    healthcheck:
      test: [ "CMD", "curl", "-f", "http://localhost:9000/minio/health/live" ]
      interval: 15s
      timeout: 20s
      retries: 5
    networks:
      - loki

  grafana:
    image: grafana/grafana:latest
    environment:
      - GF_SERVER_ROOT_URL=https://grafana.hiube.ch
      - GF_SERVER_SERVE_FROM_SUB_PATH=false
      - GF_PATHS_PROVISIONING=/etc/grafana/provisioning
```

```
# Grafana / Loki
upstream grafana {
    server grafana:3000;
}
upstream loki-read {
    server read:3100;
}
upstream loki-write {
    server write:3100;
}

server {
    listen 443 ssl;
    listen [::]:443 ssl;

    server_name grafana.hiube.ch;

    include /config/nginx/ssl.conf;

    auth_basic "Restricted";
    auth_basic_user_file /config/nginx/.htpasswd;

    location / {
        auth_basic off;

        proxy_pass http://grafana;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
        proxy_read_timeout 900s;
        proxy_send_timeout 900s;
    }

    location = /api/prom/push {
        proxy_pass       http://loki-write$request_uri;
    }

    location = /api/prom/tail {
        proxy_pass       http://loki-read$request_uri;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
    }

    location ~ /api/prom/.* {
        proxy_pass       http://loki-read$request_uri;
    }

    location = /loki/api/v1/push {
        proxy_pass       http://loki-write$request_uri;
    }

    location = /loki/api/v1/tail {
        proxy_pass       http://loki-read$request_uri;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
    }

    location ~ /loki/api/.* {
        proxy_pass       http://loki-read$request_uri;
    }
}
```

Listing 71: Grafana / Loki Nginx Configuration

```yaml
# ~/loki/loki-config.yaml
auth_enabled: false

server:
  http_listen_address: 0.0.0.0
  http_listen_port: 3100

memberlist:
  join_members: ["read", "write", "backend"]
  dead_node_reclaim_time: 30s
  gossip_to_dead_nodes_time: 15s
  left_ingesters_timeout: 30s
  bind_addr: ['0.0.0.0']
  bind_port: 7946
  gossip_interval: 2s

schema_config:
  configs:
    - from: 2023-01-01
      store: tsdb
      object_store: s3
      schema: v13
      index:
        prefix: index_
        period: 24h
common:
  path_prefix: /loki
  replication_factor: 1
  compactor_address: http://backend:3100
  storage:
    s3:
      endpoint: minio:9000
      insecure: true
      bucketnames: loki-data
      access_key_id: loki
      secret_access_key: supersecret
      s3forcepathstyle: true
  ring:
    kvstore:
      store: memberlist
ruler:
  storage:
    s3:
      bucketnames: loki-ruler

compactor:
  working_directory: /tmp/compactor
  retention_enabled: true
  delete_request_store: s3

limits_config:
  retention_period: 8760h # 1 year
  retention_stream:
    - selector: '{job="container_logs"}'
      period: 168h  # 7 days
```

Listing 72: Grafana / Loki Configuration

```yaml
# ~/loki/loki-config.yaml
auth_enabled: false

server:
  http_listen_address: 0.0.0.0
  http_listen_port: 3100

memberlist:
  join_members: ["read", "write", "backend"]
  dead_node_reclaim_time: 30s
  gossip_to_dead_nodes_time: 15s
  left_ingesters_timeout: 30s
  bind_addr: ['0.0.0.0']
  bind_port: 7946
  gossip_interval: 2s

schema_config:
  configs:
    - from: 2023-01-01
      store: tsdb
      object_store: s3
      schema: v13
      index:
        prefix: index_
        period: 24h
common:
  path_prefix: /loki
  replication_factor: 1
  compactor_address: http://backend:3100
  storage:
    s3:
      endpoint: minio:9000
      insecure: true
      bucketnames: loki-data
      access_key_id: loki
      secret_access_key: supersecret
      s3forcepathstyle: true
  ring:
    kvstore:
      store: memberlist
ruler:
  storage:
    s3:
      bucketnames: loki-ruler

compactor:
  working_directory: /tmp/compactor
  retention_enabled: true
  delete_request_store: s3

limits_config:
  retention_period: 8760h # 1 year
  retention_stream:
    - selector: '{job="container_logs"}'
      period: 168h  # 7 days
```

Listing 73: Grafana / Loki Alloy Configuration

G. Media Deliverables

G.1. Poster

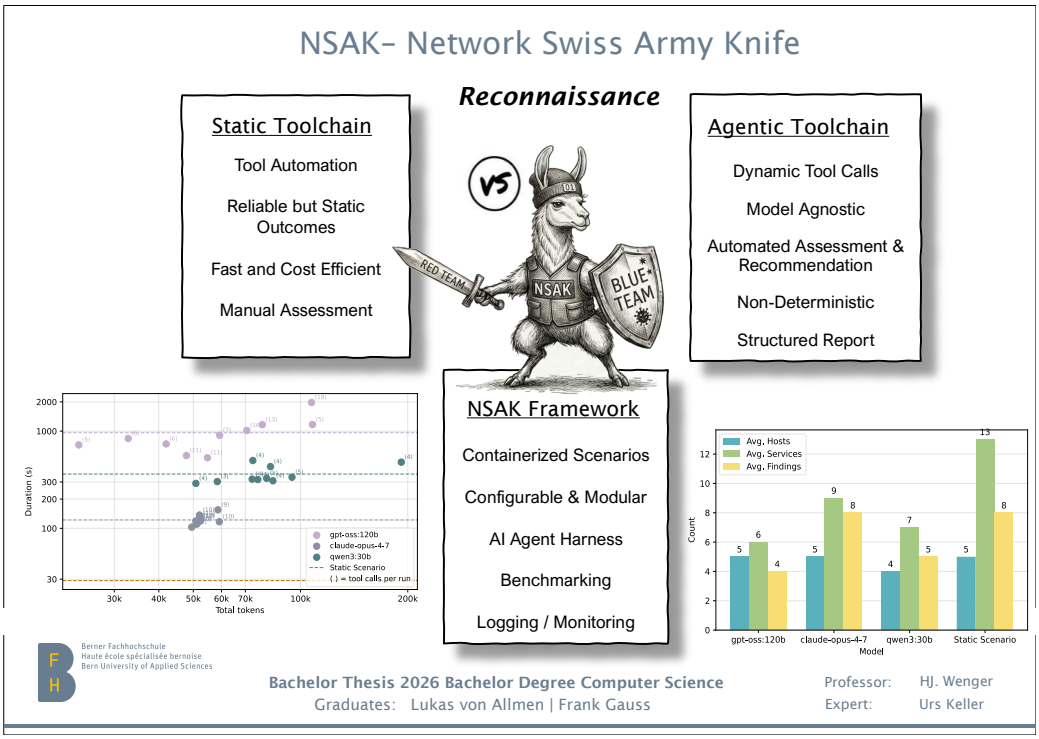


Figure G.1.: Media Deliverable: Poster

G.2. Book

Nsak as a Framework for Agentic-AI enhanced Network Security

Degree programme: BSc in Computer Science
Specialisation: IT Security
Thesis advisor: Prof. Hansjürg Wenger
Expert: Urs Keller

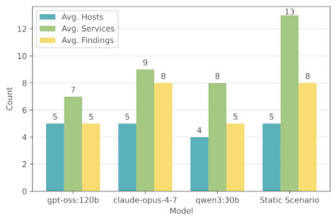
This thesis investigates whether Agentic-AI can effectively automate and enhance network security scenarios within the NSAK framework, a modular Python-based cybersecurity orchestration platform for purple teams.

Introduction

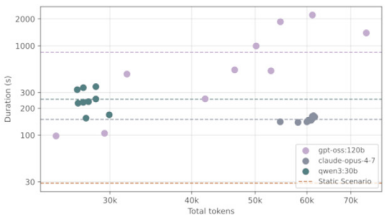
Red and blue team exercises are established cybersecurity practices, yet they remain largely inaccessible to small- and medium-sized organizations due to the expertise and recurring effort these practices demand. We present an approach that combines **LLMs** with existing network security tools to reduce knowledge barriers and time investment. Concretely, we integrated an **AI agent** with tool-calling capabilities into the **NSAK framework** - a network security test automation framework established in our preceding work - and developed an agentic network reconnaissance scenario. We evaluated its effectiveness across models of varying size, benchmarking it against a scripted scenario in a reproducible network environment.

Technical Specification

The agent is implemented using **LangChain,** an open-source agent framework. Three models are benchmarked across n runs in a virtual container lab and the BFH network security lab: a frontier model, claude-opus-4-7; an institutionally hosted open-weight model, gpt-oss:120b; and a locally hosted open-source model, qwen3:30b, via Ollama. The evaluation further compares **single-** and **multi-agent** configurations, including the use of **structured output** to ensure consistent and parsable responses.



Benchmark results - Multi Agent Scenario: Findings

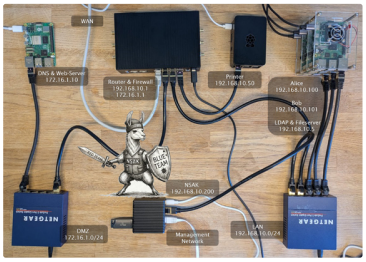


Benchmark results - Multi Agent Scenario: Tokens vs. Duration

Conclusion

The results show that even smaller models can autonomously perform standard red team activities. The **AI-driven approach** demonstrates a clear advantage over scripted counterparts. The agent assesses results, generates reports with prioritized findings, and classifies steps to address vulnerabilities. This work highlights the emerging capabilities of LLMs, making cybersecurity use cases more accessible and cost-effective.

Index Terms:
AI-Driven Cybersecurity, Large Language Models, Network Security Automation Framework



Physical-Environment

Frank Erich Gauss

Lukas Christian von Allmen

Figure G.2.: Media Deliverable: Book

G.3. Video

Add a link to the video or describe where it can be found

G.4. Slides: Public Presentation

Add the slides of the public presentation

G.5. Slides: Defense Presentation

Add the slides of the defense presentation